# 前有数据结构

Data Structures Ahead

CoreLoop

# UNKNOWN VOICE

是的，没错，这片大陆叫做《数据结构》，是历代先贤们以智慧为剑刃开辟的疆土，也是无数求知者在 0 与 1 的荒原上寻找真理的朝圣之地。寻理者皆向北而行，并了解了古老律法的真相。

位不见王影，然火焰映彻苍穹。当传承火焰燃烧之时，钟声将响彻四周，进而唤醒遍布这片大陆的思渡者们。

时秩双王 — 栈和队列，分界统御者 — 树与二叉树，以及万象之都的孤独王者 — 图。不过啊，王者们一定会筑起不可逾越的高墙吧。而无火的余灰将会纷至踏来，那是无名、成不了薪，被命运牵绊的思渡者。但也正因如此，灰烬才会如此渴求余火吧。

# — BONFIRE DISCOVERED —

欢迎来到营火，无火的余灰大人。我正是维火女，专门维护营火以及服侍您的人。

正如您所见，穿过山隘，前方就是数据结构大陆中不显眼的一角——《前有数据结构》，这片大陆的开创者，大家称其为 CoreLoop。如果您为了觐见**树、图**等王者们而需要帮助，我很愿意为此帮忙。

首先，CoreLoop 托我带给您以下谏言，愿它们能照亮您前行的路。

1. 《前有数据结构》假设您已经有了比较好的 C 语言基础，但他也会在某些合适的地方强调某些 C 语言的知识点。并且他会在非常合适的地方引入在编程题、上机、算法竞赛中最重要、最有用的 C++ 内容，保证不带来理解负担的前提下让大家不仅能学会知识，更能知道怎么用、什么时候用。

2. 在您游渡的过程中，遇到的所有“思维拓展”**都！可！以！不！必！停！留！！**我向您保证，即使跳过这些，您也决不会错过任何核心内容。您可根据适合自己的方式选择是否驻足。

3. 如果在您游渡的过程中有任何疑问、建议、需求、意见、请求等，

   欢迎通过【CoreLoop@163.com】联系他本人。

继续向前，便是 CoreLoop 所开创的大陆。在旅途中，我将为您提供必要的指引。**若我的出现偶尔成了您的负累，还请您不必驻足与我的谏言。我并非大陆的探拓者，因此这不会让您错过任何数据结构大陆的美丽风景**。无论如何，我都将不离不弃地伴您同行，直至旅途的终点。

进入大陆前，请您收下这份地图。

# 地 图


**序章 C 和结构体指针、typedef** **1**

指针 . . . . . 2

结构体与指针 . . . . . 4

结构体与指针、typedef . . . . . 6

**第一章 数据结构** **11**

1.1 数据结构 . . . . . 11

1.2 **笔者的谏言** . . . . . 13

**第二章 线性表** **15**

2.1 顺序表和链表简介 . . . . . 16

2.2 顺序表 . . . . . 18

**第三章 栈和队列** **23**

3.1 栈 . . . . . 24

3.1.1 顺序栈的常见操作实现 . . . . . 26

3.1.2 链栈的常见操作实现 . . . . . 27

3.1.3 * 浏览器的前进后退功能实现 . . . . . 28

3.1.4 *“函数栈”、“调用栈”是啥？ . . . . . 30

3.2 队列 . . . . . 30

3.2.1 队列的链式存储 . . . . . 30

3.2.2 队列的顺序存储 . . . . . 33

3.3 栈的进一步理解 . . . . . 38

3.3.1 表达式的转换与求值 . . . . . 41

习题 1 . . . . . 42

3.4 本章小结 . . . . . 48

**第四章 树与二叉树** **49**

4.1 哈夫曼树与哈夫曼编码 . . . . . 49

4.1.1 哈夫曼编码 . . . 52
4.2 堆 . . . 55
4.3 并查集 . . . 55
4.3.1 路径压缩 . . . 58
4.3.2 * 按秩合并 . . . 59
习题 2 . . . 61

**第五章 图** **63**
5.1 图 . . . 64
5.2 图的存储 . . . 66
5.2.1 邻接矩阵 . . . 66
5.2.2 邻接表 . . . 67
5.2.3 二者获取邻接信息的对比 . . . 70
5.3 图的遍历 . . . 72
5.3.1 深度优先搜索（DFS） . . . 72
5.3.2 广度优先搜索（BFS） . . . 73
5.3.3 DFS 和 BFS 的实现 . . . 75
习题 3 . . . 81
5.4 最短路径 . . . 88
5.4.1 迪杰斯特拉算法 . . . 88
5.4.2 Bellman-Ford 算法 . . . 100
5.4.3 Floyd 算法 . . . 105
习题 4 . . . 107
5.5 最小生成树 . . . 110
5.5.1 Prim（普里姆）算法 . . . 110
5.5.2 Kruskal（克鲁斯卡尔）算法 . . . 112
5.5.3 算法实现 . . . 113
5.6 AOV 网与拓扑排序 . . . 116
5.7 AOE 网与关键路径 . . . 121
5.7.1 关键路径的求解 . . . 121
习题 5 . . . 128

**第六章 排序** **133**

**习题参考答案** **135**

# 序章　C 和结构体指针、typedef

如果您已对下面这段游戏演示程序中的所有语法非常熟练，请直接跳转至—waypoint-1—（第 11 页）处，否则，建议您根据适合自己的速度阅览本章。

```c
// 定义武器类型 Weapon
struct Weapon {
    int attack_power;   // 该武器为角色提供的攻击力
    // ...
};
typedef struct Weapon* pWeapon;
// 定义角色类型
typedef struct {
    int max_hp;         // 角色的最大血量
    int current_hp;     // 角色的当前血量
    struct Weapon* lhand_weapon;    // 左手武器
    pWeapon rhand_weapon;    // 右手武器
} Role, *pRole;

// 定义一把大剑
struct Weapon great_sword;
great_sword.attack_power = 20;
// 定义一把十字镐
struct Weapon pickaxe;
pickaxe.attack_power = 15;

// 定义两个角色，并将他们的最大 hp 与当前 hp 均初始化为 100
Role role1, role2;
role1.current_hp = 100; role1.max_hp = 100;
role2.current_hp = 100; role2.max_hp = 100;
// 为两个角色装备左手武器
role1.lhand_weapon = &great_sword;
role2.lhand_weapon = &pickaxe;

// role1 用左手武器打了 role2 一下
pWeapon role1_lhand_weapon = role1.lhand_weapon;
role2.current_hp -= (*role1_lhand_weapon).attack_power;
if(role2.current_hp <= 0) {
```

```
    // 死亡处理
}

// role2 用左手武器打了 role1 一下
role1.current_hp -= role2.lhand_weapon->attack_power;
if(role1.current_hp <= 0) {
    // 死亡处理
}
```

## 指针

**指针变量**用于存储某个内存地址。可以通过“*”运算符来定义指针变量、访问数据：

```
int a = 10;
int *p = &a;    // 定义指针变量，并赋初始值
printf("%d\n", *p);     // 输出：10
```

我们一行一行来看。首先定义了一个变量 a，假设操作系统为其分配的内存地址为 1024，如图 1 右下角所示，然后：

```
int *p = &a;    // & 是取地址运算符
```

这行代码定义了一个指向 int 的指针变量 p，并将 a 的内存地址，即 1024，赋值给了 p。此时变量 p 的值是 1024，而变量 p 的地址是操作系统分配的，此处我们不关心。

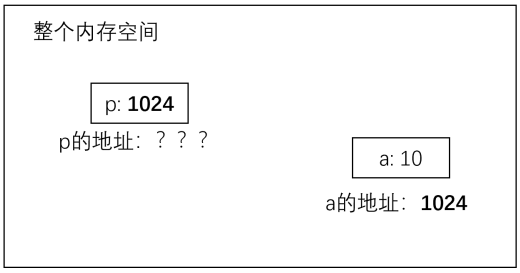



图 1 指针示意图

之后，

```
printf("%d\n", *p);     // 输出：10
```

*p 访问的并不是 p 这个变量的值，而是将 p 的值作为内存地址，访问该地址中的变量的值。

请注意，以下代码中，第二行代码和第三行代码中的“*”运算符有着不同的语义。

```
int a = 10;
int *p = &a;    // 定义指针变量，并赋初始值
printf("\%d\n", *p);     // 输出：10
```

第三行代码中“*”的功能是访问内存地址 p 中的数据，“*”的作用是**解引用**，意思是解开 p 对该内存地址的引用，露出该地址中的数据；而第二行代码中的“*”运算符则是作为**声明符**，表示“接下来声明的变量是一个指针变量”。

类似地，以下代码中的“*”是解引用的功能：

```
*p = 10;
int b = *p;
```

以下代码中的“*”被作为声明符：

```
int *p2;    // 这是在声明变量
p2 = &b;    // 为指针变量 p2 赋值
```

声明符“*”加上一种类型就可以定义指向该类型的指针，如：

```
char *pc;   // 定义一个指向 char 类型变量的指针
double *dc; // 定义一个指向 double 类型变量的指针
```

**提示**

请大家注意：int* p; 和 int *p; 是完全等价的。但由于前者是大多数公司的 C++ 代码所遵守的代码风格，也是由谷歌 C++ 风格指南（Google C++ Style Guide）所指出的代码风格，因此本书中绝大多数代码将使用 int* p; 的风格，这也更符合笔者个人的习惯。

虽然对编译器来说二者完全等价，但从人类理解的角度，最好的方式就是**“从后向前读”**：“int* p; ”强调“一个变量 p，它的类型是 int*”，即强调 p 是一个“int 的指针”；而“int *p;”则强调“p 是一个指针，指向 int 类型”。**这种理解方式对后面理解 typedef 很有帮助**。

## 结构体与指针

C 语言不仅支持 int、char 等内置类型，也支持自定义类型。结构体就是为我们提供的一种自定义类型的功能。假如你正在编写一个游戏，你可能会定义如下类型：

```
struct BlackDragon {
    int max_hp;     // 血量上限
    int current_hp; // 当前血量
    int attack_power;   // 攻击力
    // ...
};
```

以上代码定义了一个名为“BlackDragon（黑龙）”的类型。类似可以通过

```
int t;  // int 是类型名，t 是变量名
```

声明变量一样，有了我们也可以声明一个 BlackDragon 类型的变量，只不过自定义类型前面要加上 struct 关键字：

```
// struct 是关键字，BlackDragon 是类型名，dragon1 是变量名。
struct BlackDragon dragon1;
```

类似你可以通过 int 类型定义多个不同的整型变量一样，你也可以通过 BlackDragon 类型定义多条不同的黑龙：

```
struct BlackDragon happy_dragon;    // 一条开心的黑龙
struct BlackDragon bad_dragon;      // 一条坏坏的黑龙
struct BlackDragon dragon_without_lefteye;  // 没有左眼的黑龙
```

如果你正在写的游戏程序有满屏黑龙在天上飞，还可以将它们放到一个数组里：

```
// 类似数组定义：int a[100]，只不过前面多了关键字 struct
struct BlackDragon dragons[100];
```

我们可以**通过“.”运算符访问结构体中的成员**，例如黑龙 dragon1 被打了，掉了 10 滴血：

```
dragon1.current_hp -= 10;
if(dragon1.current_hp <= 0) {
    // 死了，进行相应处理。
```

```
}
```

既然结构体是一种类型，就可以像定义

```
int* p;     // 定义指向 int 类型的指针
char* p2;   // 定义指向 char 类型的指针
double* p3; // 定义指向 double 类型的指针
```

一样，定义指向结构体类型的指针，如：

```
// 定义一个指向 BlackDragon 类型的指针，使其指向 dragon1 这个变量
struct BlackDragon *p = &dragon1;
```

于是，就可以通过指针访问 dragon1 中的成员：

```
(*p).current_hp -= 10;
if( (*p).current_hp <= 0 ) {
    // 死了，进行相应处理。
}
```

而对于指向结构体的指针 p，要通过 p 访问 dragon1 的数据，写成“(*p).xxx”有些麻烦。C 语言专门提供了一个运算符：**“->”运算符**（减号和大于号两个符号组成），这个运算符很像一个箭头，p->current_hp 形象地表示了“p 所指向的对象的 current_hp”这个成员变量。p 所指向的对象也就是 (*p)。即：

(*p).current_hp 等价于 p->current_hp

这两个语句都是访问结构体指针 p 所指向的对象中的 current_hp 这个成员变量。

**提示**

为什么不能直接写成 *p.current_hp？这是因为“.”运算符的优先级大于“*”运算符。就像写 2 + 3 / 5 时，“/”运算符会先“被触发”一样。所以如果写成 *p.current_hp，你是在将黑龙的当前血量值当成一个地址，尝试对该地址进行**解引用**。

结构体中当然可以有指针成员，如：

```c
struct BlackDragon {
    //...
    int* demo;  // 无意义，仅作语法演示
    struct Armor* armor;    // 黑龙装备的铠甲（假设 Armor 类型已定义）
};
```

当然，也可以有指向自身类型的指针成员，如：

```c
struct BlackDragon {
    // ...
    struct BlackDragon* son;   // 这条黑龙的儿子
    struct BlackDragon* father;    // 它的父亲
};
```

## 结构体与指针、typedef

**提示**

对任何人来说（包括初学时的笔者本人）typedef 与指针、结构体的结合都需要时间慢慢体会。因此本书中尽量少使用 typedef，在使用时也一定不会影响到大家理解。但为了方便大家阅读其他资料，此处还是给出 typedef 较为详细的说明。建议大家阅览一遍，若后面遇到类似问题，再回来看即可。

**typedef 关键字的作用是为已有类型起个别名。**语法是

```c
typedef < 已有类型名 > < 别名 >
```

如：

```c
typedef int MyType;
```

为 int 类型起了个别名，MyType，于是我们便可以将 MyType 当作 int 类型使用：

```c
MyType a = 5;
printf("%d\n", a);  // 输出: 5
```

类似地：

```c
typedef char MyChar;
typedef double MyDouble;
MyChar c = 'c';     // 相当于 char c = 'c';
MyDouble d = 3.14;  // 相当于 double d = 3.14;
```

如前所述，既然结构体是一种类型，我们也可以为结构体起个别名，如：

```c
typedef struct BlackDragon MyBlackDragon;
```

使用 typedef 定义的别名来声明结构体类型的变量，便不再需要使用 struct 关键字：

```c
BlackDragon d1;  // 错误：需要 struct 关键字
MyBlackDragon d1;   // 正确：无需 struct 关键字
```

## 指针与 typedef

还可以用 typedef 来定义指针类型，以下两行代码是等价的：

```c
typedef int* pInt;
typedef int *pInt;

// 相当于 int* p; 或 int *p; 二者等价。
pInt p;
```

**要理解使用 typedef 定义指针，最好的办法就是“从右往左读”：**

1. 对于第一行代码，可以理解为：定义了一个类型别名“pInt”，这是一个“int*”类型，即 int 的指针类型；
2. 对于第二行代码，可以理解为：定义了一个类型别名“pInt”，这是一个指针类型，指向的是 int。

类似地：

```c
typedef struct BlackDragon* pBlackDragon;
typedef struct BlackDragon *pBlackDragon;

MyBlackDragon b1;
// 相当于 BlackDragon* p1 = &b1; 或 BlackDragon *p1 = &b1, 效果完全一样。
pBlackDragon p1 = &b1;
```

还可以一次定义多个别名：

```
typedef int MyInt1, MyInt2, *pInt;
MyInt1 a = 1;   // 正确，相当于 int a = 1;
MyInt2 b = 2;   // 正确，相当于 int b = 2;
pInt p = &a;   // 正确，相当于 int *p = &a;
```

## 结构体与指针、typedef

我们可以把

```
struct BlackDragon {
    //...
}
```

这一大坨代码直接看成是一种类型（它也确实是一种类型），从而可以直接对该类型使用 typedef:

```
typedef struct BlackDragon {
    // ...
}MyBlackDragon;
```

以上代码将一个名为 BlackDragon 的结构体类型起了个名字叫做 MyBlackDragon。

在 C 中，一种很常见的做法就是对匿名类型直接使用 typedef 为它定义名字：

```
// 将某匿名结构体定义为 BlackDragon 类型
typedef struct {
    // ...
}BlackDragon;

// 有了如上定义，就可以：
BlackDragon dragon1;    // 定义一个 BlackDragon 类型的变量，dragon1
```

上面的代码与

```
struct BlackDragon{
    // ...
};
struct BlackDragon dragon1;
```

几乎无异，都是定义了一种类型，名字叫做 BlackDragon，只不过直接使用 typedef

的好处在于后面想要定义 BlackDragon 类型的对象，可以省去 struct 关键字。

如果你完整地看完了本序章所有内容，现在应该可以理解如下代码：

```
typedef struct {
    int max_hp;
    int current_hp;
    // ...
}BlackDragon, *pBlackDragon;
```

以上代码为一个匿名类型（结构体）定义了两个名字：

1. 名字 BlackDragon 就是该结构体类型；
2. 从右往左读，pBlackDragon 是一个指针类型，是指向这个结构体类型的指针类型。

于是：

```
BlackDragon dragon1;    // 正确：定义一个 BlackDragon 类型的变量 dragon1
pBlackDragon pDragon1 = &dragon1;   // 正确：定义指针变量 pDragon1，指向
↪  dragon1。

// 上一行代码相当于：BlackDragon *pDragon1 = &dragon1;
```

# 第一章　数据结构

**waypoint-1**

欢迎您正式进入大陆。请您放心，我不会在此处引入诸如“数据”的定义、什么是“数据结构三要素”等等内容。但是同时也请您放心，我会让您彻底搞懂，什么是“数据结构”，“数据结构”究竟是在研究什么。

## 1.1　数据结构

事实上，“数据结构”看起来抽象晦涩的定义，恰恰能够最好地理解“什么是数据结构”。接下来就请你随我一起进入这段拨云见日的旅程。

**定义**

**数据结构**是相互之间存在一种或多种特定“关系”的数据元素的集合，以及定义在这些数据元素上的“操作”的集合。

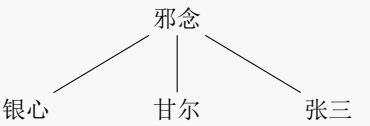
我们以一个例子开启这段旅程：假设你是一个游戏程序员，你要编写一个游戏，玩家操控具有四个“角色”组成的小队，如：邪念、银心、甘尔以及张三这四个人，那么，每个“角色”就都可以被看成是一个数据元素。

> **数据元素**，就是我们要研究和处理的一个个“东西”。这些“东西”可以是一个游戏角色、一件装备、一条聊天消息、一个整数、甚至一张地图等等。

但是，仅仅有了四个数据元素，还不能叫“数据结构”，我们还要解决一个问题：这四个人是什么关系？比如你设计的游戏玩法是将四人排成一个有前后顺序的队伍，那么四人便会有如下的线性关系：

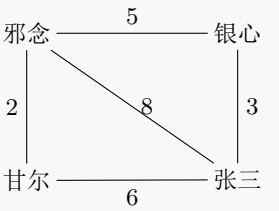
邪念 → 银心 → 甘尔 → 张三

或者你设计的玩法是一个队长三个队员，则四个人便有如下的树状关系：




**waypoint-5**

再假设，四个人之间互相存在好感度，则四个人可能有如下的图结构：




请注意，这里我们使用无向图描述，是因为我们假设了 A 对 B 的好感度与 B 对 A 的好感度相同，但现实中往往并非如此，要描述这种更现实的情况，就要使用第五章中将要介绍到的有向图。

到现在，“数据结构”这个词是不是已经变得具体了？再回看数据结构的定义，我们看到它还有后半句：“操作”。

假设现在我们的游戏队伍：

邪念 → 银心 → 甘尔 → 张三

玩家可能会在游戏中让某些人“加入队伍”（Insert），某些人“退出队伍”（Delete），如卡姐加入：

邪念 → 银心 → 甘尔 → 张三 → 卡姐

邪念退出：

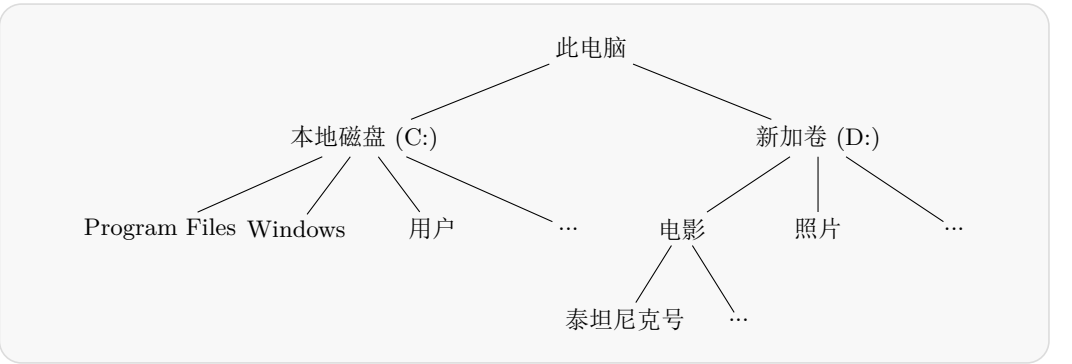
银心 → 甘尔 → 张三 → 卡姐

这里“入队”、“出队”，就都是所谓的“定义在这些数据元素上的操作”。再比如，新人入队前，你可能需要判断当前队伍是否已满，因此需要“获取队伍人数”（GetSize），这也是“定义在这些数据元素上的操作”。

有了以上的介绍，再回过头来看那个之前觉得它晦涩难懂的数据结构定义，就豁然开朗了。

任何一本《数据结构》的参考资料，都一定会有“线性表”、“队列”、“树”、“图”等这

些内容。为什么我们要重点研究这些数据结构呢？事实上，“线性表”、“队列”、“树”、“图”等，正是我们处理实际需求时，组织数据所需要的最有用的、最常见的结构。比如，一个有趣的事实是：你的个人计算机中目录结构的组织，其实就是一棵树。如：




请注意，以上出现的线性结构、树状结构、图结构，都只是逻辑意义上的结构，展示出数据元素之间的逻辑关系，而非数据元素物理存储（如在主存或磁盘中）的位置关系。这种逻辑上的关系称为数据的**逻辑结构**。《数据结构》不仅研究数据的逻辑结构，还要研究如何实现物理存储，这通常通过计算机语言来实现。本书中后面我们介绍到的每一种数据结构，都会给出其比较常见的、一般的实现方式。如果你感兴趣的话可以想一想，以上出现的数据结构，你如何通过你已有的编程语言知识来实现？

## 1.2　笔者的谏言

通过以上的介绍，我们就了解了《数据结构》或者叫《数据结构与算法》这个学科，就是研究在不同的实际需求下，数据如何存储、组织，以及高效操作的学科。

笔者认为，对于《数据结构》这个学科，“掌握知识”和“培养能力”同等重要。学习《数据结构》，真正重要的是，在学习的过程中不断思考、体会和理解这些数据结构与算法背后的思想及其代码实现原理，训练自己灵活的思维和解决问题的能力。**因此本书在组织编排上，传递知识的同时更加注重对读者严谨、细致、清晰的引导，来培养读者的思维能力和创造力。相信大家在研读的过程中不只是能学到知识，更是能有“创造”的快感，好像很多算法、理论都是自己创造出来的一样，这样，自然也会对它们记忆深刻，历久弥新**。

那接下来就让我们将一起走进这段学习、探索和创造的旅程，共同体会《数据结构》大陆中的美丽风景。

# 第二章　线性表

欢迎来到第二章。

假设某游戏的玩法是玩家操控一个四人组成的小队，队伍成员存在严格有序的位置关系，类似这样：

邪念 → 银心 → 甘尔 → 张三

这，其实就是一个由四个“游戏中的可操控角色”构成的线性表，表中有四个元素，元素的类型是“角色”，可能类似这样：

**代码 2.1**

```
struct Role{
    int hp;             // 生命值
    int attack_power;   // 攻击力
    // ...              // 其他
};
```

有了这个例子，我们给出如下线性表的定义：

**定义**

**线性表**是具有相同数据类型的 $n(n >= 0)$ 个元素构成的有限有序序列，其中 $n$ 为表长，当 $n = 0$ 时线性表是一个空表。

请注意，线性表所称元素“有序”，这并不是说元素的值大小有序，而是元素有严格的位置关系。例如，我们可以用 $L$ 将线性表表示为：

$$L = (a_1, a_2, ..., a_n)$$

其中，$a_1$ 是唯一的第一个元素，$a_n$ 是唯一的最后一个元素。将其中任意两个元素位置互换后，得到的新的线性表与原来的线性表并非同一个线性表。

## 2.1　顺序表和链表简介

线性表最常见的实现方式有两种。在 C 语言中，我们可以利用数组让这些元素在内存中连续地存放。这种利用一段连续的存储空间依次保存表中元素的线性表被称为**顺序表**。顺序表的存储结构定义如代码 2.2 所示。

**代码 2.2　线性表的顺序存储结构**

```c
#define MaxSize 100
struct SeqList{  // SeqList，即 Sequential List
    ElemType data[MaxSize];  // 存放表中元素，元素类型为 ElemType
    int current_size;        // 当前表中的元素数量
};

// 有了以上结构定义，就可以通过以下代码定义一个“顺序表”对象 L:
// struct SeqList L;
```

其对应的物理存储结构如图 2.1 所示。

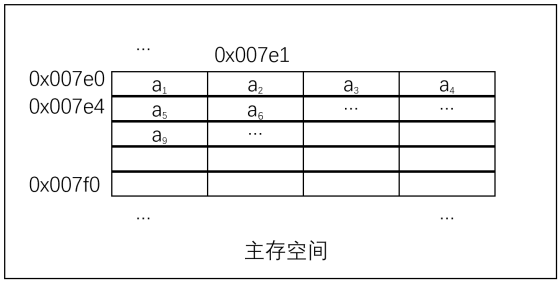



图 2.1 顺序存储的线性表

**提示**

代码 2.2 中，我们用 ElemType(Element Type) 表示表中的元素类型。比如你要处理一组 int 型数据，把它们放在线性表中，ElemType 就可以是 int 类型；再比如你要处理四个游戏角色，ElemType 就可以是 Role 类型 (类似代码 2.1)。

**初见提示**

相信你懂我的意思，但初见我还是要多啰嗦一句：图 2.1 中我们假设了操作系统为该顺序表分配的起始地址为 0x007e0，每个元素大小为 1 字节。而在后续诸多内容中，在不会引起误解的前提下，我们将省去这种用于严谨的话术，而专注于“传意”的意图。

顺序表保证了逻辑上相邻的两个元素物理上也一定相邻。因此，表中任何一个元素 $a_i$ 的地址 Address($a_i$) 都可以由

$$\text{Address}(a_i) = \text{Address}(a_1) + \text{sizeof(ElemType)} \cdot (i-1)$$

计算得来。因此，访问（读或写）顺序表中的任意一个元素，都可以在 O(1) 时间内实现，这种特性我们称为**顺序表支持随机访问**。

*** 思维拓展**

若线性表可以对任意元素 $a_i$ 进行读写，而且操作时间为 $O(1)$，与 $i$ 和数组大小 $n$ 无关，则这种访问方式称为**随机访问（random access）**。如何理解“随机”？第一，可以理解为表可以对任一位序实现 O(1) 的访问效率，即使位序是“随机”给出的。第二，与链表做对比。我们将马上看到，链表只能顺序访问（sequential access），而无法随机访问。也就是说，要访问链表中位序为 i 的元素，我们必须依次访问 $a_1, a_2, ..., a_{i-1}, a_i$。而无法跳过前面直接访问 $a_i$。

不难想到，可以使用如下代码初始化一个顺序表并为其构造元素：

```
// 假设我们要将存放在数组 a 中，长度为 n 的数据插入某个顺序表中
struct SeqList L;         // 定义一个顺序表
L.current_size = 0;      // 空表初始元素数量为 0
// 插入数据
for(int i=0; i<n; i++) {
    if(L.current_size < MaxSize) {
        // current_size 就是表尾元素的下一个位置的下标。
        L.data[L.current_size] = a[i];
        L.current_size++;
    } else {
        // 表已满，拒绝插入
    }
}
```

以上是通过数组实现线性表的方式。另一方面，通过 C 语言提供的指针，我们也可以让每个元素通过一个指针指向下一个元素，从而链接起整个线性表。这种通过指针，将一个个物理上未必连续的存储空间链接起来的线性表被称为**链表**。在这种实现方式下，表的每个节点除了存储数据元素本身，还要额外存储一个“指向下一个元素的指针”。因此，我们需要一个 struct 来描述表的一个“节点”，如图 2.2 以及如下代码所示：

```
struct ListNode{
    ElemType data;  // 该节点的数据元素
    struct ListNode* next_node; // 下一个节点的地址
```

```
};
```

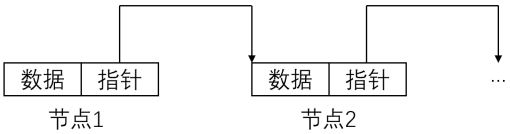



图 2.2 链表的节点

由于有了第一个链表节点的指针，就可以顺着链条找到所有节点，因此，**一个链表通常用其第一个节点的指针来表示**：我们定义

```
typedef struct ListNode* LinkList;  // LinkList 是指向 ListNode 的指针类型
```

于是，就可以用

```
LinkList L;
```

来定义一个“链表”对象 L（就像用”int a;” 来定义一个 int 型对象 a 一样）。

**提示**

LinkList 与 ListNode* 是同一种类型。只不过语义上强调的重点不同：ListNode* 强调“这是一个节点”，而 LinkList 强调“这是一个链表”。我们也经常使用

```
typedef struct ListNode* head;
```

来突出 head 是链表第一个节点的指针。

链表的物理存储结构如图 2.3 所示。

## 2.2　顺序表

经过上面的介绍我们可以看出，顺序表的优点在于：①支持随机访问，访问表中任意元素可以 O(1) 时间实现；②几乎无需额外存储空间存储额外信息，不像链表那样需要额外存储 $n$ 个指针。顺序表的缺点也很明显：插入或删除表中元素时需要移动大量元素。

相信大家对于数组已经比较熟悉了。我们这里直接给出顺序表常见操作的代码实现，通过代码大家也可以进一步体会这一点。对于一个未初始化的顺序表 L：

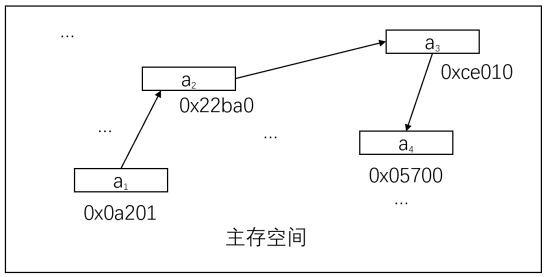



图 2.3 链式存储的线性表

```
struct SeqList L;
```

### 1. 顺序表的初始化操作

顺序表中通过一个整数变量 current_size 来维护当前表长（表中已有元素数量）。初始化时，将其置为 0：

```
void InitList(struct SeqList *L) {
    L->current_size = 0;
}
```

### 2. 顺序表的插入操作

以下函数实现了操作：在顺序表 L 中，第 index 个位置插入一个值为 value 的元素。index=1 表示在表头插入。

```
int InsertSeqList(struct SeqList *L, int index, ElemType value) {
    if (L->current_size >= MaxSize) {
        return 0; // 表已满
    }
    if (index < 1 || index > L->current_size+1) {
        return -1; // 插入位置非法
    }
    // 移动元素，为待插元素“腾出位置”
    for (int i = L->current_size; i >= index; i--) {
        L->data[i] = L->data[i - 1];
    }
    // 在第 index 个位置插入新元素，即数组下标为 index-1 的位置
    L->data[index-1] = value;
    L->current_size++;
    return 1;
}
```

插入过程如图 2.4 所示。对于顺序表，要在 index 位置处插入新元素，就要将大

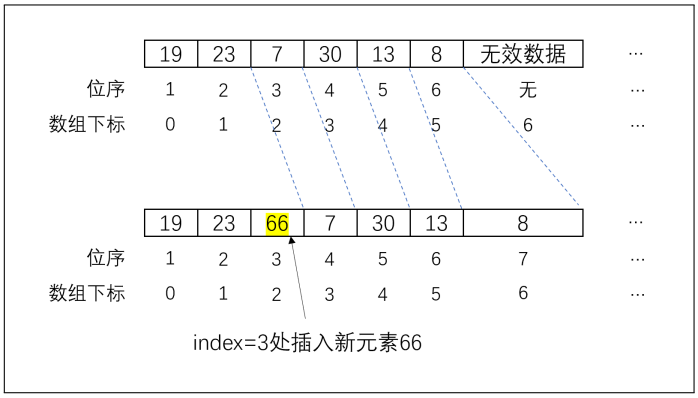



图 2.4 顺序表的插入操作

量元素依次向右推移一个位置，为新元素腾出位置。显然该算法的瓶颈在于元素移动。不难得出该算法最好情况下（在表尾插入，无需移动元素）时间复杂度为 $O(1)$。最坏情况下（在表头插入）元素需要移动 $n$ 次，$T(n)=n$，时间复杂度为 $O(n)$。平均情况下，假设在各个位置插入概率相等，则有

$$T(n)=\frac{1}{n+1}\cdot 1+\frac{1}{n+1}\cdot 2+\ldots+\frac{1}{n+1}\cdot n=\frac{n}{2}$$

所以平均情况下，算法时间复杂度为 $O(n)$。

**3. 顺序表的删除操作**

以下函数实现了操作：将顺序表 L 中第 index 个元素删除，并将其值以 value 变量返回。index=1 表示删除表头元素。

```c
int DeleteSeqList(struct SeqList *L, int index, ElemType *value) {
    if (index < 1 || index > L->current_size) {
        return -1; // 删除位置非法
    }
    // 将被删元素赋值给变量 value
    if (value != NULL) {
        *value = L->data[index-1];
    }
    // 用第 index+1 个元素（如果有）覆盖第 index 个元素，并将后面所有元素“前移一格”
    for (int i = index-1; i < L->current_size - 1; i++) {
        L->data[i] = L->data[i + 1];
    }
    L->current_size--;
    return 1;
}
```

与插入操作的思想类似，显然删除操作的时间复杂度同样为 O(n)。

**初见提示**

事实上绝大多数时候我们关注的都是平均复杂度，因此，在未指出“最好”“最坏”或“平均”时，我们提到的“时间复杂度”或“空间复杂度”都是指平均。

*** 思维拓展**

相信这里已经是一个非常合适的位置来指出这几件事：

1. 顺序表插入操作的实现代码中，我们规定表满时返回 0，插入位置非法时返回-1，插入成功时返回 1。为什么要这么规定？为何元素的位序 index 从 1 开始算，而不是从 0？为何插入操作名字叫做 InsertSeqList？叫别的名字可以吗？
回答：事实上，这些并非死规定，而是由该函数的实现者来定义的。如果你来实现这个函数，你当然可以做出其他合理规定，如：“只要插入失败都返回 0，只要插入成功就返回 1”，再如你将函数名起为“SeqListInsert”，这都是可以的，只要你正确地实现了该函数，并给出清晰的说明让该函数的使用者知道这个函数具体怎么用就行。类似的事情后面将不再赘述。

2. 对于顺序表的删除操作，你也许会问，“删除操作”不就是删掉元素吗？为何还要将被删元素的值返回？这是因为在实际需求中，我们删除一个元素往往需要同时拿到它的值。比如，若银心出队去营地休息，我们将她从玩家队伍中移出，可能同时需要将她放入营地，这就需要获取“银心”这一数据元素，因此删除的同时要返回该数据。你当然可以将删除操作实现为“只删除，不返回被删元素”，只要你实现得正确。但正如 1.1 节末尾所指出，本书中每一种数据结构，我们给出的都是较常见的、一般的实现方式。这一点后面将不再赘述。

说到这里是时候指出《数据结构》的一个重要思想，请看下面的“3”。

3. 你在解决实际问题时（很可能是某个 OJ 网站上的问题），在你玩转某个数组的时候，你顺着自己的思路写出的很可能其实就是顺序表的插入、删除等操作，而若逻辑不长，你决不会把它封装成一个函数类似 InsertSeqList(...)。但是，《数据结构》的研究在于抽象和封装。实际的工程中，你也可以将某个结构及其对应的操作封装好，并和别人说：“这个结构是一棵平衡二叉树，它支持操作 123456 等、作用分别是 xxxxxx，你可以直接拿去用（而无需关注其底层是怎么实现的，因为我已经帮你实现好了）”。当然，你最好给出每个接口实现（函数实现）的详细说明，如时间、空间复杂度，各个返回值表示的含义等等。相信随着对《数据结构》的研究，你将逐步加深对该思想的体会。

此处略去若干内容

# 第三章　栈和队列

先说栈。在实际应用中有很多类似栈的数据调度方式。想象冬天你去澡堂洗澡。你脱衣服的顺序一定是从外到内，而穿衣服刚好相反。如图 3.1。

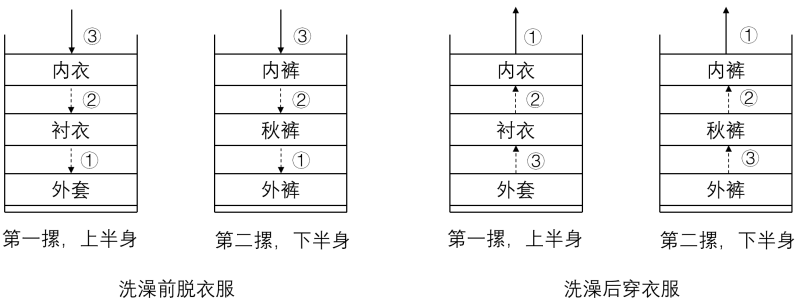



图 3.1

事实上，如果你将两摞衣物放在某个容器里，你会发现这个容器有这样的特点：先放入的在下方，后取出；后放入的在上方，先取出，称为**“后进先出”**，或者叫**“先进后出”**。这样，其实你就是在将这两个容器当成两个**栈**来使用。

而**队列**就很好理解了，正得名于生活中的“队列”。想象某门店很火爆，门前排满了人，队首的人买完东西出队，新来的人入队到队尾。

接下来我们给出完整、严谨的栈和队列的定义。

**定义**

**栈（Stack）**是只允许在一端进行插入或删除操作的线性表。

如图 3.2 所示，允许插入或删除的那一端叫做**栈顶**，不允许插入或删除的另一端叫做**栈底**，不包含任何元素的空表叫做**空栈**。栈的插入操作叫做**入栈**，删除操作叫做**出栈**，入栈和出栈通常也更生动地叫做“压栈”和“弹出”。

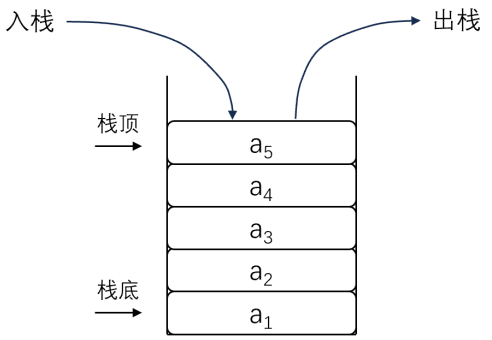



图 3.2 栈的示意图

下面给出**队列**的定义。

**定义**

**队列（Queue）**是只允许在其中一端插入、另一端删除的线性表。

如图 3.3 所示，允许出队的那一端叫做**队首**，允许入队的那一端叫做**队尾**，队列的插入操作叫做**入队**，删除操作叫做**出队**；不包含任何元素的空表叫做**空队列**。

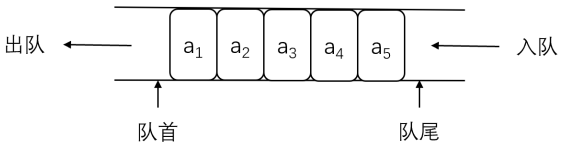



图 3.3 队列示意图

很多人喜欢将栈和队列的特征进行对比，称**队列是先进先出（First In First Out，FIFO）的，栈是后进先出（Last In First Out，LIFO）的。**

以上便是栈和队列这两种逻辑结构的介绍，接下来我们研究如何实现栈和队列。

## 3.1　栈

上一章中我们研究了顺序表与链表的实现。既然栈是一种特殊的线性表，是不是可以用同样的物理存储来实现栈，并且对操作加以特殊限制呢？笔者建议大家在这里暂停，思考一下这个问题：想象一下，分别用数组、链表该如何实现栈？如何设计是好的？会面临哪些问题吗？

此处应该有长时间留白.....

开始。

相信大多数人的都会有明确而正确的思路：

1. 如果使用顺序表来实现栈，由于顺序表“尾插”、“尾删”均为 O(1) 的特性，可以将“表尾”作为“栈顶”，通过操作的定义和实现来限制该表只能在表尾插入、删除，这样既有高效率，又满足栈的特点。如图 3.4 所示，你可能会想到用一个 int 型变量 top 来动态维护“栈顶”的位置。

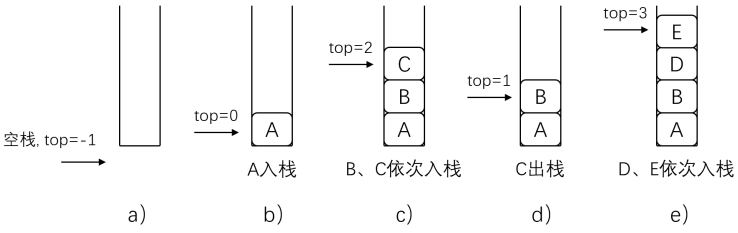



图 3.4 顺序栈实现示意图

这个思路其实就是**顺序栈**的实现思路。采用顺序存储的栈称为**顺序栈**，它利用一组地址连续的存储单元存放自栈底到栈顶的数据元素，同时附设一个指针 top，来指示当前栈顶的位置。

顺序栈的存储结构如以下代码所示：

```
#define MaxSize 100
struct SeqStack {
    ElemType data[MaxSize]; // 存储实际的数据元素
    int top;    // 栈顶指针的下标
};
```

*** 思维拓展**

你也许会发现，这特么不就是线性表么？你不就把 current_size 换成 top 了么？没错，顺序栈和顺序表的关系确实有点暧昧不清。从存储结构角度看，二者存储结构完全相同，只不过语义不同：top 表示栈顶元素的下标，空栈时为-1；而 current_size 表示顺序表中元素的数量，空表时为 0。二者之间微妙的关系在于，current_size 在数值上等于表尾元素（栈顶元素）的下标加一。而如果你将 top 的语义规定为“表尾元素（栈顶元素）的下一个位置”，那么就应将 top 初始化为 0，而不是-1，此时 top 与 current_size 语义虽然不同，但编码上是相同的。

2. 如果使用链表来实现栈，由于链表“头插”、“头删”均为 O(1) 的特性，可以将“表头”作为“栈顶”，通过操作的定义和实现来限制该表只能在表头插入、删除，这样同样既有高效率，又满足栈的特点。事实上，将链表的插入和删除操作限制在头部，即

得到了**链栈**。

接下来我们分别给出顺序栈和链栈常见操作的实现。

### 3.1.1　顺序栈的常见操作实现

#### 1. 顺序栈的初始化

不难看出，初始化一个顺序栈仅需要将其栈顶指针 top 置为-1。

```
int InitStack(SeqStack* S) {
    S.top = -1;
    return 1;
}
```

对于一个未初始化的顺序栈 S:

```
SeqStack S;
```

就可以通过以下代码对 S 进行初始化：

```
InitStack(S);
```

#### 2. 进栈操作

top 指针指向“当前栈顶元素”，因此进栈时要先移动指针到下一个“更高的”可用位置，再将元素放入该位置，此时指针仍指向栈顶。

```
int Push(SeqStack& S, ElemType e) {
    if(S.top == MaxSize - 1) {
        return 0;   // 栈已满，无法入栈
    }
    S.top += 1;
    S.data[S.top] = e;
    return 1;
}
```

如以下代码便可以为栈 S 压入一个值为 3 的元素：

```
Push(S, 3);
```

**初见提示**

注意这里我们首次使用了 C++ 的引用作为函数形参。”SeqStack& S” 中，符号”&” 并不是取地址的意思，而是表示形参 S 是一个 SeqStack 的**引用**类型。相信你也看出引用的作用了，类似使用指针一样，使用引用作为形参，同样可以改变函数栈外的原变量的值。关于引用更多内容请参考第 **??** 页。

**3. 出栈操作**

思想类似进栈，此处留白，请大家自行写出代码。

```
// 将栈 S 中当前的栈顶元素赋值给 e，并将其弹出。
    int Pop(SeqStack& S, ElemType& e) {



    }
```

**4. 获取栈顶元素，但不出栈**

略。

**5. 判断栈是否为空**

```
int Empty(SeqStack S) {
    if(S.top == -1) return 1;   // Empty 成立
    return 0;
}
```

### 3.1.2　链栈的常见操作实现

根据前面的介绍相信你已经看出来了，链栈的实现比链表还要简单：其他操作和链表几乎完全相同，而对于“插”和“删”，将链表的表头作为栈顶，你只实现链表的“头插”和“头删”操作，而不实现任意位置的插入删除操作，你就得到了一个链栈。显然链栈也可以采用带头结点和不带头结点的实现。这里拟采用带头结点的实现。

**提示**

对于带头结点的实现，链表的第一个节点指针 L 指向头节点，L->next_node 指向的是第一个数据元素，即栈顶元素。

**1. 链栈的入栈操作（实际上就是链表的“头插”操作）**

```
int Push(ListNode* L, ElemType e) {
    ListNode* new_node = (ListNode*)malloc(sizeof(ListNode));
    new_node->data = e;
    new_node->next_node = L->next_node;
    L->next_node = new_node;
    return 1;
}
```

**2. 链栈的出栈操作（实际上就是链表的“头删”操作）**

```
int Pop(ListNode* L, ElemType& e) {
    if(L == NULL || L->next_node == NULL) {
        return -1;
    }
    e = L->next_node->data;
    ListNode* node_to_free  = L->next_node;
    L->next_node = L->next_node->next_node;
    free(node_to_free );
    return 1;
}
```

显然实现一个栈没有必要使用双链表，更没有必要将首尾链接起来做成循环链表。

由于顺序栈和链栈就是有特殊限制的顺序表和链表，其优劣我们不再赘述，而是作为习题放在习题 2 中。

### 3.1.3　* 浏览器的前进后退功能实现

这里给出一个作为思维训练、理解栈的意义的极好的例子。大家应该都用过浏览器的“后退”和“前进”功能，如图 3.5。当你按住鼠标右键从右向左滑动，或者点击右键选择“后退”时，浏览器会访问你上一个访问过的网页，向右滑或点“前进”则回到上一个回退前的网页（如果你没用过，建议先打开浏览器试一下）。假设你已经访问过了：

```
网页 1 -> 网页 2 -> 网页 3
```

此时你处在网页 3，点击“后退”，会回到网页 2，再后退，回到网页 1，再前进，回到网页 2，再前进，回到网页 3。

假设：

1. 用户随时可能访问一个新的网站；

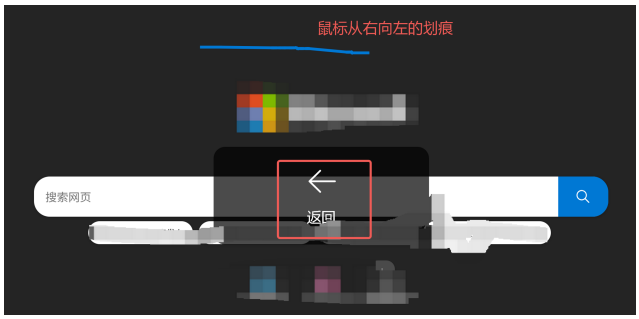



图 3.5

2. 用户随时可能点击“后退”（鼠标左滑）；
3. 用户随时可能点击“前进”（鼠标右滑）。

如果你是某浏览器的实现者，请你实现这三个功能，要求当用户点击“前进”或“后退”时，能迅速地拿到对应网页的地址。

提示：网页地址（如 https://xxxxxx）可以用一个 char 的数组（或 string 类型）来表示。你可以用任何你喜欢的方式来模拟用户的输入，并让你的程序做出合理反应。

此处应该有留白。

具体的模拟方式各有千秋，我们只给出核心算法的实现思路。不难看出，当用户访问了若干页面之后：

```
网页 1 -> 网页 2 -> 网页 3
```

“后退”操作实际上就是依次访问与之前的顺序相反的页面，满足“后进先出”的特点，因此用户每访问一个新网页 B，我们就可以将用户所处在的当前网页 A 压入一个栈 back_stack，当用户进行后退时，取出栈顶网页访问即可。back_stack 语义上可以理解为：为用户的后退行为而预备的一个栈，栈顶元素永远是用户此时点击后退时所要访问的页面。

还有前进操作。“后退”的顺序是与用户主动访问的新网页顺序相反，不难看出，“前进”的顺序是与“后退”相反的。因此可以维护令一个栈 forward_stack，用户未进行过后退操作时该栈为空，当用户连续进行“后退”时，将网页依次压入栈 forward_stack。需要注意：当用户访问了一个新网页后无法“前进”，需要清空 forward_stack 重新期待用户的下一次后退操作。流程如图 3.6 所清晰地展示。

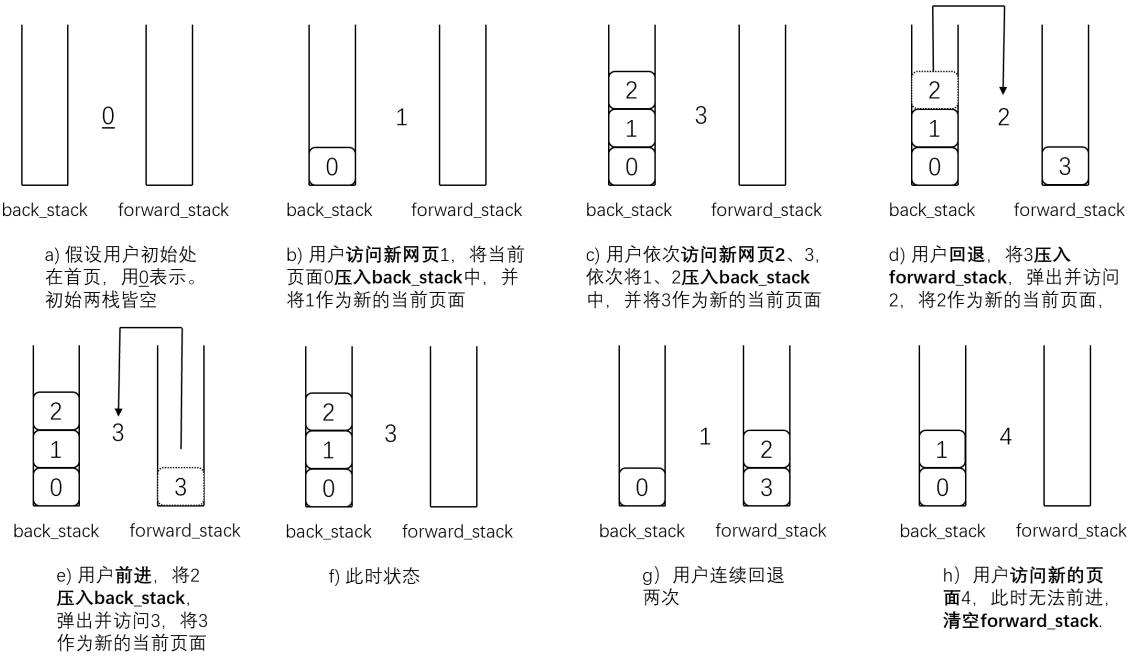



图 3.6 使用两个栈实现浏览器前进后退

### 3.1.4 *“函数栈”、“调用栈”是啥？

# 此处略去若干内容

## 3.2 队列

如前所述，队列同样是一种特殊的线性表，因此队列同样可以有顺序存储和链式存储两种物理结构，并在顺序表或链表的基础上实现特殊机制。队列的链式存储稍简单些，我们先研究链式存储，再研究顺序存储。

### 3.2.1 队列的链式存储

队列需要实现两个操作：在链表的头部删除元素、尾部插入元素。单链表头插本身可以达到 O(1)。我们只要额外保存一个链表的尾指针，那么在尾部插入元素的操作也可以达到 O(1)。显然链式队列同样可以带头结点或不带头节点，这里拟采用带头节点的实现。

第二章中已经给出过链表节点的定义，这里为了体现出节点是队列的节点，我们把结构改个名字：

```
struct QueueNode {
    ElemType data;
    struct QueueNode* next_node;
};
```

额外保存一个表尾指针来使出队操作时间达到 O(1)：

```
typedef struct {
    struct QueueNode* front;     // 表头（队首）指针，指向头节点
    struct QueueNode* rear;     // 表尾（队尾）指针，指向最后一个节点
}LinkQueue;
```

以上代码将 LinkQueue 定义为一个“链队列”类型。有了如上定义，我们就可以用如下代码定义链队列对象：

```
LinkQueue happy_linkqueue;  // 定义一个开心的链队列
LinkQueue sad_linkqueue;    // 定义一个悲伤的链队列
```

一个链队列如图 3.7 所示。前面的 happy_linkqueue 以及 sad_linkqueue 都是定义了但未初始化的。接下来我们给出链式队列的常见操作实现（带头结点）。

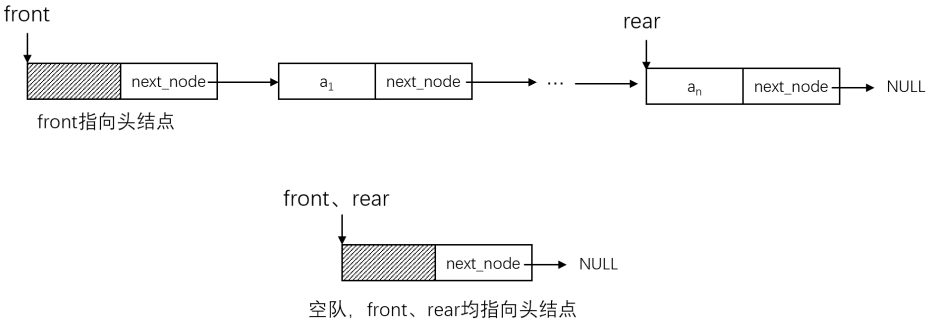



图 3.7 带头结点的链式队列示意图

### 1. 初始化操作

```
int InitLinkQueue(LinkQueue& q) {
    q.front = (QueueNode*)malloc(sizeof(LinkQueue));     // 为头节点分配空间
    q.rear = q.front;    // 由于插入时要在尾后插入，所以 rear 初始指向头节点符合条件

    q.front->next_node = NULL;
```

```
    q.rear->next_node = NULL;
    return 1;
}
```

使用：

```
InitLinkQueue(happy_linkqueue); // 初始化开心的链队列
InitLinkQueue(sad_linkqueue);   // 初始化悲伤的链队列
```

### 2. 判断是否空队

```
int Empty(LinkQueue q) {
    if(q.front == q.rear) {
        return 1;
    }
    return 0;
}
```

### 3. 入队操作（尾插）

```
int EnQueue(LinkQueue& q, ElemType e) {
    if(q.front == NULL || q.rear == NULL) {
        return -1;  // 你给我个没初始化的队列我咋给你插入元素？
    }
    QueueNode* nwe_node = (QueueNode*)malloc(sizeof(QueueNode));
    nwe_node->data = e;
    // 操作链条
    nwe_node->next_node = NULL;
    q.rear->next_node = nwe_node;

    // 更新尾指针
    q.rear = nwe_node;
    return 1;
}
```

### 4. 出队操作（头删）

一般情况下出队不会改变尾指针的指向，但需要注意一个特殊情况，即队列中仅有一个节点，如图 3.8 所示。此时队首节点就是队尾节点，出队后，队列变成空队，需要维护队尾节点指针，将其指向头节点。

```
// 将队首元素赋值给 e，并出队
int DeQueue(LinkQueue& q, ElemType& e) {
    if(q.front == NULL || q.rear == NULL) {
        return -1;
    }
    if(q.front == q.rear) {
        return 0;   // 空队
    }   // 注意代码若能走出这个 if，说明队中至少有一个数据节点，可以直接安全地访
    ↪  问 q.front->next_node 的 data 域或 next_node 域。
    e = q.front->next_node->data;
    QueueNode* t = q.front->next_node;
    q.front->next_node = q.front->next_node->next_node;

    // 若队列仅有一个节点，更新 q.rear
    if(q.rear == t)
        q.rear = q.front;
    }
    free(t);
    return 1;
}
```

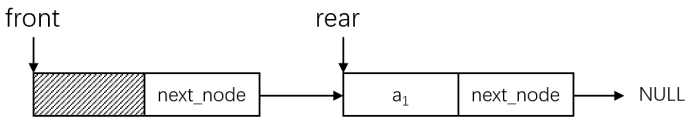



图 3.8 仅有一个数据节点的队列

### 3.2.2　队列的顺序存储

不难想到，要在顺序存储的线性表上实现一个队列，最简单的做法就是标记两个指针（数组下标）front 和 rear，分别指向队头和队尾，初始时将 front 和 rear 均置为 0 即可，这样我们可以规定 front 始终指向队首元素，rear 始终指向队尾元素的下一个可插入位置。

```
#define MaxSize 100
struct SeqQueue {
    ElemType data[MaxSize];
    int front;  // 队头，初始时为 0
    int rear;   // 队尾，初始时为 0
};
```

如图 3.9，入队时将元素插入到队尾并推移队尾指针，出队时推移队头指针。

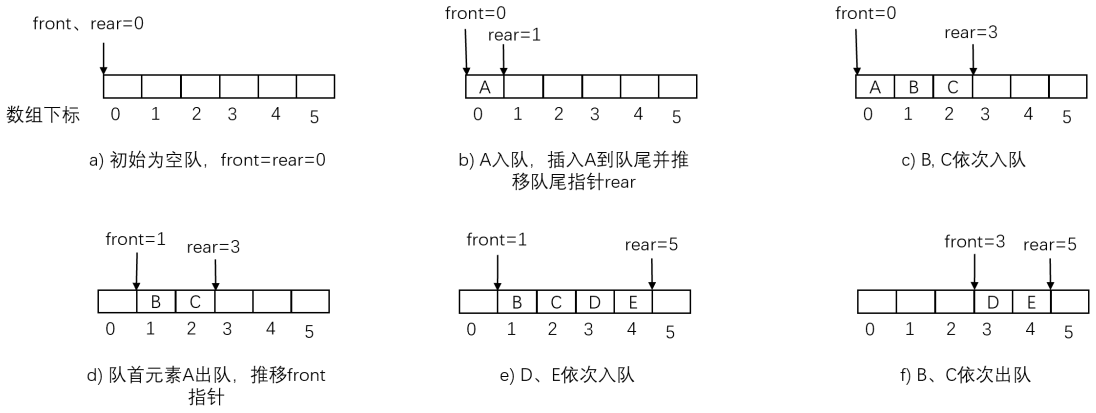



图 3.9 在顺序表上实现队列

然而，一个看似简单的问题很快就会出现：

> 当 rear 已经走到了数组的尽头，而数组的前半部分却明明还有大量空闲空间时，我们还能继续入队吗？

例如，一个容量为 10 的队列，经过几次入队、出队操作后，可能变成：

$$\underbrace{\square\square\square\square}_{\text{已经出队，空闲}}\quad\underbrace{A\ B\ C\ D\ E\ F}_{\text{队列中的元素}}$$

此时数组中明明还有 4 个空闲位置，但如果我们把数组看成一条只能从左向右移动的“道路”，rear 已经走到了道路的尽头。按照普通顺序表的存储方式，我们似乎只能无奈地宣布：

“队列满了。”

可是，真的满了吗？

显然不是。相信聪明的你已经想到了，如图 3.10 所示，当 rear 走到尽头时，我们可以让 rear“回到起点”，继续利用那些已经空闲了的空间，**让 rear 始终“追着 front 跑”，当 rear 真正追上 front 时，确实没有可利用的空间了，此时才是真正的“队满了”**。

这正是**循环队列**所要解决的问题。

循环队列不再把数组看成一条有起点、有终点的直线，而是把数组的首尾连接起来：当指针走到数组末尾时，就“绕回”数组开头，整个数组物理上是一条直线，但逻辑上是一个环，因此叫**循环队列**或**环形队列**。

**因此，队列的顺序存储通常都会实现为循环队列。**

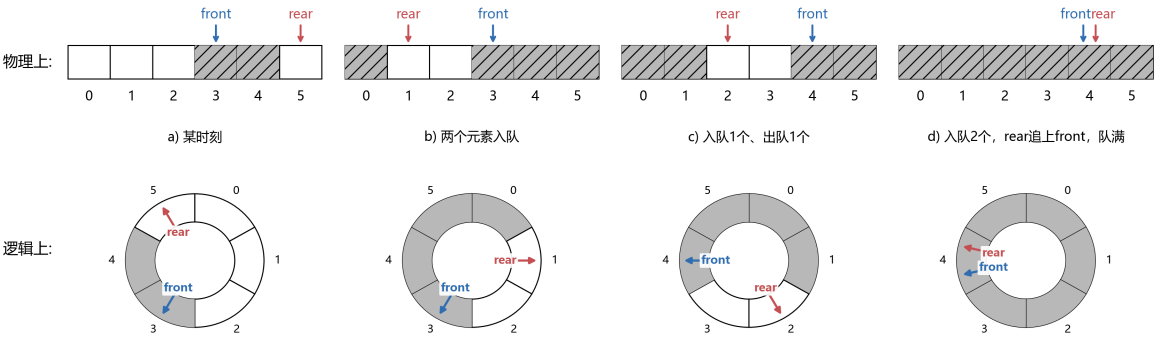



图 3.10 循环队列示意图

“回到起点”，代码上如何实现？举个例子某数组大小为 100，下标为 0-99，当 rear 的值已经为 99 时，如果不加以控制，下一步将走到 100，但我们希望它回到 0。不难发现，这正是“取模”运算的功能。因此入队或出队时，若指针从 99 推移到 100，我们将其对 100 取模，即可重新回到起点 0：

```
if(rear > MaxSize-1) {    // 若 MaxSize 为 100，MaxSize-1 为最大下标 99
    rear = rear % MaxSize;
}
```

*** 思维拓展**

在计算机科学中大家往往喜欢将“取模”和“取余”同等对待，但事实上存在负数参与运算时，取模（modulo）和取余（remainder）数学上的定义可能不同。“%”运算符事实上更接近数学上的取余运算。但这里还是用“取模”来描述，原因在于二者在语感上有差异：“取余”关注的是除法“除完还剩多少”；而“取模”关注的是“映射到哪个模类/固定范围”。此处“%”运算的语义更接近“取模”。

还有第二个问题需要考虑。循环队列如何判断“队空”？可以简单地通过

$$\text{front} == \text{rear} \;\&\&\; \text{front} == 0$$

来判断吗？经过分析发现是不可以的。因为队满的时候，上面的条件也可能成立。如图 3.10 ，对于 d）所示场景，若再进行两次出队、两次入队，就可以达成 front == rear && front == 0 的条件，并且此时队列是满的。如何解决这个问题？相信大家自然地想到的做法是多维护一个字段，动态维护队列中当前元素个数：

```
#define MaxSize 100
struct SeqQueue {
```

```
    ElemType data[MaxSize];
    int front;
    int rear;
    int current_size;   // 新增字段
};
```

这当然没问题。但更多教材给出的做法是少利用一个数组空间，通过 (Q.rear+1)%MaxSize == Q.front 来作为队满的条件，如图 3.11 所示。我们这里遵从多数教材给出的做法，

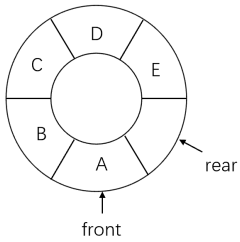



图 3.11 队满示意图

少利用一个数组空间，**使用 (Q.rear+1)%MaxSize == Q.front 来作为队满的条件**。同时可以看到，在此做法下，除队空时 front 和 rear 初始化为 0 之外，其余情况不可能出现 front == rear 的情况（因为我们少利用一个空间，所以 rear 永远无法追上 front，最多只能在它后面一格）。**所以可以将 front == rear 直接作为判断队空的条件**。

至此我们已经完成了所有“思维”上的准备，接下来给出循环队列的常见操作实现。

**1. 初始化循环队列**

```
int InitQueue(SeqQueue& q) {
    q.front = q.rear = 0;
    return 1;
}
```

**2. 判断队空**

```
int Empty(SeqQueue q) {
    if(q.front == q.rear) {
        return 1;
    }
    return 0;
}
```

### 3. 入队

```
int EnQueue(SeqQueue& q, ElemType e) {
    if( (q.rear+1) % MaxSize == q.front) {
        return 0;   // 队满，无法入队
    }

    // 先“放入元素”，再“推移指针”
    q.data[q.rear] = e;
    q.rear += 1;
    if(q.rear>=MaxSize) {
        q.rear = q.rear % MaxSize;
    }
    return 1;
}
```

### 4. 出队

```
int DeQueue(SeqQueue& q, ElemType& e) {
    if(Empty(q)) {
        return 0;   // 空队
    }

    // 先“取出元素”，再“推移指针”
    e = q.data[q.front];
    q.front += 1;
    if(q.front >= MaxSize) {
        q.front %= MaxSize;
    }
    return 1;
}
```

实际应用中，通常不会有“获取队列中的元素数量”的需求，这里笔者稍作提示：对于循环队列，可以通过下式在 O(1) 时间得到队列长度：

$$(\text{rear}-\text{front}+\text{MaxSize}) \ \% \ \text{MaxSize}$$

而对于链队列，需要求队列长度，则需要遍历一遍队列（链表），时间复杂度为 O(n)。

在解决实际问题时，有时会顺其自然地使用到两端均能入队、出队，或一端能入两端能出、两端能入一端能出的队列，它们通常被称为**双端队列**。相关内容我们通过习题 2 作为练习。

## 3.3　栈的进一步理解

提示：本节内容略有难度。

栈后进先出的特性，决定了其可以应用于一些很有趣的场景。我们这里先给出一般化的理解：许多场景下，可以用栈来保存这样的数据，即：**“那些仍然等待未来某个事件发生，并且优先处理后来者”的数据**。现在看这句话过于抽象，不过别急，本节过后你的心会怦怦直跳。

**waypoint-3**

我们给出一例：现在假设天气预报已经报导了未来 n 天的温度，保存在一个数组 temperatures[] 中，请你设计一种算法，对于每一天 i，找出其后方第一个温度高于第 i 天的日子。

最直接的办法就是“检查”每一天 i：从 i 开始向后扫描，直到找到温度高于 i 的那一天，之后回到 i+1，重复这个过程，伪代码如下所示：

```
for(int i=0; i<n; i++) {
    int found = 0;
    for(int j=i+1; j<n; j++) {
        if(temperatures[j] > temperatures[i]) {
            // 找到了.
            found = 1;
            // j 就是要找的那一天

            break;
        }
    }
    if(found == 0) {
        // 第 i 天的后面没有比第 i 天温度更高的了。
    }
}
```

显然这个算法时间复杂度为 $O(n^2)$。分析以上过程我们会发现，事实上我们做了很多查找是“不必要”的。比如，考虑 temperatures[] 输入如下：

25, 23, 19, 18, 20, 27, 26, 27

当我们“检查”25 时，就由前往后查找到了第一个 27 的位置，事实上此时我们就已经能够知道 23 的“答案天”就是 27，并且 19、18 的“答案天”就是 20。只不过，此时我们虽然已经“看到了”这些信息，却没有把它们保存下来。于是，一个很自然的想法出现了：如果我们能通过某种手段将这些**尚未找到答案的天**记录下来，那么未来

的温度出现时，就可以立即检查这些等待中的天是否能够得到答案。当我们检查 25 时，将它记下来：

some_structure: {25}

检查 23，some_structure 中没有元素会得到答案，将 23 记下来：

some_structure: {25, 23}

继续检查 19、18：

some_structure: {25, 23, 19, 18}

当检查到 20 时，终于找到了 19 和 18 的答案，19 和 18 两个元素期待的事情终于发生，因此将 19 和 18 从 some_structure 中移出，将 20 记下来：

some_structure: {25, 23, 20}

你会惊奇地发现一件事情：留存在 some_structure 中的温度序列一定是单调不增的，因为低温一定会在高温来到时“找到答案”而被移出。因此当新的温度 t 来到时，它能移出哪些温度？其实只需要依次从 some_structure 的尾部依次向前比较，“清除”掉 some_structure 所有小于 t 的温度即可，这样一定不会漏。

说到这里你会发现，some_structure 正是满足“只会在一端进行插入删除”。或者换个角度想，当新温度 t 到来时，后进入 some_structure 的元素一定要先被检查，因为后进入的一定是更小的，如果都不能被 t“清理掉”，前面的就更不会被清理掉。

这不就是栈吗！

想通了这一点，这个问题就变得非常简单：我们设一个栈 s，保存所有“尚未找到答案”的天，并用一个数组 ans[] 保存已经得到的答案：ans[i]=j 表示 i 的“答案天”是 j。如图 3.12，我们遍历一遍数组，依次检查 temperatures[] 中的每一个温度 temperatures[j] = t，将其与栈顶温度 temperatures[s.top] 进行比较，若 t 大于栈顶温度，则栈顶的这一天 s.top 的“答案天”就是 j，并继续将 t 与新的栈顶进行比较，重复这个过程，直至 t 不大于栈顶温度或栈空后，将当前检查的温度 j 入栈。检查完所有温度后，栈中剩余的这些天，就是后面没有温度高于他们的。最后留在栈中的（下标为 5 和 7 的）两天，后面没有温度高于它们。伪代码如下所示：

```
for(int i=0; i<n; i++) {
    while(!Empty(s) && temperatures[i]>temperatures[s.top]) {
        ans[s.top] = i;
```

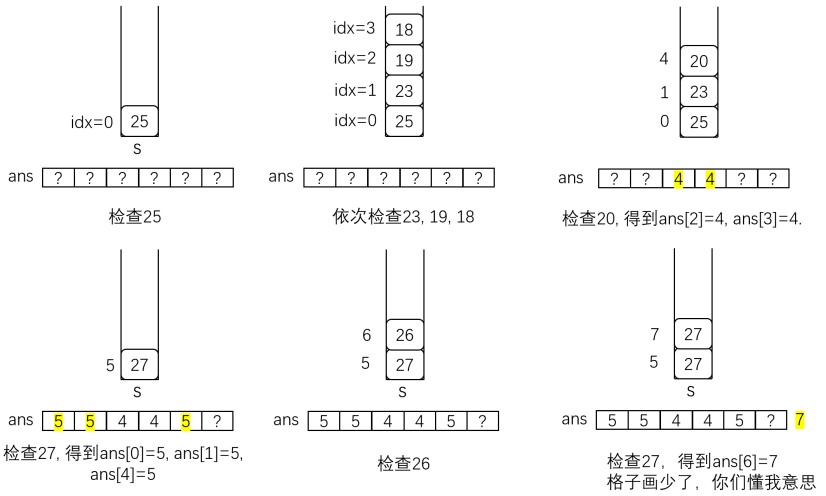



图 3.12

```
4           Pop(s);
5       }
6       Push(s, i);
7   }
8   while(!Empty(s)) {
9       ans[s.top] = 0; // 这里拟用 0 表示后面不存在一天，比 s.top 这一天温度更高
10      Pop(s);
11  }
```

该算法的时间复杂度为 O(n)。至于为什么是 O(n)，相比于上一种算法，该算法的提升在哪里，这里留给读者自行思考。可以看到，栈 s 中的元素是单调不增的，很多人喜欢将这种元素具有单调性的栈称为**单调栈**，但笔者希望强调的更多是思考问题、解决问题的这个创造性的过程，次之才是：下次遇见类似的问题，可以用“单调栈”来解决。

相信通过该例大家已经建立了足够的感觉，我们来回看这句话：许多场景下，可以用栈来保存这样的数据，即：**“那些仍然等待未来某个事件发生，并且优先处理后来者”的数据**。该例并非巧合，而是存在大量类似的问题，比如大名鼎鼎的**“表达式求值”**问题。

**提示**

实际上队列也有类似的用途：存放“那些仍然等待某事发生，并且优先处理先来者”的数据。我们将在第 5.3.3 节，BFS 的实现中看到。

### 3.3.1　表达式的转换与求值

此处略去若干内容

**习题 1**　（参考答案见习题 1 参考答案，第 150 页）

1. [多选] 下列说法正确的是 _____。

A. 栈是一种有特殊要求的线性表。

B. 队列是一种有特殊要求的线性表。

C. 最先入栈的元素一定最先出栈。

D. 最先入队的元素一定最先出队。

2. 以下哪一项不是栈可以进行的操作？_____

A. 将元素压入栈中　B. 判断栈是否为空

C. 删除栈底元素　D. 将栈置为空栈

3. 以下哪一项不是队列可以进行的操作？_____

A. 将元素插入队尾　B. 获取队首元素的值

C. 位序为 i 的元素出队　D. 队首元素出队

4. 某个栈，元素的入栈序列是 $1, 2, 3, 4$ 则以下序列中，不可能作为出栈序列的是：_____

A. 2, 1, 3, 4　B. 2, 3, 4, 1　C. 3, 2, 4, 1　D. 3, 4, 1, 2

5. 关于时间复杂度，以下说法正确的是：

A. 顺序栈的 Push 和 Pop 操作一定是 $O(n)$

B. 链栈的 Push 和 Pop 均可实现为 $O(1)$

C. 使用单链表实现队列时，只要有 head 指针，入队和出队均可实现为 $O(1)$

D. 用数组实现的循环队列，入队操作必须移动大量元素，因此时间复杂度为 O(n)

6. 采用数组实现容量为 $N$ 的循环队列，并规定：

front 指向队首元素；

rear 指向下一个插入位置；

front 和 rear 初始全部指向下标 $N-1$ 处，队列采用保留一个空闲位置的方式区分队满和队空。

则判断队列为空和队满的条件分别为：

A. front == 0 && rear == 0, rear + 1 == front

B. front == 0 && rear == 0, (rear + 1) % N == front

C. front == rear, rear + 1 == front

D. front == rear, (rear + 1) % N == front

7. [多选] 某同学做了如下设计：采用一个长度为 100 的数组实现两个栈 S1 和 S2。该同学规定两栈栈底分别在两个端点处，栈顶向中间生长，并使用两个变量 top1 和 top2 记录两栈栈顶位置，如图 3.13。同时规定两栈栈顶元素相邻时，存储空间满，均无法再入栈。以下说法中正确的是 _____




图 3.13

**A.** 两栈入栈时间复杂度均可以达到 O(1)
**B.** 设两栈元素数量分别为 n、m，则两栈栈顶元素的出栈时间复杂度分别为 O(n)、O(m)
**C.** 要计算两栈栈中已有元素数量之和，时间复杂度为 O(1)

**D.** 若 S1.top 初始化为-1，S2.top 初始化为 100，且两个栈顶指针始终指向各自的栈顶元素，则 S1.top+1==S2.top 时，空间满，两栈均无法再入栈
**E.** 若 S1.top 初始化为-1，S2.top 初始化为 100，且两个栈顶指针始终指向各自的栈顶元素，则可能出现 S1.top>S2.top
**F.** 若 S1.top 初始化为 0，S2.top 初始化为 99，且两个栈顶指针始终指向各自栈顶元素的下一个位置，则 S1.top+1==S2.top 时，空间满，两栈均无法再入栈
**G.** 若 S1.top 初始化为 0，S2.top 初始化为 99，且两个栈顶指针始终指向各自栈顶元素的下一个位置，则可能出现 S1.top>S2.top

8. 一个初始为空的栈和一个初始为空的队列，都依次接收：

$$1, 2, 3, 4, 5$$

如果只允许进行正常的入栈、出栈、入队、出队操作，关于它们能够产生的输出顺序，下列说法正确的是：
A. 栈和队列都可以产生任意排列
B. 栈只能产生一种输出顺序，队列也只能产生一种输出顺序
C. 队列的输出顺序固定为 $1, 2, 3, 4, 5$，而栈的输出顺序受到栈操作过程限制
D. 栈的输出顺序固定为 $5, 4, 3, 2, 1$，队列的输出顺序受到操作过程限制

9.[多选] 假设输入序列是 1,2,3,4,5，利用两个队列进行出入队操作，不可能的输出序列有 _____

A. 5,4,3,2,1　B. 5,1,2,3,4

C. 5,2,3,1,4　D. 2,3,4,5,1

10. [多选] 与数组实现的循环队列相比，链式队列 _____

A. 优点是进队和出队时间复杂度更低

B. 优点是长度可以在内存限制内动态变化

C. 缺点是节点除数据外需要额外的存储空间存放指针

D. 缺点是不能根据队首和队尾指针以 O(1) 时间算出队列长度

11. 假设某队列采用带头结点的循环双链表实现，第一个数据节点表示队头，最后一个数据节点表示队尾。若只设头指针指向头节点，则进队和出队的时间复杂度分别为：

A. O(1), O(1)

B. O(1), O(n)

C. O(n), O(1)

D. O(n), O(n)

12. （某年 408 真题）现有队列 Q 和栈 S，初始时 Q 中的元素依次是 1,2,3,4,5,6（1 在队头），S 为空。若仅允许下列的 3 种操作：①出队并输出出队元素；②出队并将出队元素入栈；③出栈并输出出栈元素。则不能得到的输出序列是 _____

A. 1,2,5,6,4,3　B. 2,3,4,5,6,1

C. 3,4,5,6,1,2　D. 6,5,4,3,2,1

13. （某年 408 真题）某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若入队序列为 a,b,c,d,e，则不可能得到的出队序列是 _____

A. b,a,c,d,e　B. d,b,a,c,e

C. d,b,c,a,e　D. e,c,b,a,d

14. （某年 408 真题）初始为空的队列 Q 的一端仅能进行入队操作，另外一端既能进行入队操作，又能进行出队操作。若 Q 的入队序列是 1,2,3,4,5，则不能得到的出队序列是 _____

A. 5,4,3,1,2　B. 5,3,1,2,4

C. 4,2,1,3,5　D. 4,1,3,2,5

15. （某年 408 真题）假设栈初始为空，将中缀表达式 a/b+(c*d-e*f)/g 转换为等

价的后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是 _____

A. +(*-　B. +(-*　C. /+(*-*　D. /+-*

16. （某年 408 真题）若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F() 依次执行下述的各步操作：

1) 从 S1 中依次弹出两个操作数 a 和 b；

2) 从 S2 中弹出一个运算符 op；

3) 执行相应的运算 b op a；

4) 将运算结果压入 S1 中。

假定 S1 中的操作数依次是 5,8,3,2（2 在栈顶），S2 中的运算符依次是 *, -, +（+ 在栈顶）。调用 3 次 F() 后，S1 栈顶保存的值是 _____

A. -15　B. 15　C. -20　D. 20

**waypoint-7**

17. C++ 标准库为我们封装好了某个数据结构：stack<>。这是一个栈，你可以在尖括号中定义你想要存放的元素类型，如 stack<int> 表示“元素类型为 int 的栈”。你可以使用如下代码定义一个空的栈 s：

```
/* 类似 “int a;”中 int 是类型，a 是变量名。
   这里 stack<int> 是类型，s 是你定义的变量名。*/
   stack<int> s;
```

stack 对象支持如下操作：

1） s.push(e); // 将值为 e 的元素入栈

2） int t = s.top(); // 返回 s 的栈顶元素的引用（但不弹出该元素），将其值拷贝给 t

3） s.pop(); // 弹出栈顶元素

4） s.empty(); // 若栈 s 为空，此处返回值为 true，否则返回 false。

5） s.size(); // 返回栈 s 中元素的数量

现对于空栈 s，给定如下操作序列，说出每次调用 s.top()、s.size() 时的返回值。

1） s.push(3);

2） s.size(); //

3） s.push(5);

4） s.top(); //

5） s.top(); //

6） s.size(); //

7） s.push(6);

8） s.pop();

9） s.pop();

10） s.size(); //

11） s.top(); //

18. C++ 标准库为我们封装好了某个数据结构：queue<>。这是一个队列，你可以在尖括号中定义你想要存放的元素类型，如 queue<int> 表示“元素类型为 int 的队列”类型。你可以使用如下代码定义一个空的队列 q：

```
/* 类似 “int a;”中 int 是类型，a 是变量名。
   这里 queue<int> 是类型，q 是你定义的变量名。*/
   queue<int> q;
```

queue 对象支持如下操作：

1） q.push(e); // 将值为 e 的元素入队

2） int t = q.front(); // 返回 q 的队首元素的引用（但不将该元素出队），将其值拷贝给 t

3） q.pop(); // 将队首元素出队

4） q.empty(); // 若队列 q 为空，此处返回值为 true，否则返回 false。

5） q.size(); // 返回队列 q 中元素的数量

现有一数组：

```
int arr[5] = {1,2,3,4,5};
```

请你将数组中的五个元素依次入栈，然后从栈顶将所有元素依次出栈，打印每个出栈元素的同时将出栈元素入队，之后从队首依次出队，打印每个出队元素，直至队列为空。请利用上题中介绍的 stack 以及本题中介绍的 queue 实现这个过程。

*19. 某游戏服务器持续记录某位名为 CoreLoop 的玩家的网络延迟情况。现假设服务器连续记录了 $n$ 个时刻的网络延迟，第 $i$ 个时刻的延迟为 delay[i]，这些数值均为正整数。为防止数值闪动，该服务器将时刻 $i$ 的近 $k$ 个时刻延迟的最大值作为该玩家在时刻 $i$ 的**显示延迟**。也就是说，时刻 $i$ 的显示延迟为：

$$max(delay[max(0, i-k+1)], delay[max(0, i-k+2)], ..., delay[i])$$

如，对于：

$$delay[] = 40, 33, 38, 37, 42, 35$$

k=3 时，显示延迟为

$$ans[] = 40, 40, 40, 38, 42, 42$$

k=4 时，显示延迟为

$$ans[] = 40, 40, 40, 40, 42, 42$$

请你设计一个尽可能高效的算法，求出该玩家这 n 个时刻的**显示延迟**。

```
// 将时刻 0 到 n-1 的显示延迟分别保存在 ans[] 数组下标 0 到 n-1 处。
    void MaxDelay(int delay[], int n, int k, int ans[]) {

    }
```

## 3.4　本章小结

实际需求中有许多“先来先处理”或“后来先处理”的需求场景，这一章我们便接触到了这样两种有着不同时间秩序的数据结构—栈和队列。它们是特殊的线性表，因此均可以通过顺序存储实现或链式存储实现。

我们在习题中引入了 C++ 中为我们封装好了的栈和队列的数据结构：stack<> 和 queue<>，这是在上机考试和算法竞赛中非常重要的两个数据结构。

在考试/考研中对这部分内容偏向理论考察，包括顺序以及链式存储下各种操作的原理、时间复杂度等；入队出队、入栈出栈的顺序等。

而机试/算法竞赛中则偏重考察灵活应用栈、队列解决问题的能力，类似“天气预报”问题或习题中的“服务器延迟”问题以及各种比较具有创造性的问题。抑或是用栈或队列实现 DFS、BFS 等算法，我们将在后面的内容中看到。

# 第四章　树与二叉树

有了前面的铺垫，相信你对树与二叉树的基本理论内容已经有了比较深刻的认识，并且对树中的递归代码建立起了一定的感觉。接下来就让我们进入本章中真正有趣（但也有一定难度）的部分，来看一些有实际应用价值的特殊的树。

## 4.1　哈夫曼树与哈夫曼编码

请读者考虑以下最为经典的“合并柴薪”问题：

在初始火焰的熔炉周围，散落着 $n$ 堆柴薪，重量分别为 $w_1, w_2, ..., w_n$，你每次可以搬运任意两堆柴薪 $i$ 和 $j$，将它们合成一堆重量为 $w_i + w_j$ 的新柴薪，这个过程需要消耗的体力为 $w_i + w_j$。显然，$n - 1$ 次搬运之后，就只剩下一堆柴薪了。请问你能否设计最优的合并次序方案，使得耗费的体力最少？并给出消耗的体力值。

例如有 3 堆柴薪，重量分别为：

$$[3, 6, 11]$$

第一次，你将 6 和 11 合并到一起，消耗体力 17，还剩两堆柴薪：

$$[3, 17]$$

第二次，你将 3 和 17 合并到一起，消耗体力 20，只剩一堆柴薪：

$$[20]$$

合并过程一共消耗了 $17 + 20 = 37$ 点体力。显然，若你先合并 3 和 6，则可以达成更少的体力消耗：(3+6)+(9+11)=29 点体力。

我们看一个稍微复杂一点的例子，五堆柴薪重量分别是：3, 6, 2, 3, 4。我们先随意将它们合并，看看过程中会发生什么。合并过程可以如图 4.1 所示：

可以看出，一堆柴薪参与合并后，并不会从后续的合并过程中消失，而是会作为新柴薪的一部分，继续参与后续合并。例如，初始重量为 3 的柴薪与重量为 6 的柴薪合

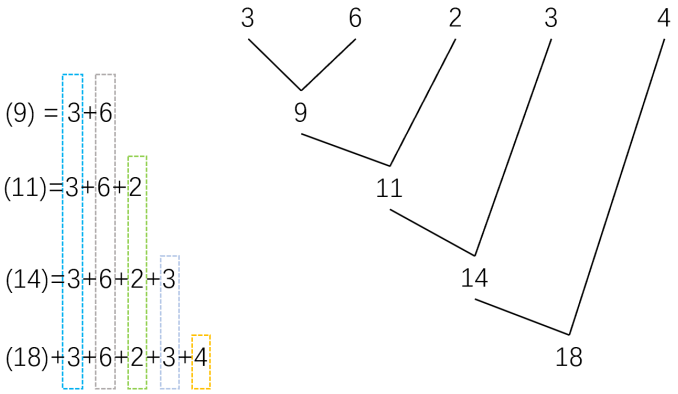



图 4.1 柴薪的合并过程

并后，形成一堆重量为 $9 = 3 + 6$ 的柴薪。虽然新柴薪的重量变成了 9，但其中仍然包含原来重量为 3 的柴薪（9=3+6 嘛）。因此，当这堆重量为 9 的柴薪再次参与合并时，产生的消耗中仍然包含原来那 3 的重量。也就是说：**每堆原始柴薪在每次参与合并时，都会为总体力消耗贡献一次自己的初始重量**。于是，若初始重量为 $w_i$ 的柴薪参与了 $k_i$ 次合并，那么它对总消耗的贡献就是 $w_i k_i$。因此，整个合并过程的总消耗可以表示为

$$W = \sum_{i=1}^{n} w_i k_i$$

另一方面，由图 4.1 可以看出，原始柴薪越靠后参与合并，其总共参与的合并次数就越小，如 4 只参与了 1 次合并，而 3 和 6 却参与了 4 次合并。因此，自然要将原始重量更大的柴薪放在更后面合并，这样其参与的合并次数就小，因而总体造成的体力消耗就更小。

基于以上讨论，很容易设计出最优的合并方案：每次从当前所有柴薪中，选出两堆重量最小的进行合并，直至最后合并完成。对于 3, 6, 2, 3, 4 这五堆柴薪，一种最优的合并方式如图 4.2 所示。

第①步，将最小的 2 和 3 合并，消耗体力 5，合并后新的柴薪：

$$[5, 6, 3, 4]$$

第②步，将这四堆中最小的 3 和 4 合并，消耗体力 7，合并后新的柴薪：

$$[5, 6, 7]$$

第③步，将最小的 5 和 6 合并，消耗体力 11，合并后新的柴薪：

$$[11, 7]$$

第④步，将最小的 (也是仅剩的)11 和 7 合并，消耗体力 18，合并成最终的一堆柴薪：18。可以看出，消耗的总体力 $= 5 + 7 + 11 + 18 = 41$。换个角度，2、3 贡献了 3

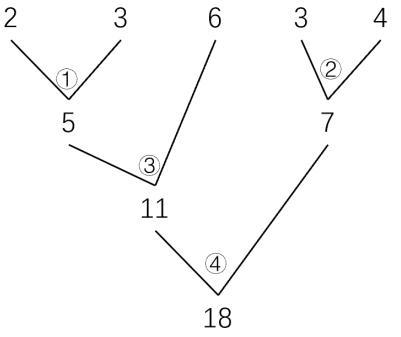



图 4.2 最优合并过程

次自己的初始重量、6 贡献了 2 次，3 和 4 贡献了 2 次，因此 $W = \sum_{i=1}^{n} w_i k_i = (2+3)*3+6*2+(3+4)*2 = 41$。

相信你早已看出来了，合并的过程其实就构成了一棵二叉树，原始的五堆柴薪就是五个叶子节点，而每堆初始柴薪参与的合并次数 $k_i$ 就是该叶子节点到根节点的**路径长度**（即经过的边的数量）。

在以上二叉树中，每个节点的数值代表了要搬动这堆柴薪需要消耗的体力。事实上在许多应用中都要类似现象，我们在许多场景下会将树中的节点赋予一个表示某种意义的数值，称为节点的**权**。从树根到一个节点的路径长度与该节点上权值的乘积，就叫做该节点的**带权路径长度 (WPL)**（可以理解为这堆柴薪在后续合并过程中会贡献的总体力消耗），而树中所有叶子节点的带权路径长度之和就称为这棵树的**带权路径长度 (WPL)**，记为。

$$\mathrm{WPL} = \sum_{i=1}^{n} w_i k_i$$

其中 $w_i$ 是第 $i$ 个叶节点的权值，$k_i$ 是该叶节点到根节点的路径长度。在含有 $n$ 个带权叶节点的二叉树中，带权路径长度最小的二叉树就称为**哈夫曼树**。可以看到在该定义下，“合并柴薪”的问题其实就是求以这 $n$ 堆柴薪为叶节点的哈夫曼树，其带权路径长度是多少。

通过这个例子，我们也理解了“带权路径长度”以及哈夫曼树的应用价值。图 4.2 所示的过程其实也就是由给定的若干个节点，构造一棵哈夫曼树的过程。这里将该过程完整地叙述如下：

1. 初始化。根据给定的 $n$ 个权值，构造 $n$ 棵仅包含一个节点的二叉树，每个节点的权值分别为 $w_1, w_2, \ldots, w_n$。此时，每棵树都只有一个根节点，且该节点同时也是叶节点。

2. 选择与合并。从当前所有二叉树中，选择根节点权值最小的两棵二叉树，将它们合并为一棵新的二叉树。具体而言，以一个新节点作为根节点，将选出的两棵二

叉树分别作为新根节点的左、右子树，并将新根节点的权值设置为这两棵树的根节点权值之和。

3. 更新集合。将刚刚合并得到的新二叉树加入当前的二叉树集合，同时从集合中移除原先被合并的两棵二叉树。这样，集合中的二叉树数量便减少一棵。

4. 重复执行步骤 2 和步骤 3，直到当前集合中只剩下一棵二叉树。

5. 最终剩下的二叉树就是所构造的哈夫曼树。其所有叶节点的带权路径长度之和最小。

需要注意的是，选择权值最小的两棵树进行合并时，若存在权值相同的节点，可以任选符合条件的两棵树。不同的选择可能构造出形态不同的哈夫曼树，但所得的最小带权路径长度相同。

**哈夫曼树的性质**

从以上哈夫曼树的构造过程可以看出哈夫曼树具有如下特点：

1. 最初有 $n$ 个叶节点，合并需要 $n-1$ 步，因此新增共 $n-1$ 个节点。所以：**具有 $n$ 个叶节点的哈夫曼树，其总结点数一定是** $n+n-1=2n-1$。

2. 所有有孩子的节点一定都是由 2 棵子树合并而来，因此**哈夫曼树中不存在度为 1 的节点**。

这两条性质在某种角度其实是相通的。

### 4.1.1　哈夫曼编码

我们知道数字信号是一种二值信号，通过有不同特征的电信号来表示 0 或 1。将字符编码成二进制信号，就可以实现数字信号的传输。如最经典的 ASCII 编码，'A'的编码为 65，传输过程中的数字信号为 01000001，而'a'对应的数字信号为 01100001。这种编码方式是**等长编码**，对所有字符的编码长度相等，都是 8 位，都需要 8 个 bit。但是，实际应用中，各个字符的使用率却大有差异。例如一篇英文新闻报道中，'e'和' '（空格）等字符的数量一定远大于'$'和'x'等字符。

显然，如果我们能够将出现频率更高的字符编码得更短，而频率低的字符编码较长，就可以减少所需的传输空间。

然而这种**不定长编码**也带来了一个新的问题：对方收到之后，如何解码？比如你将 A、B、C 分别编码为 0、1、01，那么收到 0101，应解码为 ABAB 还是 ABC 或 CC？

因此，我们还需要为编码方案增加一个重要的约束：**任何一个字符的编码，都不能是另一个字符编码的前缀**。满足这一条件的编码称为**前缀编码（Prefix Code）**。

**＊ 思维拓展**

在诸多英文文献中，**前缀编码（Prefix Code）**也被称为“Prefix-free Code”，即“无前缀编码”。笔者认为后者在语义上更加直观。

于是问题就变成了：给定 n 个字符 $c_1, c_2, ..., c_n$ 及它们的权重（即字符出现的次数）$w_1, w_2, ..., w_n$，求一种前缀编码方案让总编码长度达到最小。

我们可以借助二叉树来解决这样的编码问题。我们规定从根节点向左走代表数字 0，向右走代表数字 1。如图 4.3 所示，这样，从根节点到任意节点的路径，就可以用一串二进制数字表示。如果我们将每个字符放在一片叶节点上，那么从根节点到该叶节点的路径，就可以作为这个字符的编码，路径长度也即是编码长度。该例中，A 的编码为 000，到根节点的路径长度为 3，即是编码长度；B 的编码为 01，路径长度、编码长度为 2；C 的编码为 1。

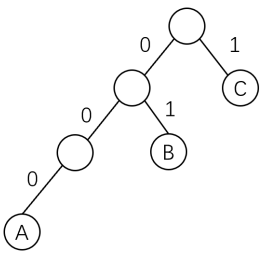



图 4.3 用二叉树进行不定长编码

假设字符 $c_i$ 出现的次数为 $w_i$，其在二叉树中对应叶节点的深度为 $k_i$，则传输所有字符所需的总比特数为

$$L = \sum_{i=1}^{n} w_i k_i$$

而这恰好就是该树的带权路径长度（WPL）。因此，在字符及其出现次数固定的情况下，使总编码长度最小，等价于使二叉树的带权路径长度最小。因此求解该问题，实际上就是构造哈夫曼树的过程。例如，A, B, C, D, E 五个字符出现次数依次是 3, 6, 2, 3, 4，则可以构造如图 4.4 所示的哈夫曼树。

A、B、C、D、E 的编码与长度如下：

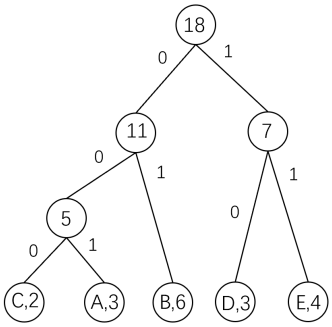



图 4.4 哈夫曼编码

| | 编码值 | 编码长度/路径长度 |
|---|---|---|
| A | 001 | 3 |
| B | 01 | 2 |
| C | 000 | 3 |
| D | 10 | 2 |
| E | 11 | 2 |

同时，由于叶节点不可能是其他节点的祖先，因此，任何一个字符的编码都不会成为另一个字符编码的前缀。这恰好满足了前缀编码的要求。这种由哈夫曼树产生的编码称为**哈夫曼编码**。显然对于相同字符集存在多种哈夫曼编码，但它们的编码长度是相同的。

**提示**

当然，你也可以将向左定义为 1，向右定义为 0。类似的思想已经提示过多次，此处是最后一次了，我保证。

在“合并柴薪”问题中，一堆柴薪参与合并的次数越多，它的重量就会在总代价中被累加越多次；在“字符编码”问题中，一个字符的编码越长，它消耗的传输空间就越多。二者共同的底层逻辑是：**每个对象都有自己的权值，而我们可以通过不同的安排，决定这个权值会被计入最终结果多少次。**

**思考**

对于“合并柴薪”问题，你能否写出一个函数，输入一组重量，输出最优的体力消耗值（只要输出这个数值就行）？

通过上面的思考你也许会发现，解决此类问题并不需要真的构造出一棵哈夫曼树。

编码问题亦是如此，同样不需要在计算机中存储哈夫曼树。事实上，哈夫曼树的意义在于，对于这类问题，它为我们提供了统一的数学模型和统一的求解方法。

还有一个问题，以“合并柴薪”问题为例，我们需要每次从当前所有柴薪堆中选出两个重量最小的，合并后再放回去。使用什么数据结构保存柴薪堆，能使这个操作达到最快？显然若使用普通线性表，对于频繁插入删除的场景，链表一定更好。但无论选择顺序表还是链表，选择两个最小、合并后再插入的时间都需要 O(n)。

我们在接下来的介绍中将看到，如果使用**堆**保存所有柴薪的重量，这个选择加合并后插入的过程可以达到 O(logn)。如果你熟悉 C++，应该已经想到了使用一个 std::priority_queue 来保存所有柴薪堆，实际上，C++ 标准库中的 priority_queue 正是通过**堆**来实现的。

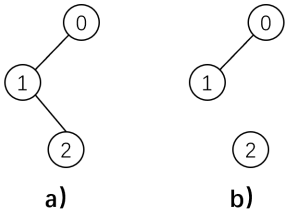
## 4.2　堆

# 此处略去若干内容

## 4.3　并查集

现在考虑这样一个问题：某个游戏的世界地图上，存在 $N$ 个城镇，编号为 0 到 $N-1$，初始时它们彼此之间是不连通的。玩家可能在游玩过程中修建道路，使得两个城镇连通起来。显然对任意的城镇 0、1、2，若 0 与 1 连通，1 与 2 连通，则 0 与 2 一定连通。不妨将处于同一个连通块中的城镇看成是在同一个集合中，则最初有 $N$ 个不同的集合。

现在假设游戏的玩法规定：仅当两个城镇 0 和 1 不连通（不在同一个集合中）时，玩家才可以修建 0 到 1 的道路。如下图 a），0 与 2 已经连通了，因此不允许修建道路 $0 \to 2$，而 b）中 0 与 2 并未连通。




因此当玩家尝试修建某条 A->B 的道路时，程序需要判断 A 与 B 是否在同一个集合中，若是，则拒绝玩家的请求；否则，需要将 A 与 B 所在的两个集合合并成一个集合。

**事实上，在诸多需求中（如，5.5.2 节中将要介绍的 Kruskal 算法）都会有类似的需求场景：**

**1）判断两个元素是否属于同一个集合；**

**2）将两个不同集合合并为同一个集合。**

如何能够高效地实现以上两个操作呢？最直接的思路就是记录每一个元素所属的集合编号，如：

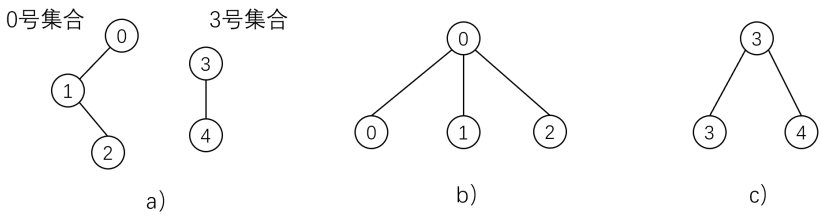
| 城镇 | 0 | 1 | 2 | ··· |
|---|---|---|---|---|
| 集合编号 | 0 | 0 | 2 | ··· |

这样，判断两个元素是否属于同一个集合只需要比较编号即可：

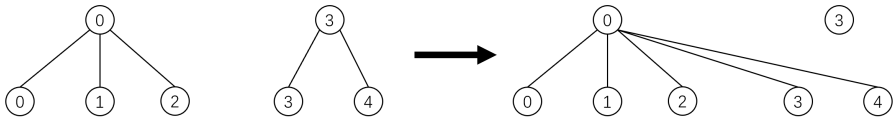
```
bool SameSet(int a, int b) {
    return id[a] == id[b];
}
```

查询操作非常简单，时间复杂度为 $O(1)$。但是，当集合合并时，就要将其中一个集合的所有元素编号都改为另一个集合的编号，若集合元素非常多，则需要大量修改。这种设计查询很快，但合并可能很慢。

事实上，这种设计可以这样理解，将每个集合中其中一个元素的编号作为该集合的编号，如下图 a）所示的两个集合：




0、1、2 这三个元素属于 0 号集合，可以认为它们处在根节点为 0 的树中，如 b）；3、4 这两个元素属于 3 号集合，可以认为它们处在根节点为 3 的树中，如 c）。查找某元素所属集合的过程，就是向上找其根节点的过程；而合并过程实际上是将 3 和 4 这两个元素重新“挂在了 0 上面”，如下图：



这样，若是 3 号集合有十万个元素，就需要更改十万次。

说到这，你想到了什么更好的做法吗？

此处应该有留白……

没错，如果我们直接将 3 号集合的根节点 3 挂在 0 号集合的根节点 0 之下，则无论 3 号集合有多少元素，合并时均只需要一次操作即可实现，如图 4.5 所示：并且当

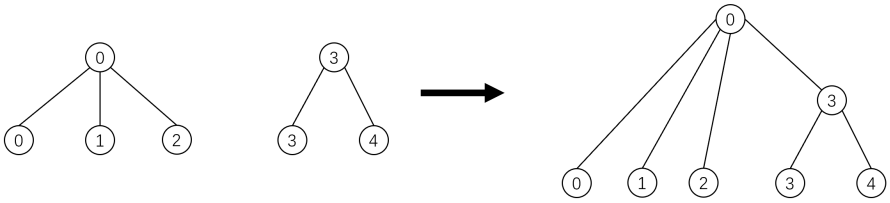



图 4.5

我们要查询 3、4 号节点所属的集合编号时，只需要向上找两次父节点即可，依然可以较为高效地实现。这种用树形结构表示集合，并通过维护节点之间的父子关系来实现集合合并与查询的数据结构，我们将其称之为**并查集（Union-Find）**，这个名字生动地表达了它的功能：并（Union）和查（Find）。

**显然，并查集由若干棵树组成，每棵树的根节点编号表示该树中所有元素所处的集合编号。要查询某节点所在集合，只需要顺着树向上找到根节点即可。而合并操作，仅需将一个集合的根节点作为另一个集合根节点的孩子即可。**图 4.6 展示了一个并查集，该并查集目前有 2 个集合，编号分别为 0 和 6，集合 0 有 0、1、2、3、4 这五个元素，集合 6 有 6、8、5、9、7 这五个元素。

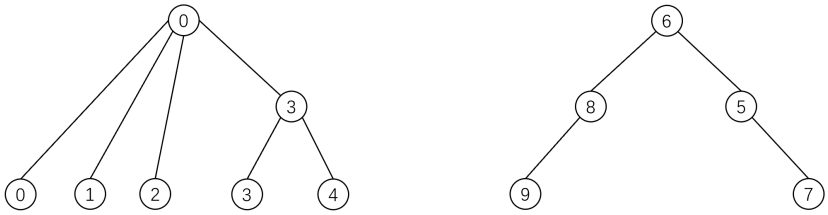



图 4.6 一个并查集示例

并查集最常见的实现方式是通过一个数组实现：

```
int father[N];
```

father[i] 的值为 j 就表示元素 i 在树中的父节点为 j。若 father[i] == i 成立，则表示 i 是这棵树的根节点，也即是该集合的编号。比如图 4.5 合并后的图中有：

```
father[0] = 0;    father[1] = 0;    father[2] = 0;    father[3] = 3;    father[4] = 3;
```

这样，若最初 $N$ 个城镇分别属于 $N$ 个不同集合，则可以做如下初始化：

```
for(int i=0; i<N; i++) {
    father[i] = i;
}
```

在这种设计下，查找和合并操作是非常容易实现的：

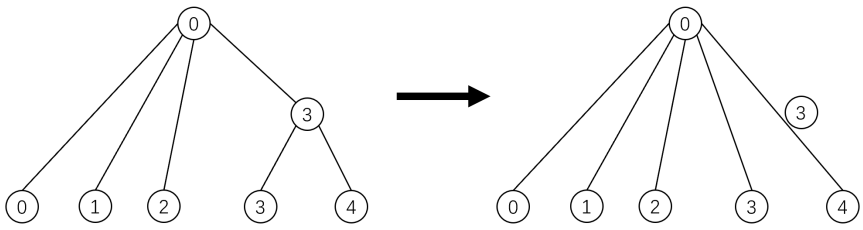
1. 查找操作：顺着节点的 father[] 一路向上找，直到找到了根节点：father[x] == x。
2. 合并操作：要合并 x 和 y 两个元素，首先调用查找操作查找它们的根节点编号（所在集合编号），若相等，说明它们已经在同一个集合中了，无需操作；若不等，将其中一个根节点作为另一个的孩子即可。

两个操作的实现代码如下：

```
// 查找操作：查找元素 x 所属的集合编号 ret
int Find(int father[], int x) {
    int ret = x;
    while(father[ret] != ret) {
        ret = father[ret];
    }
    return ret;
}
// 合并操作：若 x 与 y 不属于同一个集合，将它们所属的集合合并
int Union(int father[], int x, int y) {
    int x_id = Find(father, x); // x 所属集合的 id
    int y_id = Find(father, y); // y 所属集合的 id
    if(x_id != y_id) {
        father[x_id] = y_id;
        return 1;
    }
    return 0;
}
```

### 4.3.1　路径压缩

随着合并次数的增加，树的层数可能越来越高，则查找每个元素的根节点时开销可能越来越大。因此我们可以在合适的时机对树进行调整，将每个元素跳过所有父节点而直接指向根节点，如下图：



这种优化策略被称为**路径压缩**。显然查找元素（Find）时，会一路向上找到根节点，因此在 Find 时顺手对路径上所有进行调整是非常合适的。此外，路径压缩不仅可以出现在独立的 Find 操作中，也会在 Union 调用 Find 时自动发生。因此，路径压缩不需要在 Union 中额外重复实现。

带路径压缩的 Find 函数代码如下：

```
// 查找元素 x 所属的集合编号
int Find(int father[], int x) {
    int ret = x;
    while(father[ret] != ret) {
        ret = father[ret];
    }
    // 路径压缩，将 x 到 ret 路径上的所有节点的 father[] 均指向 ret
    int to_adjust = x;
    while(to_adjust != ret) {
        int fa = father[to_adjust];
        father[to_adjust] = ret;
        to_adjust = fa;
    }
    return ret;
}
```

### 4.3.2　* 按秩合并

事实上很多人可能想到了，在合并两棵树时，将高度更小的树挂在高度更大的树下面一定是更好的，这样查找路径会更短。可是对于带路径压缩的并查集，我们难以对树的高度进行准确的维护。因此，我们对每棵树记录一个数值，表示它“最大达到过多高”，这个数值称为**秩**。

初始时每棵树只有一个节点，因此秩为 1。合并两棵树时：
1）如果两棵树的秩不同，就将秩较小的树挂到秩较大的树下面，新树的秩保持不变。
2）如果两棵树的秩相同，就任选一棵树作为新树的根节点，另一棵树挂到它下面，新树的秩增加 1。

需要注意的是，秩并不一定等于树当前的实际高度。在不使用路径压缩时，按照上述规则，秩等于树的高度；引入路径压缩后，树的实际高度可能下降，但秩不会随之减小。因此秩不是一个精确的指标，而是一个足以支持正确决策、同时又容易维护的指标。

我们通过一个 rank[] 数组记录树的秩，rank[i] 表示元素 i 所在的树的秩。按秩合并的 Uinon 函数实现如下：

```
// 若 x 与 y 不属于同一个集合，将它们所属的集合合并
int Union(int father[], int rank[], int x, int y) {
    int x_id = Find(father, x); // x 所属集合的根节点
    int y_id = Find(father, y); // y 所属集合的根节点

    if(x_id != y_id) {
        // x 所在的树秩更小
        if(rank[x_id] < rank[y_id]) {
            father[x_id] = y_id;
        }
        // y 所在的树秩更小
        else if(rank[x_id] > rank[y_id]) {
            father[y_id] = x_id;
        }
        // 两棵树秩相等
        else {
            father[x_id] = y_id;
            rank[y_id]++;
        }
        return 1;
    }
    return 0;
}
```

在 Find 函数中实现路径压缩、在 Uinon 函数中实现按秩合并，通常可以让并查集始终保持较高效的合并、查找。关于并查集相关操作具体的复杂度分析较为复杂，感兴趣的话请大家参考相关资料。

**习题 2**　（参考答案见习题 2 参考答案，第 155 页）

6. C++ 标准库已经封装好了一个数据结构：priority_queue，这是一个最小堆，可以通过：

```
priority_queue<int, vector<int>, greater<int> > min_heap;
```

定义并初始化一个初始为空的，名为 min_heap 的最小堆，可容纳 int 类型元素。其中

```
priority_queue<int, vector<int>, greater<int> >
```

是一种类型，只不过看起来比较长而已。读者大可不必关注其细节，只知道上述代码用该类型定义的 min_heap 是一个最小堆，并直接使用相关操作即可。min_heap 支持以下操作：

1） min_heap.push(e); // 向堆中插入一个值为 e 的元素；

2） int a = min_heap.top(); // 获取堆顶元素，赋值给 a(但不删除堆顶元素)；

3） min_heap.pop(); // 删除堆顶元素；

4） min_heap.size(); // 返回堆中元素数量；

5） min_heap.empty(); // 若堆为空，返回 true，否则返回 false。

对初始为空的 minHeap 依次执行以下操作，请指出每一次使用 min_heap.top() 获取堆顶元素时，获取到的元素值是多少。

1） min_heap.push(5);

2） min_heap.top(); // 1

3） min_heap.push(3);

4） min_heap.push(4);

5） min_heap.top(); // 2

6） min_heap.pop();

7） min_heap.top(); // 3

7. 请你借助上题给出的 priority_queue 及其相关操作来解决“合并柴薪”问题。输入一个长度为 n 的数组 w[] 表示各堆柴薪的重量（均为正整数），返回一个正整数，表示最优情况下消耗的体力值。

```
int MinCost(int w[], int n) {
    priority_queue<int, vector<int>, greater<int>> min_heap;
    // 在下方完成你的代码
}
```

13. 某大陆散布着多个地下遗迹，这些地下遗迹都可以被建模成一棵二叉树，根节点为遗迹的入口，向下探索，每个节点代表一个房间，所有处于叶节点的房间都能挖出宝藏，weight(weight>0) 表示宝藏的价值。由于遗迹深处的宝藏获取难度更大，某位名为 CoreLoop 的探险者决定权衡风险与收益，将 $\frac{1}{\text{节点深度}} * \text{宝藏价值}$ 定义为房间的“诱人值”。请你实现一个算法，帮 CoreLoop 算出二叉树 T 中诱人值最大的房间诱人值是多少。（根节点深度定义为 1；空树诱人值定义为 0）。

```
double CalcAppeal(node* T) {
    double ans = 0;
    // your codes...

    return ans;
}
```

二叉树的节点定义同上一题所示。你完全可以实现其他函数，并在 CalcAppeal 中调用。

14. 某位名为 CoreLoop 的探险者误入了一扇魔法门从而被传送到了一片地底世界，他只能通过前方的地下城回到大陆。该地下城可以被建模为一棵二叉树，每个节点代表一个房间，根节点表示入口，也是必须经过的房间。节点的 weight(weight>0) 表示进入该房间所需消耗的能量，叶节点代表所有可以回到大陆的出口。CoreLoop 可以沿任何一条路径经出口走出地下城。请你实现一个算法帮助 CoreLoop 求出最优路径（消耗能量最小）下，逃离地下城需要消耗的能量值。root 保证不是空树。

```
struct node {
    int weight;     // 进入房间需要消耗的能量
    node* lchild;
    node* rchild;
};
int MinEscapeCost(node* root) {

}
```

# 第五章　图

深夜，王都的钟声突然响起。

来自边境的信使带来了一个令人不安的消息：早已叛国的碎片君王们正带着大批军队向王国腹地推进，预计将于数月后抵达。国王立即召集所有军事指挥官，要求在天亮前制定出一套防御方案。

王国境内分布着许多城镇、要塞、仓库等据点，它们通过道路相互连接。然而，道路的长短各不相同，并且有些需要穿梭险峻的山脉，有些需要横跨湍急的河流，还有一些因年久失修，早已无法通行。



如何快速确定哪些据点依然互相连通？若某个军事要点来犯，从首都派出军队，走哪条路能以最快速度到达？如何最优地铺设通信电线，以连接所有城镇？

……

时间正在飞速流逝，而摆在指挥官们面前的，是城镇、道路与错综复杂的连接关系。

经过不懈努力，指挥官们制定出了一套行之有效的防御模型。最终，入侵者被尽数驱逐，王国重归安宁。而这套曾经帮助王国渡过危机的模型，并未随着战争的结束而被遗忘。相反，王国的后来者在此基础上不断探索、推演与完善，逐渐建立起一套严谨而完整的理论体系。虽然以上内容都是古老的传闻，但在这片大陆上，你看到的一切都并非虚妄。在你觐见王者—图的途中，这段古老的传说将伴随你一同前行。

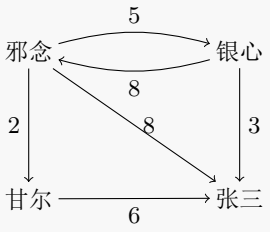
## 5.1　图

经过讨论，指挥官们决定将王国建模为一个**图**。他们将王国中的城镇、要塞等据点作为图的**顶点**，将据点之间的道路作为**边**。

**定义**

图（Graph）是一种由**顶点**集合和顶点之间的**边**集合组成的数据结构，通常记为 $G = (V, E)$。其中，$V$ 是有穷非空的顶点集合，$E$ 是表示顶点之间关系的边集合。根据边是否具有方向，图可分为**无向图**和**有向图**：无向图中的边表示两个顶点之间不区分方向的关系，有向图中的边则表示从一个顶点指向另一个顶点的关系。

在第一章中（—waypoint-5—（第 12 页））我们曾经提到小队队员之间好感度的关系。更现实的情况是 A 对 B 的好感度与 B 对 A 的好感度并不相同，这时就要使用**有向图**，来描述这种从一个顶点指向另一个顶点这种有方向的关系。比如邪念对银心好感度是 5，而银心对邪念的好感度是 8：




王国的指挥官们认为，据点之间的道路若存在，则一定是可以双向通行的，因此他们使用**无向图**来描述王国。为方便交流，他们对无向图做了如下约定：

1. 将某顶点 v 的**度**定义为依附于顶点 v 的边的数量；
2. 在无向图 G 中，若顶点 v 和顶点 w 之间有路径存在，则称这两个顶点是**连通的**。若图 G 中任意两个顶点都是连通的，则称图 G 为**连通图**，否则称为**非连通图**。无向图中的一个极大连通子图称为一个**连通分量**，孤立顶点被认为是一个连通分量。

3. 由于无向图中边是双向的，因此一条边既可以看作出边，也可以看作入边。

王国的后来者对类似的概念，在有向图上做了如下约定：

1. 有向图中，某顶点 v 的度分为**入度**和**出度**。**入度**是“进来到该顶点的边数”，即以顶点 v 为终点的有向边的数量，而**出度**是“从该顶点出去的边数”，即以顶点 v 为起点的有向边的数量。顶点的出度和入度之和称为顶点的**度**。

2. 在有向图 G 中，若从顶点 v 到 w，以及从 w 到 v 之间都有路径，则称这两个顶点是**强连通的**。若图中任意一对顶点都是强连通的，则称该图为**强连通图**。类似地，有向图中的一个极大强连通子图称为一个**强连通分量**，孤立顶点被认为是一个强连通分量。

连通分量和强连通分量通常也被称为**连通块**。

为加深记忆，指挥官们以如图 5.1 的例子进行了一番交流：$G_1$ 是一个有 5 个顶点、4 条边的图，由于它存在两个顶点不连通（如 D 与 A），因此它是一个非连通图；由 ABC 及连通它们的边 $e_1$、$e_2$、$e_3$ 所构成的**子图**，是图 $G_1$ 的一个连通分量，同理 D、E 及 $e_4$ 也构成了一个连通分量。

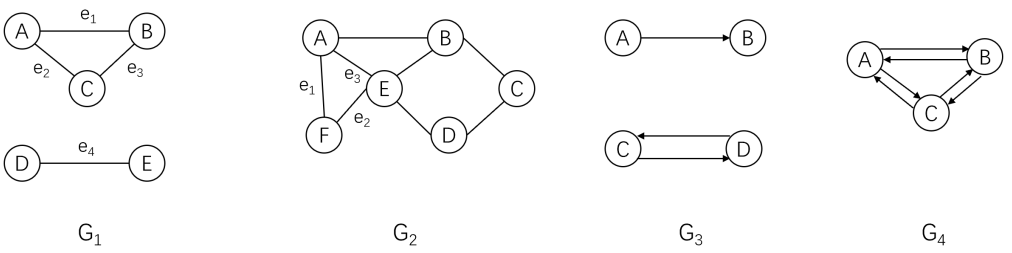



图 5.1

图 $G_2$ 中，A、E、F 及 $e_1$、$e_2$、$e_3$ 构成的子图不能称为一个连通分量，因为它不是**极大**的。

> **极大连通子图**是指：在一个无向图中，不能再加入原图中的其他顶点及相关边而保持连通性的连通子图。有向图中**极大强连通子图**的概念类似。

图 $G_2$ 是一个连通图，因为任何两个顶点之间都有路径。图 $G_2$ 中，顶点 A、B 的度为 3，顶点 E 的度为 4。

图 $G_3$ 是一个有四个顶点、三条有向边的有向图。A 的出度为 1，入度为 0；C、D 的出度为 1，入度为 1。显然图 $G_3$ 不是强连通图。注意：由于 B 到 A 不存在路径，因此顶点 A 与 B 并非强连通。但该图并非只有一个强连通分量，而是有如下的三个：



图 $G_4$ 是一个强连通图，顶点 A、B、C 的出度和入度均为 2；该图只有一个强连通分量，就是它自身。

根据实际应用的不同，顶点和边可能会携带一些信息，通常将边上的某种量化属性称为**边权**。如，指挥官们可能将顶点信息和边的信息分别用如下结构记录：

```
struct VertexInfo {      // 顶点携带的信息
    char* name; // 据点的名字
    int weight; // 该据点的战略价值（重要性）
    //...
};

struct EdgeInfo {    // 边携带的信息
    int weight;      // 边权，可能是两据点间行军需要的时间，也可能是两据点间铺设通信电线需要的花费等等
}
```

这种边上带权的图称为**带权图**。绝大多数场景下，边只需表示顶点之间是否存在连接，或者额外记录一个权值，就足以解决实际问题。因此我们主要讨论带权图以及无权图。对于确实需要利用边的其他属性或更复杂信息的应用场景，我们再根据具体问题进行说明。

王国的城镇、要塞等据点成百上千，道路错综复杂。因此，指挥官们决定将建模输入他们的信息系统来解决问题。

## 5.2　图的存储

要存储一个由 $N$ 个顶点，$M$ 条边构成的图，就需要存储所有顶点的信息、明确“哪些顶点之间存在边”以及存储这些边的权值。要存储这 $N$ 个顶点的信息非常简单，只需要一个数组即可：

```
VertexInfo v[N];
```

那么如何存储边与顶点的关系，以及边上携带的信息？经过研究，两位聪明的指挥官格佛莱和麦丽凯分别提出了两种策略。

### 5.2.1　邻接矩阵

格佛莱指出，若将 $N$ 个顶点编号为 0 到 $N-1$，则通过一个矩阵（二维数组）：

int AdjM[N][N]; // Adjacency Matrix

就可以明确“哪些顶点之间存在边”，以及这些边的权值是多少。例如若 AdjM[0][2] 的值是 7，说明存在一条 0 号顶点到 2 号顶点的边，并且权值是 7。显然对于无向图，AdjM[i][j] 和 AdjM[j][i] 的值一定是相等的，因此无向图的邻接矩阵一定是一个对称矩阵。而对于有向图，AdjM[i][j] 和 AdjM[j][i] 可以分别表示 i 到 j 以及 j 到 i 是否存在边、权值是多少。若边不存在，可以用一个在此场景无意义的数表示权值，如 0、-1 或正无穷等。

对于无权图，则无需记录边的权值，因此简单地用 0、1 表示边是否存在即可。

*** 思维拓展**

CoreLoop 提示：若在你的应用场景下，-1，0，负无穷、正无穷等都是可能存在的有意义边权，则可以在边的信息中加入一个 bool 值，标识这条边是否存在。

EdgeInfo AdjM[N][N]; // EdgeInfo 中包含一个 bool 变量。

以上这种图的存储方式被称为**邻接矩阵**法。使用邻接矩阵法可以将一个图描述如下：

```
// MAX_VNUM 为该结构可存储的图的最大顶点数
    struct MGraph { // M: Matrix
        VertexInfo v[MAX_VNUM] ;
        int AdjM[MAX_VNUM][MAX_VNUM];   // 邻接矩阵
        int current_vnum;   // 当前顶点数
        int current_enum;   // 当前边数
    };
```

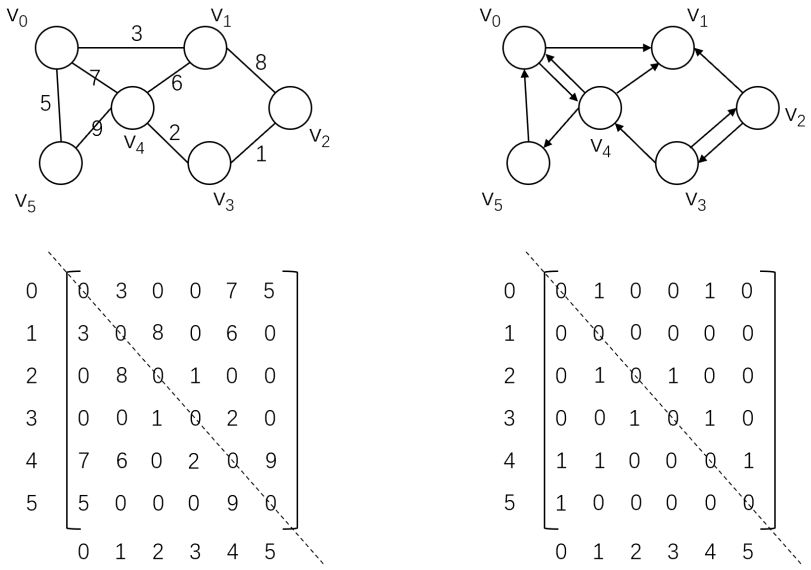
拟用 0 表示边不存在，格佛莱通过图 5.2 展示了两个图（Graph）以及其对应的邻接矩阵 AdjM。

可以看出，a）是对称矩阵。

另一个指挥官麦丽凯研究了格佛莱的提议后，对此法大为认同，但也指出了该方法的一个缺点：该方法存储图需要一个二维矩阵，存储具有 $N$ 个顶点的图，空间复杂度就是 $O(N^2)$。若图存在大量顶点，这种方法在空间上难以接受。经过思考，麦丽凯提出了一种新的方法，后来被称为**邻接表**法。

### 5.2.2　邻接表

邻接矩阵的方法着眼于全局，直接从全局视角记录边与点的关系。而邻接表法则是将每一个顶点作为主角，记录每个顶点的“局部信息”。




a）无向图及其临接矩阵AdjM(边带权)　　b）有向图及其临接矩阵AdjM(边不带权)

图 5.2

具体来说，对于有 $N$ 个顶点（编号为 $0, 1, ..., N-1$）的图 G，邻接表法会存储所有顶点的所有**出边**的信息。如图 5.3 中，对于编号为 2 的顶点 $v_2$，我们需要记录它的三条出边，就需要分别记录这三条边的终点是哪个顶点，以及边权是多少。

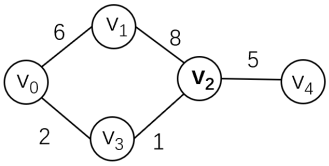



图 5.3

于是我们定义一个结构 AdjEdge，来表示这种“出边”的信息：

```
struct AdjEdge {
    int u;  // 邻接顶点的编号
    int weight; // 当前顶点到 u 的边的权值。若需要更多边的信息，可自定义 EdgeInfo 类型
};
```

由于无法确定每个顶点有多少条出边，因此，—waypoint-6—（第 ?? 页）处介绍的不定长数组 vector 用在此处就非常合适。对于 $v_2$，我们就可以通过一个元素类型为 AdjEdge 的 vector，将其所有出边的信息记录下来：

```
vector<AdjEdge> AdjL_v2;  // 顶点 v2 的所有出边信息
```

这个表就称为 $v_2$ 的邻接表。$v_2$ 的邻接表如图 5.4 所示。

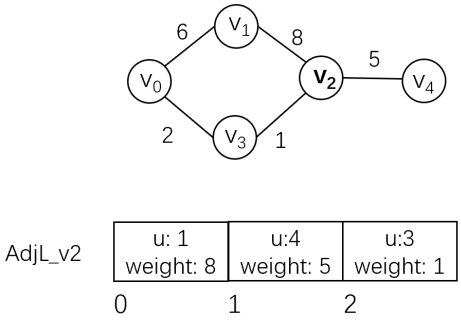



图 5.4

**提示**

事实上很多时候，更好的做法是通过链表来实现邻接表（可以想想为什么），但二者思想是类似的。链表实现复杂，况且大多数情况下 vector 性能已经足够好。为了给大家带来更强的实用性，本章中邻接表拟采用 vector 实现。

某个顶点的邻接表记录了与该顶点有关的所有出边信息。那么我们将图中所有顶点的出边信息都存储下来，也就是为每个顶点记录一个邻接表，一共记录 $N$ 个邻接表，这也就完整地记录了图中所有边的关系：

```
vector<AdjEdge> AdjList[N];
```

这 $N$ 个顶点的邻接表共同构成了整个图 G 的邻接表。注意：以上代码中 AdjList 是一个 C 原生数组，而 AdjList[i] 才是一个 vector，该 vector 是顶点 $i$ 的邻接表，保存的是顶点 $i$ 的所有出边的信息。

于是，使用邻接表存储图中边的信息，我们就可以将一个图表示为如下的结构：

```
struct AdjEdge {
    int u;  // 邻接顶点的编号
    int weight; // 当前顶点到 u 的边的权值。
};
struct LGraph {
    vector<AdjEdge> AdjList[MAX_VNUM];  // 图的邻接表
    VertexInfo v[MAX_VNUM];     // 存放图的顶点信息
```

```
    int current_vnum;           // 图的当前顶点数
    int current_enum;           // 图的当前边数
};
```

由于顶点 v 的邻接表记录的是顶点 v 的**出边**的信息，因此对于无向图，若存在边 $(u,v)$，那么同时需要在 AdjList[u] 中记录 v，并在 AdjList[v] 中记录 u。因此，无向图中的一条实际边通常对应邻接表中的两个表项。

显然对于无权图，AdjEdge 类型只存储邻接顶点的编号 u 即可。

前面的例子中的图及其完整的邻接表如图 5.5 所示。

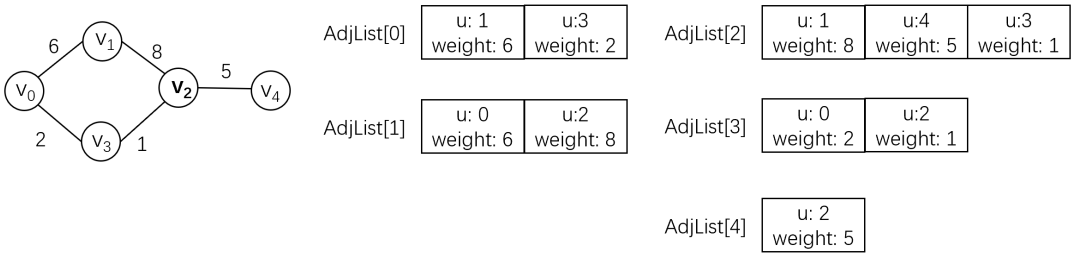



图 5.5

容易知道邻接矩阵表示法下，存储一个图需要的存储空间与图的边数 $M$ 无关。而对于邻接表法，要存储有 $N$ 个顶点、$M$ 条边的图，则需要维护 $N$ 个邻接表，对于无向图，总共需要 $2M$ 个表项，对于有向图总共需要 $M$ 个表项，因此，邻接表法存储一个图需要的存储空间为 $O(N+M)$。

### 5.2.3　二者获取邻接信息的对比

可以看到，无论是邻接表存储法，还是邻接矩阵存储法，对于 $N$ 个顶点的信息存储都是一致的，使用一个额外的数组来存储。二者区别就在于对于每个顶点的邻接信息，即顶点与顶点、顶点与边的关系如何存储。二者存储方式的不同，也带来了一些获取邻接信息上的差异。

① 邻接矩阵

对于有向图，要知道 v 的所有出边信息，遍历一遍图的邻接矩阵的下标为 v 的那一行即可：

```
for(int i=0; i<G.current_vnum; i++) {
    if(G.AdjM[v][i] != 0) {
        // v 有一条到 i 的出边, G.AdjM[v][i] 的值即是该边的权值
```

```
    } else {
        // v 到 i 不存在出边
    }
}
```

类似地要知道 v 的所有入边信息，遍历下标为 v 的那一列即可。显然遍历的过程中若记录有效边的数量，就得到了顶点出度、入度的信息。

对于无向图，只遍历下标为 v 的行或下标为 v 的列即可得到该顶点的所有邻接信息。

```
for(int i=0; i<G.current_vnum, i++) {
    if(G.AdjM[v][i] != 0) {
        // v 到 i 存在无向边，G.AdjM[v][i] 的值即是该边的权值
    } else {
        // v 到 i 不存在边
    }
}
```

同样地，若遍历过程中记录有效边的数量，就得到了无向图某顶点度的信息。

② 邻接表

对于邻接表存储的有向图，v 的邻接表存储的就是 v 的所有出边信息。要知道顶点 v 的所有出边信息很简单，遍历 v 的邻接表即可：

```
for(int i=0; i<G.AdjList[v].size(); i++) {
    // G.AdjList[v][i] 就是 v 的邻接表的第 i 个表项，即与 v 邻接的第 i 个顶点和
    ↪ 边权的信息。
}
```

同时，表长 G.AdjList[v].size() 就是顶点 v 的出度。

但是，对于邻接表存储的有向图，要知道顶点 v 的所有入边信息，就需要将所有顶点的邻接表均查找一遍：

```
// 遍历一遍所有顶点的邻接表
for(int i=0; i<G.current_vnum; i++) {
    // 遍历顶点 i 的邻接表的所有表项
    for(int j=0; j<G.AdjList[i].size(); j++) {
        // 顶点 i 的第 j 个邻接点为 v
        if(G.AdjList[i][j].u == v) {
            // 找到了 v 的一条入边，这是一条从 i 指向 v 的边，权值为
            ↪ G.AdjList[i][j].weight
            // 若需要计算 v 的入度，在此处累加。
```

```
        }
    }
}
```

二者对比如表 5.1 所示：这里 $N$ 表示顶点数，$M$ 表示边数，$\deg^+(v)$ 表示顶点 $v$ 的出度。

表 5.1: 邻接矩阵与邻接表的操作效率对比

| 操作 | 邻接矩阵 | 邻接表 |
|---|---|---|
| 查找顶点 $v$ 的所有出边 | $O(N)$ | $O(\deg^+(v))$ |
| 查找顶点 $v$ 的所有入边 | $O(N)$ | $O(N+M)$ |
| 获取有向图顶点 $v$ 的出度 | $O(N)$ | $O(1)$（直接读取表项数量） |
| 获取有向图顶点 $v$ 的入度 | $O(N)$ | $O(N+M)$（未额外维护入边信息） |

有了以上扎实的模型基础，指挥官们计划了第一步行动。他们决定先从首都派出一名信使，将大敌来犯的消息通知到每一个据点的士兵以及居民们。然而，各据点之间的道路错综复杂，信使不可能毫无章法地四处奔走。怎样规划通知顺序，才能确保每个据点都不会遗漏？

## 5.3　图的遍历

指挥官们给出了两种方式，也是王国的后来者总结出的两种图的遍历方式，总结如下。

### 5.3.1　深度优先搜索（DFS）

深度优先搜索（Depth First Search, DFS）的过程就像是“不分黑白地一股脑儿往前冲，直到撞了墙，才按原路返回，继续探索未探索过的其他路（当然也是继续冲、撞墙、返回，冲、撞墙、返回.....）。”

具体来说，信使从首都（起点，记为 v）出发，访问与 v 邻接且未被访问过的任意一个顶点 u，之后从 u 出发（而非回到 v），再访问与 u 邻接且未被访问过的任意一个顶点 u'，之后从 u'出发，继续访问与 u'邻接且未被访问过的任意一个顶点 u''，.....，如此重复，直至不能继续访问下去之后，依次退回到最近被访问的顶点，若其还有未被访问的邻接顶点，则访问之..... 如此重复，直至图中所有顶点均被访问一遍。

我们以图 5.6 直观地感受一下深度优先搜索的过程。

假设起点为 $v_0$：

a）访问 $v_0$，然后看到与 $v_0$ 邻接的顶点有 $v_1$ 和 $v_3$，他们都未被访问，任意选一个访问即可。假设选 $v_1$。

b）访问 $v_1$，然后看到邻接顶点仅有 $v_2$ 未被访问，于是

c）访问 $v_2$，看到 $v_4$、$v_5$ 均未被访问，假设选 $v_4$，于是

d）访问 $v_4$，此时已深入最深处，与 $v_4$ 邻接的所有顶点均已访问完成，于是退回到上一个顶点 $v_2$，发现它还有 $v_3$、$v_5$ 两个邻接顶点未访问，任选一个，假设选 $v_3$。

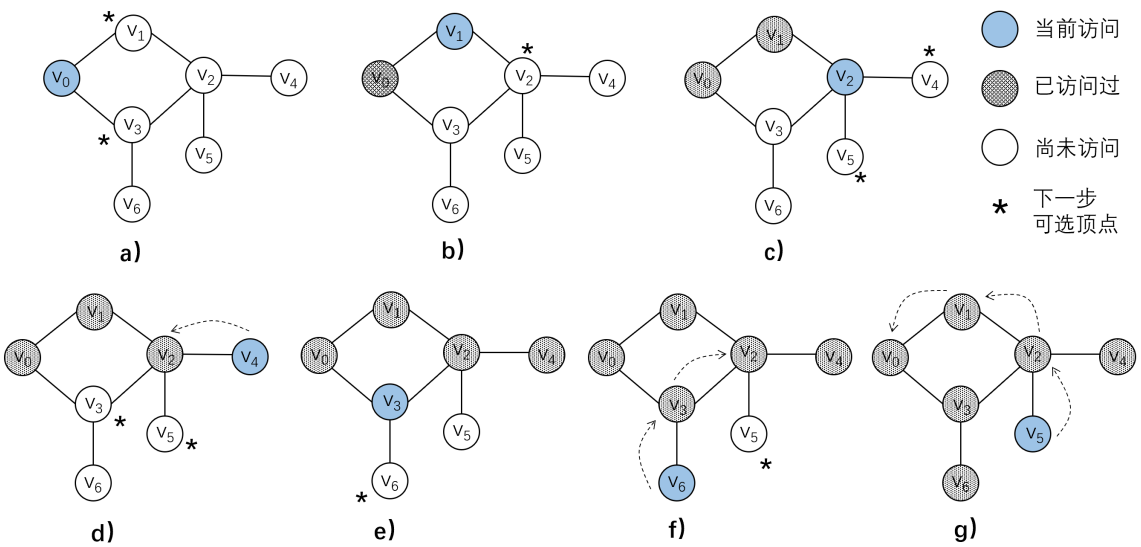



图 5.6 深度优先搜索

e）访问 $v_3$，看到邻接的 $v_6$ 还没被访问。于是

f）访问 $v_6$，此时无法继续访问下去，于是退回到 $v_3$，发现 $v_3$ 的所有邻接顶点也都已经访问完成了，于是退回 $v_2$，发现与 $v_2$ 邻接的 $v_5$ 还未访问。于是

g）访问 $v_5$。之后退回到 $v_2$（无顶点未访问），$v_1$（无顶点未访问）、$v_0$（无顶点未访问），深度优先搜索过程结束。

以上过程得到的深度优先访问序列为：$\{v_0, v_1, v_2, v_4, v_3, v_6, v_5\}$。这就是深度优先搜索，总是尽可能“深”地探索，当无法再深入之后才原路返回，探索那些未走过的路。

### 5.3.2　广度优先搜索（BFS）

如果说深度优先搜索（DFS）是“一路走到底，不撞南墙不回头”，那么广度优先搜索（Breadth First Search, BFS）就是“浅尝辄止，先慢慢探清周围环境，再继续往深处走”。

实际上广度优先搜索和树的层序遍历非常相似。可以想象平静的水面落下一个小铁球而泛起层层涟漪，广度优先搜索即是先访问中心点，之后访问距中心点最近的“第一层涟漪”，之后访问“第二层涟漪”，第三层……，直到全部访问完成。

具体来说，假设从 v 出发，首先访问起始顶点 v，并“发现”v 的所有未被访问的邻接顶点 $v_1, v_2, ..., v_n$，这些顶点即是“第二层涟漪”，依次访问它们，访问时，又会发现第三层涟漪 $v_{11}, v_{12}, ..., v_{nm}$…… 如此往复，直至图中所有顶点均被访问一遍。

我们以图 5.7 来直观地感受一下广度优先搜索的过程。拟以 $v_0$ 作为起点：

a）访问 $v_0$，看到 $v_0$ 的邻接顶点有 $v_1$ 和 $v_3$，于是将这两个顶点作为“第二层”的顶点，此时“第一层涟漪”已访问完毕，“第二层涟漪”有两个，任选一个，假设选择 $v_1$；

b）访问 $v_1$，此时发现了“第三层涟漪”的一个顶点 $v_2$，先将它记下来。准备继续访问第二层未访问完的顶点 $v_3$。

c）访问 $v_3$，发现 $v_3$ 的邻接顶点 $v_6$ 尚未被访问，因此也将它记下来，作为“第三层涟漪”的一部分。至此，第二层顶点均已访问完毕，准备访问第三层，假设先选择 $v_2$；

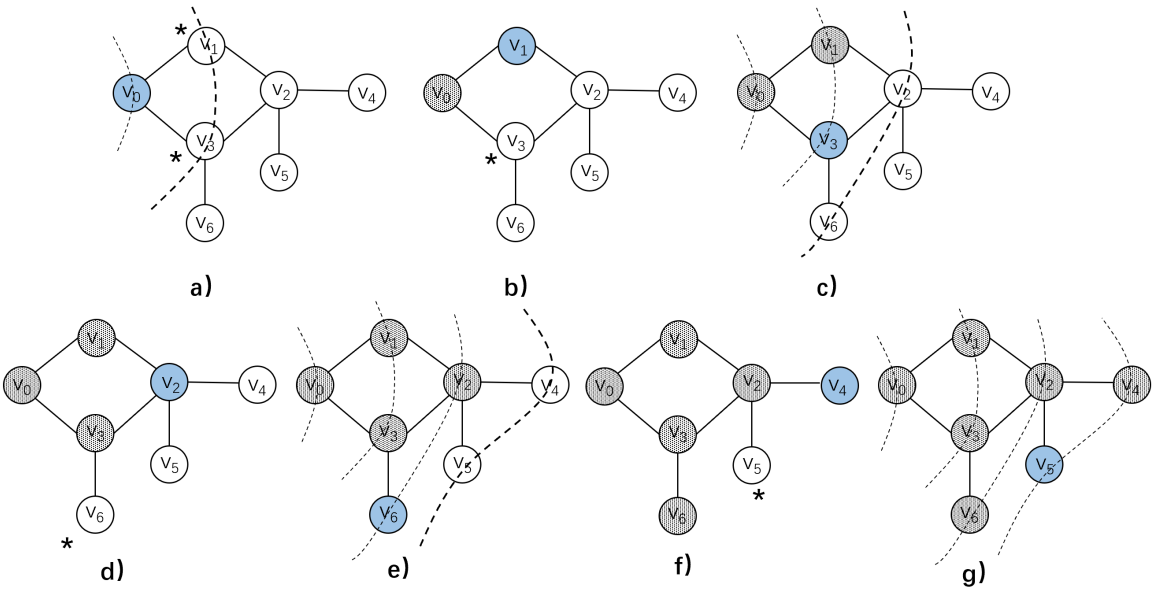



图 5.7 广度优先搜索

d）访问 $v_2$，发现与 $v_2$ 邻接的有 $v_4$ 和 $v_5$ 还未访问，因此将它们记下来，作为“第四层涟漪”的一部分；然后准备访问第三层剩下的 $v_6$；

e）访问 $v_6$，$v_6$ 没有尚未访问的邻接顶点，因此从 $v_6$ 没有发现新的“第四层涟漪”。至此第三层均已访问完毕。准备开始访问第四层；

f）访问 $v_4$，没发现新的顶点；

g）访问 $v_5$，没发现新的顶点。此时所有顶点均已访问过一遍，没有新的层了，结束。

以上过程得到的广度优先搜索访问序列是 $\{v_0, v_1, v_3, v_2, v_6, v_4, v_5\}$。这就是广度优先搜索，总是先“摸清楚周围的环境，再继续向更深处进发”。BFS 算法可以求解无权图或等权图的最短路径问题，我们将在习题 6 第 16、17 题中详细介绍。

至此，“深”和“广”的语义已经很明确，“深度优先”总是冲到最“深”处，而“广度优先”总是先向“广”阔的四周探索。

### 5.3.3　DFS 和 BFS 的实现

可以看到，广度优先搜索（BFS）总是“越早看见的顶点越先被访问”（因为层数小），而深度优先搜索（DFS）总是“越晚看见的顶点越先被访问”（因为一路往前冲而不是回头）。这正与队列和栈的“先进先出”与“后进先出”的特点相符合。

如果你研究过—waypoint-3—（第 38 页）处的问题，可以进一步理解：实际上广度优先（BFS）的过程中，“顶点们”成为了“仍然等待某事发生，并且优先处理先来者的数据”，而深度优先（DFS）的过程中，“顶点们”则是“仍然等待某事发生，并且优先处理后来者的数据”：

1. 在广度优先（BFS）中，顶点们等待被访问，并且先“被看见”的先访问；
2. 而在深度优先（DFS）中，顶点们等待被访问，并且后“被看见”的先访问。

有了以上思维过程，我们就可以非常清晰地给出二者的代码实现过程。

**深度优先搜索（DFS）的实现**

创建一个栈 s，访问起点，并将起点的所有邻接且尚未被访问过的顶点入栈。之后：

1. 若栈空，说明没有待访问的顶点了，算法结束，否则，进入第二步；
2. 从栈顶取出一个顶点，访问之，并将该顶点的所有邻接且未被访问过的顶点入栈；回到第一步。

如此循环直至栈空，则所有顶点均被访问一遍。

如何确定某顶点是否已经访问过？最常见的办法是维护一个一维数组：

```
1 bool visited[MAX_VNUM]; // 所有元素初始为 false，表示还未访问
```

visited[i] 为 true 则表示已经访问过了。

以邻接矩阵存储的图为例，深度优先搜索的代码如下所示：

```
/*************** 邻接矩阵结构定义 ***************/
// // MAX_VNUM 为该结构可存储的图的最大顶点数
//     struct MGraph {
//         VertexInfo v[MAX_VNUM] ;
//         int AdjM[MAX_VNUM][MAX_VNUM];
//         int current_vnum;   // 当前顶点数
//         int current_enum;   // 当前边数
//     };

// 输入图 G 以及起点编号 start_idx，以 DFS 方式对每一个顶点执行某函数 visit()
void DFS(MGraph G, int start_idx) {
    bool visited[G.current_vnum];
    // 初始化 visited[] 数组
    for(int i=0; i<G.current_vnum; i++) {
        visited[i] = 0;
    }

    stack<int> s;   // 栈中存储顶点编号即可
    s.push(start_idx);
    visited[start_idx] = true;
    while(!s.empty()) {
        // 访问栈顶顶点 v
        int v = s.top();
        s.pop();
        visit(G.v[v]);   // 访问 v

        // 将与 v 邻接的所有尚未访问的顶点入栈
        for(int i=0; i<G.current_vnum; i++) {
            if(G.AdjM[v][i] != 0 && visited[i] == false) {
                s.push(i);
                visited[i] = true;
            }
        }
    } // end while
}
```

若图是通过邻接表存储的，与邻接矩阵存储的代码有什么区别呢？其实仅在寻找 v 的所有邻接点时，代码有区别。邻接表版本代码如下：

```
/************* 邻接表结构定义 *************/
// struct AdjEdge {
//     int u;  // 邻接顶点的编号
//     int weight; // 当前顶点到 u 的边的权值。
// };
// struct LGraph {
//     vector<AdjEdge> AdjList[MAX_VNUM];  // 邻接表
```

```
//     VertexInfo v[MAX_VNUM];     // 存放顶点信息
//     int current_vnum;           // 当前顶点数
//     int current_enum;           // 当前边数
// };

// 输入图 G 以及起点编号 start_idx，以 DFS 方式对每一个顶点执行某函数 visit()
    void DFS(LGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
            visited[i] = 0;
        }

        stack<int> s;   // 栈中存储顶点编号即可
        s.push(start_idx);
        visited[start_idx] = true;
        while(!s.empty()) {
            // 访问栈顶顶点 v
            int v = s.top();
            s.pop();
            visit(G.v[v]);   // 访问 v

            // 将与 v 邻接的所有尚未访问的顶点入栈
            for(int i=0; i<G.AdjList[v].size(); i++) {
                // G.AdjList[v] 是 v 的邻接表，G.AdjList[v][i] 是该表中第 i 个
                ↪ 元素
                int u = G.AdjList[v][i].u;
                if(visited[u] == false) {
                    s.push(u);
                    visited[u] = true;
                }
            }
        } // end while

    }
```

### 广度优先搜索（BFS）的实现

根据前面的思维过程很容易知道，将深度优先搜索代码中的栈换成队列即是广度优先搜索。邻接矩阵版：

```
// 输入图 G 以及起点编号 start_idx，以 BFS 方式对每一个顶点执行某函数 visit()
    void BFS(MGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
```

```
            visited[i] = 0;
        }

        queue<int> q;   // 队列中存储顶点编号即可
        q.push(start_idx);
        visited[start_idx] = true;
        while(!q.empty()) {
            // 访问队首顶点 v
            int v = q.front();
            q.pop();
            visit(G.v[v]);   // 访问 v

            // 将与 v 邻接的所有尚未访问的顶点入队
            for(int i=0; i<G.current_vnum; i++) {
                if(G.AdjM[v][i] != 0 && visited[i] == false) {
                    q.push(i);
                    visited[i] = true;
                }
            }
        } // end while
    }
```

邻接表版：

```
// 输入图 G 以及起点编号 start_idx，以 BFS 方式对每一个顶点执行某函数 visit()
    void BFS(LGraph G, int start_idx) {
        bool visited[G.current_vnum];
        // 初始化 visited[] 数组
        for(int i=0; i<G.current_vnum; i++) {
            visited[i] = 0;
        }

        queue<int> q;   // 队列中存储顶点编号即可
        q.push(start_idx);
        visited[start_idx] = true;
        while(!q.empty()) {
            // 访问队首顶点 v
            int v = q.front();
            q.pop();
            visit(G.v[v]);   // 访问 v

            // 将与 v 邻接的所有尚未访问的顶点入队
            for(int i=0; i<G.AdjList[v].size(); i++) {
                // G.AdjList[v] 是 v 的邻接表，G.AdjList[v][i] 是该表中第 i 个
                ↪ 元素
                int u = G.AdjList[v][i].u;
```

```
                if(visited[u] == false) {
                    q.push(u);
                    visited[u] = true;
                }
            }
        } // end while
    }
```

上面代码中使用到的 stack 和 queue 是 C++ 标准库为我们提供的栈和队列的数据结构。我们曾在—waypoint-7—（第 45 页）处介绍过。

**思考**

你也许会有疑问，将 visited[i] 标记为 true 的时机，难道不应该是访问 i 之后吗？代码中怎么将 i 入队（还未访问）时就将 visited[i] 标记为 true 了？

访问后标记当然是正确的做法。但事实上，入队时即标记同样是正确的，并且是更优的做法。你可以想一想，为什么正确？为什么更优？（可以结合图 5.7 的搜索过程思考）。

以上便是图最常见的两种遍历方式：广度优先搜索和深度优先搜索。信使任选一种方式，即可确保每个据点都不会遗漏。但是，信使很快发现了这样一个问题，如图 5.8 所示，信使走到据点 A 后，发现通往 D、E 的桥梁已经倒塌，而他无论如何也找不到其他路通往 D、E 这两个城镇。

信使回去汇报后，指挥官们发现类似的**非连通**区域还有很多，于是立即快马加鞭地组织人手修路搭桥，将王国地图修复成了一个**连通图**。

那么，若现实中某图存在从起点出发不可达的顶点，即图是一个非连通图，该如何遍历它的所有顶点呢？其实只需要对图中每一个顶点均执行一次上面的 DFS 或 BFS 即可，以图 5.8 为例，伪代码如下：

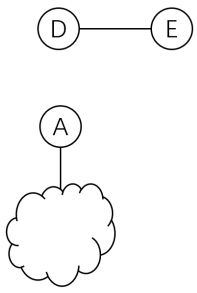



图 5.8 无法抵达对岸的城镇

```
// 假设 G 已经存储了图 5.8 所示的图
Graph G;
for(int i=0; i<N; i++) {
    if(!visited[i]) {
        DFS(G, i);
    }
```

```
}
```

以上代码会对 G 中所有顶点（包括顶点 D、E、A 等）依次判断，若未访问过，则对其执行 DFS。显然，当首次对 A 所在的连通块执行 DFS 时，会将其中所有顶点标记为已访问，之后循环中判断时，则不会对连通块中的其他顶点执行 DFS 了。D 和 E 所在的连通块有类似的情况。

以上两种遍历的实现都是非递归实现。事实上，由于深度优先搜索的特性，通过递归的方式实现深度优先搜索是非常简单的：

```
bool visited[N];    // 初始化为 false
void DFS(int v) {
    visit(v);   // 访问 v
    visited[v] = true;
    // 递归地访问 v 的邻接点，适配邻接矩阵或邻接表就行了。
    for(v 的每一个邻接点 u) {
        if(!visited[u]) {
            DFS(u);
        }
    }
}
```

要遍历非连通图，分别以每一个顶点为起点调用以上函数即可：

```
void Travel() {
    for(int i=0; i<N; i++) {
        DFS(i);
    }
}
```

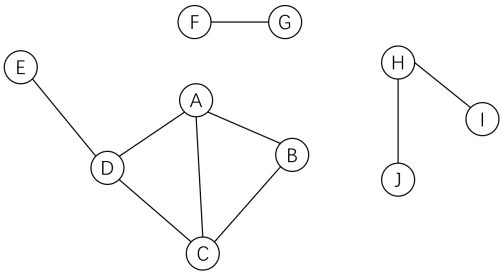
由于递归调用过程中，操作系统要开出很多个函数栈，因此递归的方式往往空间复杂度更大，但递归算法在很多情况下也是有用的。

根据指挥官给出的方法，信使无一疏漏地将大敌来犯的消息传遍了王国的每一座城池、每一座要塞。士兵和居民们纷纷武装起来，布起防御系统，准备迎接那场即将到来的大战。

# — BONFIRE ENCOUNTERED —

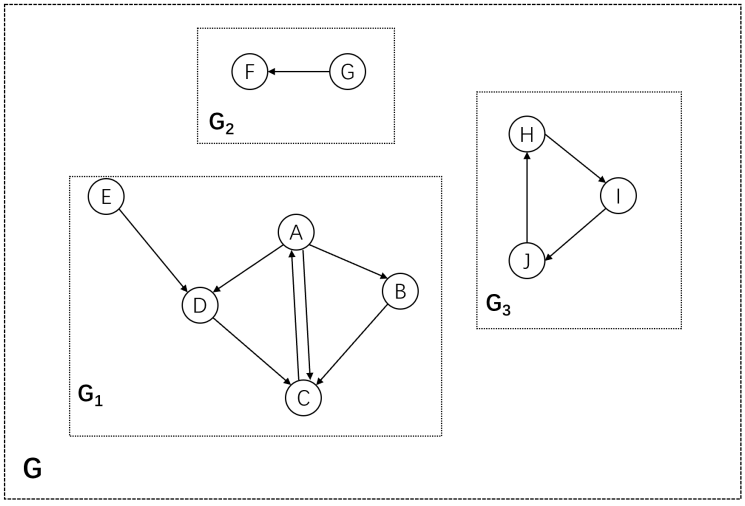
**习题 3** （参考答案见习题 3 参考答案，第 158 页）

1. 关于下图所示的无向图，下列说法错误的是_____




A. 顶点 A 的度为 3
B. 顶点 A 与顶点 E 是不连通的
C. 该图有 3 个连通分量
D. 对于顶点 H 所在的连通分量，若以 H 为起点，其深度优先搜索序列可以是 {H, I, J}

2. 关于下图所示的有向图 $G$ 以及其三个子图 $G_1, G_2, G_3$，下列说法错误的是_____



A. 三个子图 $G_1, G_2, G_3$ 中，仅 $G_3$ 是强连通的
B. 图 G 的强连通分量数大于 3
C. 顶点 A 的出度为 3
D. 顶点 F 的出度为 1

3. （某年 408 真题）下列关于无向连通图特性的叙述中，正确的是_____。
I. 所有顶点的度之和为偶数。
II. 边数大于顶点个数减 1。
III. 至少有一个顶点的度为 1。
A. 只有 I　B. 只有 II　C. I 和 II　D. I 和 III

4. 思考：有 $N$ 个顶点的无向图，至多有多少条边？（不允许存在自环和重复边）。

5. 思考：有 $N$ 个顶点的无向图，要保证它是连通图，至少要有多少条边？

6. （某年 408 真题）若无向图 $G=(V,E)$ 有 7 个顶点，无边。现在要向图 $G$ 中添加若干条边。请问，要保证无论怎样添加，添加 $N$ 条边后图一定是连通的，则 $N$ 最小是_____

A. 6　B. 15　C. 16　D. 21

7. 设某无向图的邻接矩阵 A 如下所示，若 0 表示边不存在，则各顶点的度依次是_____

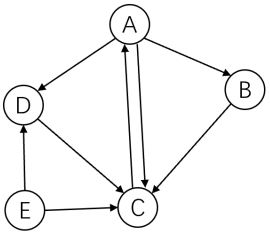
$$A=\begin{bmatrix}0&1&0&1\\1&0&1&1\\0&1&0&0\\1&1&0&0\end{bmatrix}$$

A. 2,3,1,2　B. 4,6,2,4　C. 0,1,2,3　D. 999,1000,1001,7820

8. 若上题的矩阵 A 表示的是某有向图的邻接矩阵，则各顶点的度依次是_____
提示：有向图中顶点的度定义为该顶点的入度和出度之和。

A. 2,3,1,2　B. 4,6,2,4　C. 0,1,2,3　D. 999,1000,1001,7820

9. 如下图所示的图 $G$ 若使用邻接表存储，顶点 A 和 C 的邻接表表长分别是多少？
_____



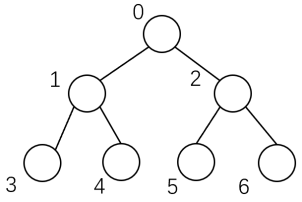

A. 3,3　B. 1,4　C. 3,1　D. 4,4

10.　对于上题图以顶点 E 为起点进行深度优先搜索，不可能得到的搜索序列是_____

A. {E,D,C,A,B}　B. {E,C,A,D,B}

C. {E,C,A,B,D}　D. {E,C,D,A,B}

11. 对于下图所示的图 $G$，以 0 号顶点为起点进行广度优先搜索，不可能得到的序列是_____




A. {0,1,2,3,4,5,6}　B. {0,2,1,6,5,4,3}

C. {0,3,4,5,6,1,2}　D. {0,1,2,3,4,6,5}

12. 关于图的两种遍历算法：深度优先搜索（DFS）和广度优先搜索（BFS），下列说法错误的是_____

A. 深度优先搜索通常通过队列实现，而广度优先搜索通常通过栈实现
B. 对于同一个图，深度优先搜索、广度优先搜索序列可能不唯一，取决于具体实现
C. 深度优先搜索能够访问到起点所在连通块的每一个顶点，不会遗漏
D. 广度优先搜索能够访问到起点所在连通块的每一个顶点，不会遗漏

13. 对于有 N 个顶点，未知边数的无向无权图 $G$, 分别编写以下两个函数：

1）若图 G 采用邻接矩阵存储（0 表示无边，1 表示有边），求其所有顶点的度数之和：

```
// 假设该数组已经正确存储了 G 的邻接矩阵
bool AdjM[N][N];
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
}
```

2）若图 G 采用邻接表存储，求其所有顶点的度数之和；

```
// 该数组已经正确存储了 G 的邻接表
vector<int> AdjList[N];
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
}
```

随意画出一个无向图，脑中模拟你代码的运行过程并计算出结果，你发现了什么规律吗？对于有向图有类似的规律吗？

【14、15、16、17、18 题基于此背景】传说在《前有数据结构》大陆某处神秘的角落有一片森林，其中有一棵樱花树无时不刻地施展着她的魔法，为世人祈福，带来幸运。可是该魔法也无时不刻地侵蚀着她自身，这些痛苦会凝聚成泪滴积于花瓣之上，将其饮下，便能抚平樱树的悲怆，化作自身的庇佑。

火光摇曳中，你在帐篷中沉睡过去。维火女凝视着你满身的疲惫与疮痍，眼底盛满了心疼与依恋。她不再忍心见你独自背负这苦难，便悄然别过营火，孤身涉足那片未知的森林，去寻找并带回那代表着祝福与庇佑的樱花之泪。

天亮了。维火女已到达了那片森林。你通过火焰之环看到了那片森林的地图。森林中有 N(N<=100) 棵树木，编号为 0 到 N-1，一些树木之间通过双向的灵魂通路相连，而有些树木则无法抵达。

14. 维火女从编号为 0 的树木处出发，试图探索可达的所有树木，以找到那具有魔法的樱花树。假设地图采用邻接表存储，请你编写一个递归版本的 DFS 函数，计算维火女从起点 0 出发，总共可以访问到的可达树木有多少棵？（包含起点本身）。

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

```

```
// AdjList[i] 已存放编号为 i 的顶点的邻接表，邻接表元素类型 int，含义为与 i 邻接的顶点编号
int CountAccessibleTrees(vector<int> AdjList[N]) {

}
```

15. 假设地图采用邻接表存储，请你用非递归的 DFS 或 BFS 计算，这片森林有多少个连通块？

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

// AdjList[i] 已存放编号为 i 的顶点的邻接表，邻接表元素类型 int，含义为与 i 邻接的顶点编号
int CountConnectedComponents(vector<int> AdjList[N]) {

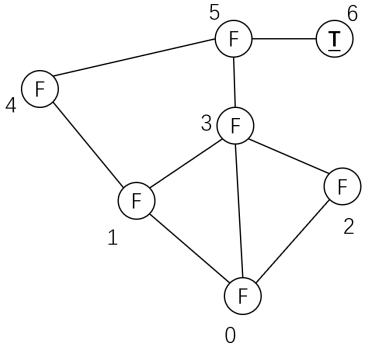
}
```

*16. N 棵树木的信息已存储于一个 char 类型的数组中：

```
char trees[N];
```

若某棵树是普通树木，其值为'F', 若它是那棵魔法樱树，则其值为'T'。假设地图采用**邻接矩阵**存储，维火女从起点 u 出发，能否抵达樱花树？若能，输出一条可行的路径，否则输出“.....”。

以下图地图为例，若起点 u 为 0，你可以输出“01356”、“0356”或“0231456”等，但不能输出“0456”、“06”等等不合法的路径；若起点 u 不在此连通块，输出“.....”。



```
const int N/* = 7 */;

char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;

void FindPath(int u) {

}
```

*17. 若上题条件下，假设所有灵魂通路的通行时间**相等**，当樱花树可以抵达时，要求输出所有可行路径中距离最短的一条（经过的边数最少的），不可抵达时同样输出“.....”，请写出代码。

若有多条路径最短，任意输出一条即可。

*18. 若 16 题条件下，假设所有灵魂通路的通行时间**不等**，当樱花树可以抵达时，要求输出所有可行路径中距离最短的一条（花费时间最少的），不可抵达时同样输出“.....”，请写出代码。

若有多条路径最短，任意输出一条即可。邻接矩阵中已存储了各灵魂通路的通行时间。

```
const int N/* = 7 */;

char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵，0 表示无路，非 0 表示通行时间
    { 0, 3, 2, 2, 0, 0, 0 },
    { 3, 0, 0, 2, 5, 0, 0 },
    { 2, 0, 0, 1, 0, 0, 0 },
    { 2, 2, 5, 0, 0, 6, 0 },
    { 0, 1, 0, 0, 0, 2, 0 },
    { 0, 0, 0, 6, 2, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;

void FindPath(int u) {
```

```
15
16 }
```

19.（某年 408 真题）已知无向连通图存在 EL 路径的充要条件是度为奇数的顶点数为 0 或 2。设无向连通图 $G$ 采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numVertices, numEdges;  // 图中实际的顶点数和边数
    char VerticesList[MAXV];    // 顶点表。MAXV 为已定义常量
    int Matrix[MAXV][MAXV];       // 邻接矩阵
}MGraph;
```

若顶点 $i$ 到 $j$ 存在边，则 Edge[i][j] 为 1，否则为 0。请设计算法 int IsExistEL(MGraph G)，判断 $G$ 是否存在 EL 路径，若存在，返回 1，否则返回 0。要求：
1）给出算法的基本设计思想；
2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释；
3）说明你所设计算法的时间和空间复杂度。

20.（某年 408 真题）已知通常将有向图中出度大于入度的顶点称为 K 顶点。若图采用邻接矩阵存储，类型定义如下：

```
typedef struct {
    int numVertices, numEdges;  // 图的顶点数和有向边数
    char VerticesList[MAXV];    // 顶点表，MAXV 为已定义常量
    int Matrix[MAXV][MAXV];       // 邻接矩阵
}MGraph;
```

若顶点 $i$ 到 $j$ 存在边，则 Edge[i][j] 为 1，否则为 0。请你设计一个算法 int PrintKVertices(MGraph G)，输出 G 中的所有 K 顶点，并返回 K 顶点的个数。要求：
1）给出算法的基本设计思想；
2）根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。

## 5.4　最短路径

王国的故事还在继续。根据指挥官给出的遍历方法，信使无一疏漏地将大敌来犯的消息传遍了王国的每一座城池、每一座要塞。士兵和居民们纷纷武装起来，布起防御系统，准备迎接那场即将到来的大战。

### 5.4.1　迪杰斯特拉算法

接下来，一位指挥官提出了这样一个问题：若某个据点有重兵来犯，从首都派出军队去该据点支援，走哪条路，能够最快到达？这种求解从某顶点到图中其他所有顶点的最短路径的问题，后来被称为**单源最短路径**问题。

为了解决这个问题，指挥官们将王国建模成了一个无向带权图，边的权表示走此路径需要消耗的时间。如图 5.9 所示。

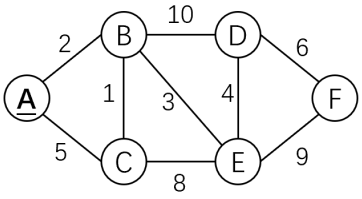



图 5.9

假设 A 为首都，如何求 A 到 F 的最短路径？一位指挥官指出，最简单的做法就是把所有可能的路径都找出来，然后比较它们的长度。这种做法通过 DFS 进行路径搜索即可实现，如**习题 6 第 18 题**（第 86 页）所做的那样。但是，这种做法存在一个明显的问题：即使其中许多路径已经不可能成为最短路径，我们仍然要花费时间去搜索它们。对图 5.9 进行 DFS 路径搜索的搜索树如图 5.10 所示。可以看到每个叶节点即代表了一条路径的终点，要么是找到了目标顶点，要么是非目标顶点的死路。

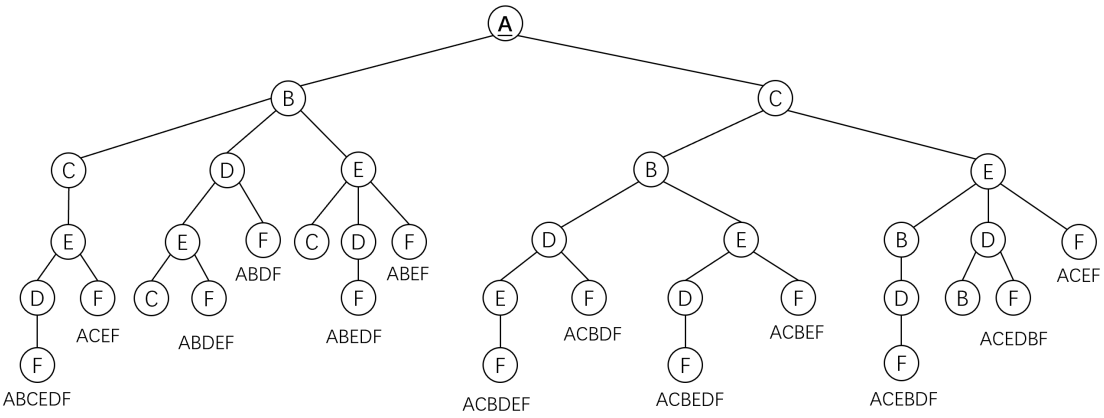



图 5.10 搜索树

当我们已经探索到一条可行的路径 ABEF 时，就已经知道最短路径不可能大于 14，

于是很多路径其实完全没有必要探索，即使使用**剪枝策略**（见习题 6 第 18 题参考答案），也要至少探索到当前路径长度超过 14 后再回头。DFS 的穷举过程最差时间复杂度是 $O(N!)$ 量级，对于稍微复杂的图，该算法绝对无法接受。况且如果需要求出 A 到所有顶点的最短路径，就需要再重复类似如图 5.10 的搜索过程若干次。

**CoreLoop**

我们暂时把思路拉回现实世界。

执笔至此，笔者以为，接下来要介绍的迪杰斯特拉算法，大抵是本书中最令人惊叹、最具创造性的算法。我不想直接介绍迪杰斯特拉算法的过程与实现。我要尝试做一件事情：与你一起思考并探索迪杰斯特拉算法“诞生”的过程：当 DFS 绝对无法接受时，最短路径问题就摆在我们面前，我们将会有哪些思考和结论？以此过程，一方面加深大家对于算法的理解和记忆（而不是枯燥地记住这复杂的过程），另一方面，通过探索过程中的艰难到最后的柳岸花明，来体会算法开创者非凡的创造力，以此向艾兹赫尔·迪杰斯特拉（Edsger W. Dijkstra）—这位计算机领域的传奇人物致敬。

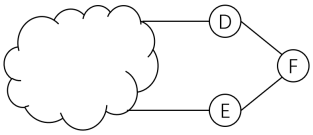
**如你对此过程兴趣无多，请直接跳至—waypoint-8—（第 94 页）处，这不会影响后面的旅途。**

首先，我们很容易想到这样一个想法：

**假设顶点 $i$ 正确的事实最短路径为 $mind[i]$**，那么，一定有：

$$mind[i] = min(mind[j_1] + cost(j_1, i), mind[j_2] + cost(j_2, i), ..., mind[j_n] + cost(j_n, i))$$

其中，$j_1, j_2, ..., j_n$ 为 $i$ 的所有邻接点，$cost(j, i)$ 表示顶点 $j$ 到 $i$ 的花费。




例如如图 5.9 中，要到达 F，必然经过 D 或 E，因此对于顶点 F，一定有：

$$mind[F] = min(mind[D]+cost(D, F), mind[E]+cost(E, F))$$

也就是说，F 的最短路径就是① A 到 D 的最短路径 + D 到 F 的花费与② A 到 E 的最短路径 + E 到 F 的花费这二者之间更小的。以上是**第一个事实**。那么只要确定了 D、E 的最短路径，F 的最短路径也就确定了。

那么我们能不能先确定某些顶点的最短路径呢？

显然，顶点 A 到自身的花费（即 $mind[A]$）一定是能确定的：就是 0。这是因为路程花费时间不可能为负，所以 A -> xxx -> A 的花费一定更大。我们将这种思想记为**“不绕弯思想”**。

同时，知道了 mind[A] 为 0 之后，对于顶点 C，它有 A、B、E 三个邻接点，因此

有：

$$mind[C] = min(mind[A] + cost(A, C), mind[B] + cost(B, C), mind[E] + cost(E, C))$$

其中的 $mind[A] + cost(A, C)$ 这一项就能计算得出：$0 + 5 = 5$。于是我们可以记录下来**当前已知的（并非全局的）C 的最短路径 $cur_mind[C] = 5$**，并在未来某时刻确定了 B、E 的 $mind[B]$ 和 $mind[E]$ 时，若 $mind[B] + cost(B, C) < cur_mind[C]$ 或 $mind[E] + cost(E, C) < cur_mind[C]$，则可以用 B 或 E 来更新 $cur_mind[C]$，这样最终得到的 $cur_mind[C]$ 一定是全局最短路径。于是我们顺着这个思路会自然地有这样的想法：**在确定了一个顶点的全局最短路径 mind[] 时，要尝试更新其所有邻接点的最短路径，否则可能错过正确解。**

于是在知道了 $mind[A]$ 之后我们更新所有 A 能到达的邻接点的“当前最短路径 $cur_mind[]$”，此时：

$$cur_mind[B] = 2$$
$$cur_mind[C] = 5$$

而其他顶点暂未找到一条到达 A 的路径。如图 5.11 a）所示，带下划线的数字表示该顶点当前已知的最短路径 $cur_mind[]$，顶点有填充则表示该顶点的全局事实最短路径已经确定。

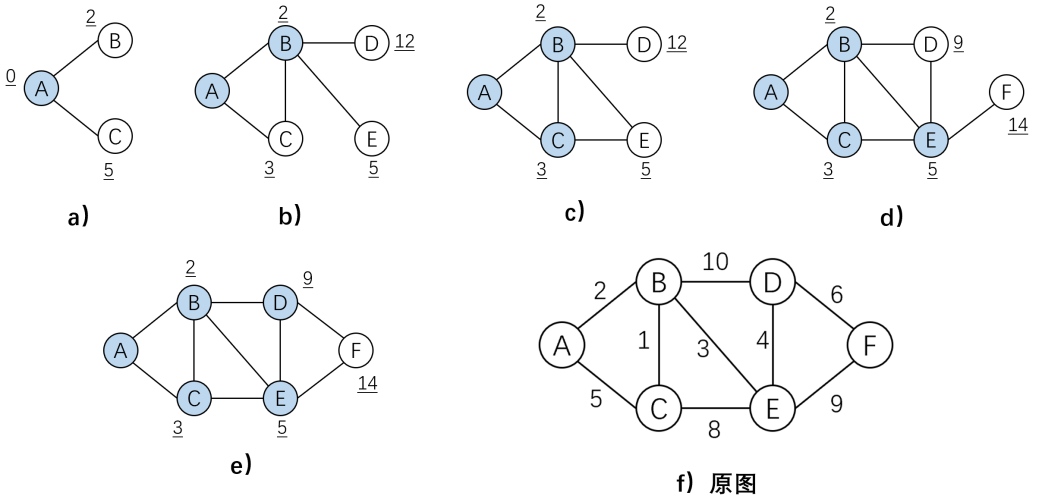



图 5.11

我们会发现，此时 $cur_mind[B] = 2$ 一定是 B 的全局最短路径（因为 B 是所有 A 的邻接点中花费最小的那个，因此 A -> B 一定比 A -> 非 B 更短，结合“不绕弯思想”，B 此时一定全局最短）。我们用集合 $S$ 记录已经找到全局最短路径的顶点，此时

$$S = \{A, B\}$$

如前所述，确定了 $mind[B]$ 后，就可以更新与 B 邻接的 C、D、E 的当前已知最短路径 $cur_mind[]$ 信息：

$$cur_mind[C] = min(5, mind[B] + cost(B, C)) = min(5, 2 + 1) = 3$$
$$cur_mind[D] = mind[B] + cost(B, D) = 2 + 10 = 12$$
$$cur_mind[E] = mind[B] + cost(B, E) = 2 + 3 = 5$$

如图 5.11 b）所示。此时所有 $cur_mind[]$ 中，最小的就是 $cur_mind[C]$。根据“不绕弯思想”，显然此时 C 已知的当前最短一定也是全局最短。可是，如何证明？接下来这部分便是迪杰斯特拉算法最为精彩也是最难理解的部分，还请听我慢慢道来。

如何证明当前已知的所有最短路径 $cur_mind[]$ 中最小的那个一定就是该顶点的全局最短路径？

如图 5.11 b）所示，由于 A、B 已确定全局最短路径，C 还未确定，且所有可以到达 C 的路径中一定至少存在一个还未确定全局最短路径的顶点（至少有一个 C 还未确定），我们假设存在某条路径 PATH 到 C 的距离比 $cur_mind[C]$ 更短：

```
PATH: A -> xxx -> P -> X -> xxx -> C
```

我们用 X 表示路径 PATH 中的第一个尚未确定全局最短路径的顶点，则它的前一个顶点 P 的全局最短路径一定已经确定。记这条路径 PATH 的长度为 $L_PATH$，则我们实际上假设了

$$L_PATH < cur_mind[C]$$

接下来，我们来一点一点证出该假设与事实相悖，从而推翻该假设。

由于 P 的全局最短路径已经确定，因此 $cur_mind[X]$ 一定已经由 P 点更新过，即此时一定有：

$$cur_mind[X] <= mind[P] + cost(P, X) \quad \textbf{// 式（1）}$$

上不等式右边即是假设的那条路径 PATH 到 X 时的长度（A -> xxx -> P -> X 的长度）。而由于图没有负权边，因此

```
A -> xxx -> P -> X
```

的长度一定小于等于

```
A -> xxx -> P -> X -> xxx -> C
```

的长度，即下式一定成立：

$$mind[P] + cost(P, X) <= L_PATH \quad \text{// 式（2）}$$

于是整理式 1 式 2，有：

$$cur_mind[X] <= mind[P] + cost(P, X) <= L_PATH \quad \text{// 式（3）}$$

而事实上 C、X 均是此时未确定最短路径的顶点，且 C 是所有 $cur_mind[]$ 中最小的那个，因此一定有：

$$cur_mind[C] <= cur_mind[X] \quad \text{// 式（4）}$$

由式 3、式 4，得：

$$cur_mind[C] <= cur_mind[X] <= mind[P] + cost(P, X) <= L_PATH$$

这与我们假设的 $L_PATH < cur_mind[C]$ 相悖，因此假设不成立，即不存在一条路径，其距离比当前已知的 C 的最小距离 $cur_mind[C]$ 更小。

**由以上的过程，我们就证明了：当前已知的所有最短路径 $cur_mind[]$ 中最小的那个一定就是该顶点的全局最短路径。**

于是，接下来的思路就豁然开朗了：不断地选择当前已知的所有最短路径中最小的那个，作为该顶点的全局最短路径（已由上面的过程证明这是绝对正确的），接下来，以该顶点为新的“中介”，去尝试更新其所有邻接点的 $cur_mind[]$，不断重复这个过程，直至所有顶点的全局最短路径都被确定。我们从 C 开始，看一遍接下来的过程。如

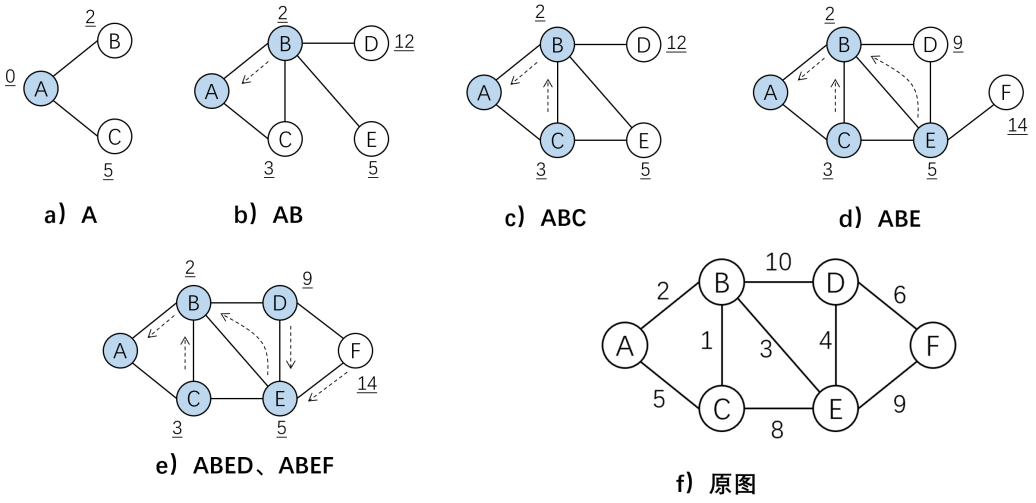



图 5.12

前所述，接下来将 C 的 $cur_mind[C]$ 作为 C 的全局最短，C 也就成了“已求解”的顶点。所有已求解的顶点集合 S 就变成了：

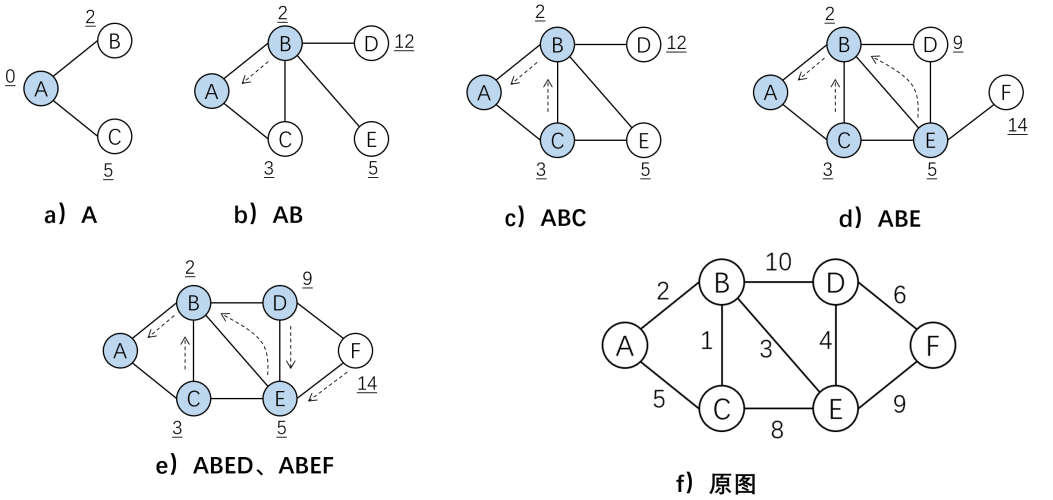
$$S = \{A, B, C\}$$

并且尝试更新 $cur_mind[E]$，但发现通过 C 到达 E 的距离：$3 + 8 = 11$ 不会比当前已知的：A -> B -> E（5）更短，因此 $cur_mind[E]$ 依然是 5。如图 5.12 c）所示。这一次我们在图中加入了路径回溯，记录每一个顶点最终的最短路径是由哪个顶点作为“中介”更新来的，如 B 是由 A 更新来的，因此指向 A，C 是由 B 最终更新来的，因此指向 B，以此方法记录最短路径本身（如 B 是 A -> B, C 是 A -> B -> C）。

继续从图中的 c）开始，此时 $cur_mind[D] = 12$，$cur_mind[E] = 5$，最小的是 $cur_mind[E]$，因此 E“已成功求解”，即 $mind[E] = cur_mind[E] = 5$，并以 E 作为新的中介，计算：

$mind[E] + cost(E, D) = 5 + 4 = 9$，小于当前的 $cur_mind[D] = 12$，于是更新 D
$mind[E] + cost(E, F) = 5 + 9 = 14$，发现了一条新的到达 F 的路，所以更新 F

更新后如图 d）所示。最后一步，D 作为“已成功求解”的顶点，并作为新的中介尝试更新 F，但通过 D 到 F 不会更小，因此 F 的路径前驱依然是 E，最终 F 也已成功求解，无其他顶点了，如 e）所示。




至此，所有顶点到达 A 的最短路径均已求解，$cur_mind[i]$ 中最终保存的就是顶点 $i$ 到 A 的全局最短路径（事实最短路径），因为它一定已经经过了其所有邻接点的更新操作。

最后，需要注意我们以上讨论，都是基于一个假设：图中不存在负权边。通过上述过程也可以看得出来，若存在负边，“不绕弯思想”将不复存在，因此整个推理过程也将崩塌。正是由于这样的道理，我们马上会看到，**迪杰斯特拉算法要求图中不存在从源**

**点可达的负权边**。并且由以上思维过程我们知道，若图非连通，迪杰斯特拉算法也可以得出哪些顶点从源点出发是不可达的。

**waypoint-8**

接下来我们正式介绍赫赫有名的迪杰斯特拉算法（Dijkstra 算法）。我们先给出算法正确、完整的步骤，再由一个具体例子直观地感受一下。

首先，我们给出如下定义：

> **路径长度**是指一条路径的所有边的权值之和。两顶点 u 和 v 之间的**最短路径**是指 u 到 v 的所有路径中，权值之和最小的路径；相反，**最长路径**则是指所有路径中权值之和最大的。

**迪杰斯特拉算法**可用于求解单源最短路径问题。设图 $G=(V,E)$ 有 $N$ 个顶点，$cost(u,v)$ 为 u 到 v 的权值（路径的长度），我们记 $cur_mind[i]$ 为顶点 $i$ 当前已知的最短路径长度，则求解起点 $s$ 到其他任意顶点的最短路径的过程如下：

1. 除起点 s 距离 s 为 0（$cur_mind[s]=0$）外，当前不知道任何其他顶点到 s 的最短路径，暂时认为所有顶点的最短路径都是正无穷 +inf，将其他所有 $cur_mind[]$ 初始化为 +inf。此时所有顶点均**未求解**。

2. 从所有**未求解**顶点中，选出已知的最短路径长度 $cur_mind[]$ 最小的那个顶点 v（第一次选的一定是 s），该长度一定就是该顶点 v 到 s 的全局最短路径长度，于是 v **“已求解”**。

3. 以 v 为中介点，尝试更新所有 v 的邻接点的 $cur_mind[]$。具体来说，对于 v 的一条指向 u 的边，若 $cur_mind[v]+cost(v,u)<cur_mind[u]$，说明经过 v 到 u 比之前已知的 u 到 s 的最短路径更短，所以将 $cur_mind[u]$ 更新为 $cur_mind[v]+cost(v,u)$。这一操作通常被称为 **“松弛操作”**。

4. 重复步骤 2 和步骤 3，直到所有可从起点 s 到达的顶点均已确定最短距离。

**事实上迪杰斯特拉算法就是一个不断地“选点”、“更新”的过程**。我们设置一个数组 Solved[] 来标记哪些顶点已求解，哪些还未求解；并设立一个数组 Pre[] 标识每个顶点的路径前驱，如 Pre[1] 值为 0 就表示顶点 1 的最短路径的上一步是从 0 走过来的。

我们以图 5.13 f）所示的图为例来直观地感受一下迪杰斯特拉的过程。

我们将 $v_0$ 作为源点 s，初始时，所有顶点均未求解，Solved[] 为 false，除源点 s 的 $cur_mind[0]=0$ 外，所有其他顶点 $cur_mind[]$ 都是 +inf，并且所有顶点的 Pre[] 均初始化为-1，表示尚未找到一条路径，暂无前驱，如表 5.2“初始”所示。

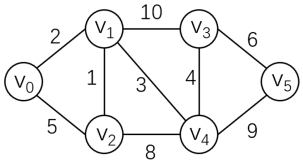
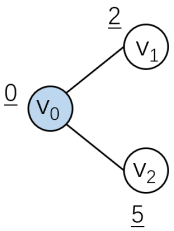


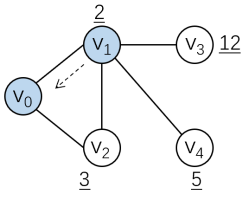

图 5.13 迪杰斯特拉算法过程演示图

1. 从未求解顶点集合 {0,1,2,3,4,5} 中选择当前已知最短路径 $cur_mind[]$ 最小的顶点（0 号顶点）。此时 0 号顶点的当前已知最短路径 $cur_mind[0]$ 一定也是全局最短路径，于是将 $v_0$ 作为“已求解”顶点，Solved[0] 标记为 true。并且 $v_0$ 就是起点，无前驱，$Pre[]$ 保持 -1 即可，如图 a）及表 5.2 步骤 1 所示。然后以 0 号顶点为中介，尝试更新其邻接点 $v_1$ 和 $v_2$ 的 $cur_mind[]$: 由于 $cur_mind[v_0] + cost(v_0, v_1) = 0 + 2 = 2 < cur_mind[v_1] = +inf$，因此将 $cur_mind[v_1]$ 更新为 $cur_mind[v_0] + cost(v_0, v_1) = 2$，同理将 $cur_mind[v_2]$ 更新为 0+5=5，由于 $v_1$ 和 $v_2$ 当前是由 $v_0$ 更新来的，因此将 Pre[1] 和 Pre[2] 均设为 0。



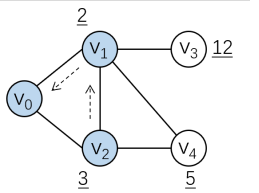

a) $v_0$

2. 从未求解顶点集合 {1,2,3,4,5} 中选择 $cur_mind[]$ 最小的顶点 $v_1$，将该顶点的全局最短路径确定为 2，并标记 $v_1$ 的 Solved[1] 为 true。然后以 $v_1$ 为中介，$v_1$ 的邻接点有 $v_2$、$v_3$、$v_4$，于是分别计算：$cur_mind[v_1] + cost(v_1, v_2) = 3 < cur_mind[v_2] = 5$，于是将 $cur_mind[v_2]$ 更新为 3；$cur_mind[v_1] + cost(v_1, v_3) = 12 < cur_mind[v_3] = +inf$，于是将 $cur_mind[v_3]$ 更新为 12，类似地，将 $cur_mind[v_4]$ 更新为 5。并且 $v_2$、$v_3$、$v_4$ 此时都是由 $v_1$ 更新来的，因此将 Pre[2]、Pre[3]、Pre[4] 记为 1。
如图 b）及表 5.2 步骤 2 所示。



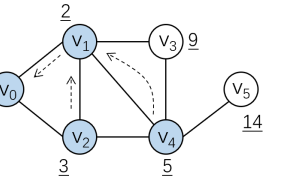

b) $v_0 v_1$

3. 从未求解顶点集合 {2,3,4,5} 中选择 $cur_mind[]$ 最小的顶点 $v_2$，将 Solved[2] 标记为 true，然后尝试更新 $v_4$ 的 $cur_mind[4]$，但由于 $cur_mind[2] + cost(v_2, v_4) = 3 + 8 = 11 > cur_mind[4] = 5$，因此无需更新。
这一步其实只将顶点 $v_2$ 标记为了“已求解”，如图 c）及表 5.2 步骤 3 所示。



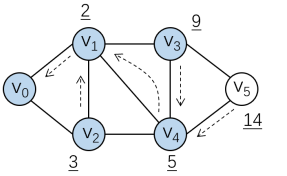
c） $v_0 v_1 v_2$

4. 从 {3,4,5} 中，选择顶点 $v_4$，标记 Solved[4] 为 true。并以 $v_4$ 为中介，将 $cur_mind[3]$ 更新为 9、Pre[3] 更新为 4；将 $cur_mind[5]$ 更新为 5+9=14，将 Pre[5] 更新为 4。
如图 d）及表 5.2 步骤 4 所示。



d） $v_0 v_1 v_4$

5. 从 {3,5} 中，选择顶点 $v_3$，标记 Solved[3] 为 true。以 $v_3$ 为中介，发现 $cur_mind[3] + cost(3,5) = 9 + 6 = 15 > cur_mind[5] = 14$，因此无需更新。此时最后一个顶点 $v_5$ 的最短路径显然也已经确定了，为 14。
如图 e）及表 5.2 步骤 5 所示。最后，我们将 $v_5$ 标记为已求解顶点即可。



e） $v_0 v_1 v_4 v_3$、 $v_0 v_1 v_4 v_5$

6. 将最后的顶点 $v_5$ 标记为已求解。如表 5.2 步骤 6 所示。

可以看到，**“选点”**就是每次从尚未求解的集合中，“选”出一个当前已知最短路径 $cur_mind[]$ 最小的顶点 v，作为“已求解顶点”，并用它尝试**“更新”**其所有邻接点的最短路径 $cur_mind[]$。每次能确定一个顶点的全局最短路径，重复 $N$ 次操作（其实 $N-1$ 次即可），即可求解所有顶点的全局最短路径，最终 $cur_mind[]$ 中保存的就是各个顶点的全局最短路径的长度，而各顶点的最短路径轨迹是什么，顺着 Pre[] 数组一路向前即可。如 $v_5$ 到 $v_0$ 的最短路径就是

> 5 -> Pre[5] = 4 -> Pre[4] = 1 -> Pre[1] = 0，即 5 -> 4 -> 1 -> 0

这就是赫赫有名的迪杰斯特拉算法。迪杰斯特拉算法可以求解出从源点 s 到各个与之连通的顶点之间的最短路径，同时也可以确定哪些顶点与 s 是不连通的。但迪杰斯特拉算法要求图中不存在从源点出发可达的负权边，**若图不满足这一条件，迪杰斯特拉算法将失效**。

迪杰斯特拉算法需要 $N$ 次“选点”、“更新”操作，更新时需要找到该顶点的所有邻接点，而对于邻接矩阵和邻接表存储的图，找邻接点的时间复杂度有所区别，因此，

表 5.2: 迪杰斯特拉算法过程演示

| 步骤 \ 顶点 | | 0 | 1 | 2 | 3 | 4 | 5 | 剩余未求解顶点集合 |
|---|---|---|---|---|---|---|---|---|
| 初始 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | {0,1,2,3,4,5} |
| 1 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>**true** | **2**<br>**0**<br>false | **5**<br>**0**<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | +inf<br>-1<br>false | {1,2,3,4,5} |
| 2 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>true | 2<br>0<br>**true** | **3**<br>**1**<br>false | **12**<br>**1**<br>false | **5**<br>**1**<br>false | +inf<br>-1<br>false | {2,3,4,5} |
| 3 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>true | 2<br>0<br>true | 3<br>1<br>**true** | 12<br>1<br>false | 5<br>1<br>false | +inf<br>-1<br>false | {3,4,5} |
| 4 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>true | 2<br>0<br>true | 3<br>1<br>true | **9**<br>**4**<br>false | 5<br>1<br>**true** | **14**<br>**4**<br>false | {3,5} |
| 5 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>true | 2<br>0<br>true | 3<br>1<br>true | 9<br>4<br>**true** | 5<br>1<br>true | 14<br>4<br>false | {5} |
| 6 | cur_mind[]<br>Pre[]<br>Solved[] | 0<br>-1<br>true | 2<br>0<br>true | 3<br>1<br>true | 9<br>4<br>true | 5<br>1<br>true | 14<br>4<br>**true** | { } |

迪杰斯特拉算法的时间复杂度与图的存储方式有关，下面我们就看看两种存储方式下，迪杰斯特拉算法如何实现。

### 1. 迪杰斯特拉算法实现（邻接矩阵版）

对于有 $N$ 个顶点的图，有了前面的思路，我们可以很轻松地写出迪杰斯特拉算法的代码。唯一需要注意的就是需要正确处理图中存在顶点不可达的情况，详请参见如下代码及注释。

```
int Matrix[N][N];   // 邻接矩阵，存放边权，若边不存在用 -1 表示
bool Solved[N];     // 已初始化为 false
int Pre[N];         // 已初始化为 -1
int cur_mind[N];    // 已初始化为 +inf。
const int inf = 0x3fffffff; // +inf
// 求解以 s 为源点的单源最短路径
void Dijkstra(int s) {
    cur_mind[s] = 0;
    // 该层循环用于执行 N 次“选点”、“更新”的操作
    for(int i=0; i<N; i++) {
        // 选出未求解的顶点中，cur_mind[] 最小的顶点 v
        int min_cur_mind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(Solved[j] == false && cur_mind[j] < min_cur_mind) {
                min_cur_mind = cur_mind[j];
                v = j;
            }
        }

        /* 这里是为了处理图中存在顶点不可达的情况。
         若存在顶点不可达，则将可达顶点全选完之后，
         接下来一定会“选不到顶点”，所以若没选到顶点，
         则 v 的值是无意义的，不应执行后面的代码。*/
        if(v == -1) {
            break;
        }

        Solved[v] = true;
        // 以 v 为中介，尝试更新其所有邻接点
        for(int u=0; u<N; u++) {
            // v 到 u 有路径并且 u 的全局最短还未确定
            if(Matrix[v][u] != -1 && Solved[u] == false) {
                if(cur_mind[v] + Matrix[v][u] < cur_mind[u]) {
                    cur_mind[u] = cur_mind[v] + Matrix[v][u];
                    Pre[u] = v;
                }
            }
```

```
        }
    }
}
```

对于以上邻接矩阵版的代码，需要 $N$ 次“选点”、“更新”操作，外层循环时间为 $O(N)$，每次选点时遍历一遍 cur_mind[] 数组的时间是 $O(N)$，更新时需要找到该顶点的所有邻接点，时间是 $O(N)$，整体的时间复杂度为 $O(N^2)$。空间方面，需要额外引入 Pre[]、Solved[]、cur_mind[] 三个数组，空间复杂度为 $O(N)$。

注意：邻接矩阵是存储图本身所需要的空间，不属于迪杰斯特拉算法带来的空间开销，不能算在算法的空间复杂度中。

**2. 迪杰斯特拉算法实现（邻接表版）**

邻接表版仅在获取顶点的邻接点信息时代码有所不同。邻接表版代码如下所示。

```
struct AdjEdge {
    int u;      // 邻接点的编号
    int weight; // 边权
};
vector<AdjEdge> AdjList[N];    // 邻接表
bool Solved[N];     // 已初始化为 false
int Pre[N];         // 已初始化为 -1
int cur_mind[N];    // 已初始化为 +inf。
const int inf = 0x3fffffff; // +inf
// 求解以 s 为源点的单源最短路径
void Dijkstra(int s) {
    cur_mind[s] = 0;
    for(int i=0; i<N; i++) {
        // 选出未求解的顶点中，cur_mind[] 最小的顶点 v
        int min_cur_mind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(Solved[j] == false && cur_mind[j] < min_cur_mind) {
                min_cur_mind = cur_mind[j];
                v = j;
            }
        }

        if(v == -1) {
            break;
        }

        Solved[v] = true;
        // 以 v 为中介，尝试更新其所有邻接点
        for(int j=0; j<AdjList[v].size(); j++) {
```

```
31              int u = AdjList[v][j].u;  // 与 v 邻接的顶点 u
32              if(cur_mind[v] + AdjList[v][j].weight < cur_mind[u]) {
33                  cur_mind[u] = cur_mind[v] + AdjList[v][j].weight;
34                  Pre[u] = v;
35              }
36          }
37      }
38  }
```

对于以上邻接表版的代码，外层循环时间同样为 $O(N)$，每次选点时遍历一遍 cur_mind[] 数组的时间同样是 $O(N)$，而更新时找到该顶点的所有邻接点的操作实际上与外层循环无关，是对整个图来说需要进行约 $M$ 次，$M$ 为图的边数。因此邻接表版时间复杂度为 $O(N^2+M)$。

空间方面，与邻接矩阵版本类似，同样是需要额外引入 Pre[]、Solved[]、cur_mind[] 三个数组，空间复杂度为 $O(N)$。

请注意，上面的代码中，每次寻找当前 cur_mind[] 最小的顶点时，我们是遍历一遍数组，为 O(N) 的时间。事实上，可以使用前面介绍过的数据结构—**堆**（C++ 中的 priority_queue）来完成，每次以 O(1) 时间即可返回最小的元素。若图采用邻接表实现，这种堆优化的迪杰斯特拉算法的**最坏**时间复杂度也能保证 $O((N+M)logN)$。由于 priority_queue 在此处的使用涉及稍复杂的 C++ 原理，因此不在此处给出代码，感兴趣的同学可以自己尝试写出。

### 5.4.2　Bellman-Ford 算法

几乎在迪杰斯特拉算法诞生的同期，也诞生了另外一个求解单源最短路径的算法：Bellman-Ford 算法。它与 Dijkstra 算法有一个重要区别：只要从源点出发能够到达的范围内不存在负权环，Bellman-Ford 算法就能够正确求得单源最短路径，即使图中存在负权边也没有问题。

Bellman-Ford 算法的核心思想是：反复对图中的所有边进行松弛操作。具体来说，假设图中有 $N$ 个顶点 $M$ 条边，源点为 s，与迪杰斯特拉算法类似，定义：cur_mind[v] 为当前已知的从 s 到 v 的最短路径长度，初始化 cur_mind[s] 为 0，其他顶点的 cur_mind[] 均为正无穷。

然后，对图中的每一条边 $(u,v)$ 进行松弛操作：

$$dist[v] = \min(dist[v], dist[u] + cost(u,v))$$

其中，$cost(u,v)$ 表示边 $(u,v)$ 的权值（路径长度）。对所有边完成一轮松弛后，再进行下一轮，则最多进行 $N-1$ 轮后，所有顶点的 cur_mind[] 就已经是全局最短路径。

为什么这个算法是正确的？

如图 5.14 所示，以 $v_2$ 为起点，$v_1$ 到 $v_2$ 的最短路径是 1 条边的路径：$v1->v2$，因此第一轮操作之后就能考虑到，并将其长度 1 赋值给 cur_mind[1]，此后无论如何 cur_mind[1] 也不会再被更新。

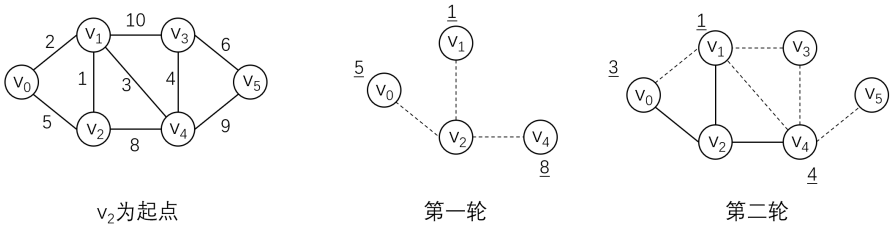



图 5.14

$v_0$ 到 $v_2$ 的最短路径是 $v_0->v_1->v_2$、$v_4$ 到 $v_2$ 的最短路径是 $v_4->v_1->v_2$，都是 2 条边的最短路径，因此第二轮操作之后就能考虑到这些路径，并且赋值给 cur_mind[0]、cur_mind[4]，之后绝对不会再改变。

Bellman-Ford 算法每完成一轮对所有边的松弛，就能够保证考虑到最多多一条边的路径，因此 $N-1$ 轮之后，就能考虑到最多 $N-1$ 条边的路径。而如果源点可达范围不存在负权环，那么任意两个顶点之间的最短路径一定不包含重复顶点，路径最多 $N-1$ 条边。

此外，如果完成 $N-1$ 轮后，再对所有边进行一轮松弛，仍然能够更新某个顶点的距离，就说明存在从源点可达的负权环。若图存在从源点可达的负环，则显然在负环中不断转圈可以让最短路径无限地小下去，直至负无穷。因此，通过以上手段 Bellman-Ford 算法可以在图不存在负环时给出正确解，并在图存在负环时给出“问题不可解”的结论。

Bellman-Ford 算法需要做约 $N$ 轮操作，每轮操作需要遍历所有边，对于邻接矩阵存储的图，遍历所有边需要遍历邻接矩阵，时间为 $O(N^2)$，而对于邻接表，遍历所有边需要遍历邻接表，时间为 $O(M)$。因此对于邻接矩阵存储的图，Bellman-Ford 时间复杂度为 $O(N^3)$，对于邻接表存储的图，Bellman-Ford 的时间复杂度为 $O(NM)$。

对于邻接表存储的图，Bellman-Ford 算法的实现如下所示：

```
struct AdjEdge {
    int u;
    int weight;
};
vector<AdjEdge> AdjList[N]; // 已存储图的邻接表
const int inf = 0x3fffffff;
int cur_mind[N];    // 已初始化为 inf
int Pre[N];     // 已初始化为 -1
bool BellmanFord(int s) {
```

```
    cur_mind[s] = 0;
    // 做 N-1 次
    for(int i=0; i<N-1; i++) {
        // 遍历所有边
        for(int v=0; v<N; v++) {
            // 与 v 邻接的所有边
            for(int k=0; k<AdjList[v].size(); k++) {
                int u = AdjList[v][k].u;
                int weight = AdjList[v][k].weight;
                if(cur_mind[v] + weight < cur_mind[u]) {
                    cur_mind[u] = cur_mind[v] + weight;
                    Pre[u] = v;
                }

            }
        }
    }

    // 再做一次，判断是否存在负环
    for(int v=0; v<N; v++) {
        for(int k=0; k<AdjList[v].size(); k++) {
            int u = AdjList[v][k].u;
            int weight = AdjList[v][k].weight;
            if(cur_mind[v] + weight < cur_mind[u]) {
                return false;
            }

        }
    }
    return true;
}
```

当图存在负环时，Bellman-Ford 算法返回 false，表示存在负环，最短路径问题不可解。否则返回 true，cur_mind[] 数组中存放的就是各顶点的最短路径。沿着 Pre[] 数组往回追溯即可得到轨迹。对于邻接矩阵版本仅在于遍历边的方式不同，这里不再给出代码。

可以看到，与 Dijkstra 算法从源点出发，逐渐向外“扩散”地求解各个顶点的最短路径的策略不同，Bellman-Ford 算法则是“放眼全局”，一步一步地让全局下的各个 cur_mind[] 慢慢向正确解“收拢”。

对于 Bellman-Ford 算法，我们自然会想到这样一个问题：我们每轮处理的时候，都需要遍历所有边，判断能否更新某个顶点的 cur_mind[]。但事实上，假设第 $i-1$ 轮仅有顶点 v 的 cur_mind[] 发生了变化，那么第 $i$ 轮其实我们没有必要遍历所有边，而是只遍历 v 的出边即可，因为只有距离发生了变化的顶点才可能继续影响其他顶点。这

样，仅处理距离发生了变化的边，就大大提高了算法的效率。因此对于上面的代码，在以下条件成立时：

```
if(cur_mind[v] + weight < cur_mind[u]) {
    cur_mind[u] = cur_mind[v] + weight;
    Pre[u] = v;
}
```

说明 u 是一个距离发生了变化的顶点，我们将其放入队列中。

于是思路就是：最初将起点 s 放入队列，并在每次处理时，将那些距离发生了改变的顶点入队。当队列为空时，说明没有新的顶点距离发生改变，也就处理完了。但是这样做有两个新的问题：

（1）反复入队问题。

如图 5.15 所示，假设 0 为起点，向上松弛的若干顶点，将这些顶点入队后，取出时，对每一个顶点的处理都可能将顶点 1 松弛一遍，如果每次都将 1 入队就造成了重复入队，会出现很多无意义的重复处理，继续向外传导，会造成指数级的性能灾难。因此要解决这个问题，就需要引入一个数组 in_queue[N]，标识每一个顶点是否已经在队列中了，顶点此时不在队列中时才将它入队。

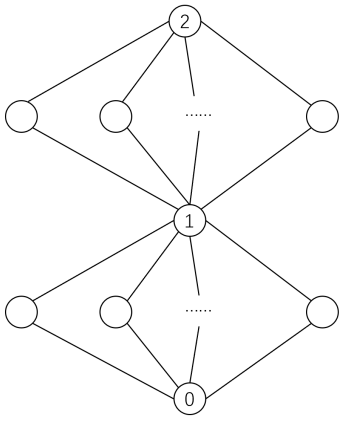


图 5.15 反复入队问题

（2）负环判断问题。这种做法一直做到队列为空，若存在负环，则永远会有新的顶点距离变小，队列永远不会空。那怎么办？常见的做法是再记录一个数组 inq_cnt[N]，标记每个顶点入队的次数。若某个顶点已经入队了超过 $N-1$ 次（它的距离已经被更新过超过 $N-1$ 次），说明存在负环，问题不可解。

这种优化后的 Bellman-Ford 算法被称为 **SPFA（Shortest Path Faster Algorithm）算法**。其代码实现（邻接表版）如下所示：

```
struct AdjEdge {
    int u;
    int weight;
};
vector<AdjEdge> AdjList[N]; // 已存储图的邻接表
const int inf = 0x3fffffff;
int cur_mind[N];    // 已初始化为 inf
int Pre[N];     // 已初始化为 -1

bool in_queue[N];   // 标记顶点是否当前在队列中，初始化为 false
```

```
int inq_cnt[N];          // 记录顶点入队次数，用于检测负权环，初始化为 0
bool SPFA(int s) {
    cur_mind[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;

    while(!q.empty()) {
        int v = q.front();
        q.pop();

        // 弹出后，别忘了将标记改为 false，允许它以后再次入队
        in_queue[v] = false;

        // 遍历当前顶点的所有出边
        for(int j = 0; j < AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int weight = AdjList[v][j].weight;

            // 松弛操作
            if(cur_mind[v] + weight < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + weight;
                Pre[u] = v;

                // 如果 u 不在队列中，将其压入队列
                if(!in_queue[u]) {
                    q.push(u);
                    in_queue[u] = true;
                    inq_cnt[u]++; // 记录入队次数

                    // 如果某个顶点入队次数超过 N-1 次，说明存在负权环
                    if(inq_cnt[u] >= N) {
                        return false;
                    }
                }
            }
        }
    }

    return true;
}
```

SPFA 算法的平均时间复杂度为 $O(kM)$，$k$ 是每个顶点的平均入队次数，而最坏时间复杂度为 $O(NM)$。在大多数场景下 SPFA 算法性能要由于堆优化后的迪杰斯特拉算法，但对于极端情况，迪杰斯特拉算法最差也能保证 $O((N+M)logN)$ 的复杂度，而 SPFA 算法可能达到 $O(NM)$。

### 5.4.3　Floyd 算法

Floyd 算法（弗洛伊德算法），是一种用于求解图中所有顶点对之间最短路径的算法，即**全源最短路径**问题。。Floyd 算法的核心思想在于：对于任意三个顶点 $i$、$j$、$k$，我们都可以尝试：

$$d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$$

也就是说，我们原来知道的 $i$ 到 $j$ 的最短距离是 $d_{ij}$，现在尝试让路径经过顶点 $k$，如果经过 $k$ 的距离更短，就更新答案。

顶点到自身的距离初始化为 0，存在直接边的顶点之间距离初始化为边权，不存在直接边的顶点之间距离初始化为正无穷。算法实现如下：

```
const int inf = 0x3fffffff;
int d[N][N];

// 初始化距离矩阵
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        d[i][j] = (i == j ? 0 : inf);
    }
}

// 根据图中的边更新距离矩阵
// 假设存在一条从 u 到 v、权重为 w 的边
degree[u][v] = w;
// ...

// Floyd 算法
for (int k = 0; k < N; k++) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (d[i][k] != inf &&
                d[k][j] != inf) {
                d[i][j] = min(
                    d[i][j],
                    d[i][k] + d[k][j]
                );
            }
        }
    }
}
```

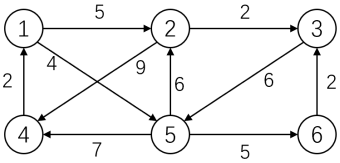
Floyd 算法支持存在负权边的图，但要求不存在从源点可达的负权环。其时间复杂度为 $O(N^3)$，空间复杂度为 $O(N^2)$。

# — BONFIRE ENCOUNTERED —

火光闪烁中，你独自坐在营火旁。

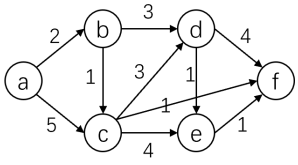
**习题 4** （参考答案见习题 4 参考答案，第 171 页）

1. （某年 408 真题）使用迪杰斯特拉算法求下图中顶点 1 到其他各顶点之间的最短路径，则依次确定最短路径的目标顶点是_____



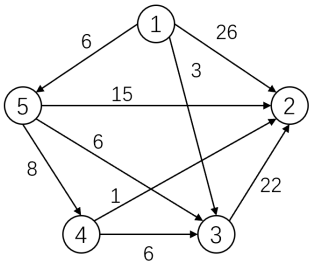

A. 5,2,3,4,6　B. 5,2,3,6,4　C. 5,2,4,3,6　D. 5,2,6,3,4

2. （某年 408 真题）对下图所示的有向带权图，若采用迪杰斯特拉算法求源点 a 到其他各顶点的最短路径，则第一个确定全局最短路径的顶点是 b，第二个确定全局最短路径的顶点是 c，后续的顶点依次是_____




A. d,e,f　B. e,d,f　C. f,d,e　D. f,e,d

3. （某年 408 真题）使用迪杰斯特拉算法求下图中从顶点 1 到其余各顶点的最短路径，将当前找到的从顶点 1 到顶点 2，3，4，5 的最短路径保存在数组 dist[] 中，则依次求出两个顶点（不包括源点 1）的最短路径后，dist 中的内容更新为_____




A. 26,3,14,6　B. 25,3,14,6　C. 21,3,14,6　D. 15,3,14,6

4. 请你说明：迪杰斯特拉算法、Bellman-Ford 算法以及 SPFA 算法应对以下情况的图时会怎样？

a）存在从源点可达的负环；
b）不存在从源点可达的负环，但存在从源点可达的负权边；

维火女被邪恶的恶魔、大陆守护神的孪生兄弟 LoopCore 抓了起来，并关在了监牢的某个牢房里。监牢可以被建模为一个图 $G=(V,E)$，顶点表示牢房。

已知该监牢有 $N$ 个牢房，编号为 0 到 $N-1$，牢房之间均存在双向通路，用邻接表存储图模型。你毫不犹豫地动身前往牢房营救维火女，但你破门而入后意外地被魔法传送进了牢房 s。

5. 已知每个牢房（顶点）的数据类型为 char：维火女所在的牢房为'T'，其余牢房均为'F'。给出在每两个牢房之间移动所需要花费的时间 time_cost（非负），请你计算从 s 出发，到达维火女的房间所需要的最短时间是多少，若无法到达，返回-1。

```
// N 已定义
struct AdjEdge {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<AdjEdge> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {

}
```

6. 在上题条件下，假设你在任何两个房间移动时需要和敌人战斗，此时邻接边（AdjEdge）使用如下结构定义：

```
struct AdjEdge {
    int u;
    int time_cost;      // 花费时间
    int energy_cost;    // 和敌人战斗需要消耗的能量值
};
```

保证所有边的 time_cost 和 energy_cost 均非负。你决定选择所有路线中，消耗时间最少的路线去营救维火女。若有多条路线消耗时间相等，你将选择其中消耗能量最少的一条路线。

编写一个函数，返回你所选择的路线所消耗的能量值。若无法到达，返回-1。

你成功营救出了维火女，你们一同回到大陆中，继续冒险。她手中正是那代表着庇佑与祝福的樱花之泪。

## 5.5　最小生成树

王国的指挥官们解决了最短路径问题之后，又提出了一个新的问题：目前据点之间通信只能通过信使传递，他们希望在据点之间铺设通信线缆，使得信息可以顺着线缆迅速传递。指挥官们认为，没有必要在任意两个据点之间都铺设线缆，而是只要能够保证任意两个据点之间都可以通过线缆连通即可。这样一来，即使信息需要经过其他据点中转，也能实现整个王国的线缆通信。

然而，铺设线缆需要耗费大量资源。不同据点之间铺设线缆所需的成本各不相同。指挥官们希望找到一种方案：既能保证所有据点都连通，又能使铺设线缆的总成本尽可能低。应该如何设计这样的方案呢？这其实就是**最小生成树**的问题：

> 给定一个连通的无向带权图 $G = (V, E)$，如果从图中选择若干条边，使得所有顶点连通且不存在环，那么图的所有顶点及这些边构成的子图就是原图的一棵**生成树**。在所有生成树中，边权总和最小的生成树称为**最小生成树（Minimum Spanning Tree，MST）**。

此问题下，边的权即是修建通信线缆的费用。

**提示**

若图不连通，则可以求每个连通块的最小生成树，将它们合在一起，就得到了最小生成森林。

对此问题，格佛莱指出：初始时随意选择一个顶点 s，然后选择依附于 s 的边权最小的一条边，将该边与相连顶点“收进来”，此时，当前的生成树中就已经有了 2 个顶点。然后，继续选择能够连通这 2 个顶点与其他顶点的一条边权最小的边，并将该边与相连的顶点“收进来”，生成树中就有了 3 个顶点。重复这个操作，直至所有顶点都被“收进来”。这样做，得到的生成树中的所有边，一定是使得顶点连通的所有边的选集中权值之和最小的 $N-1$ 条边。如图 5.16 所示，使得三个顶点连通，可选的边集有 $\{e_0, e_1\}$、$\{e_0, e_2\}$ 以及 $\{e_1, e_2\}$，权值之和最小的一定是 $\{e_0, e_2\}$。

因此此方法得到的树一定就是最小生成树。这就是 **Prim（普里姆）算法**的核心思想。

### 5.5.1　Prim（普里姆）算法

设无向连通图 $G = (V, E)$，Prim 算法的过程如下：设置集合 S 保存已经收入当前的生成树中的顶点，cur_mind[] 保存各未收入顶点到集合 S 的最小代价。初始时随意

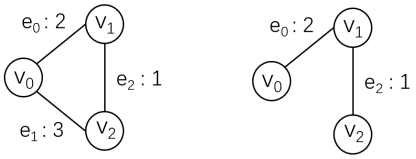



图 5.16

收入一个顶点 s（即 S={s}），令与 s 邻接的所有顶点的 cur_mind[] 为相应边权，令其他所有顶点的 cur_mind[] 为正无穷。然后执行 $N-1$ 次以下两个步骤：

1. 从集合 V-S（还未收入当前生成树的顶点集）中选择距离集合 S 代价最小的一个顶点（即 cur_mind[] 最小）u，将 u 加入 S，并将 u 以及这条边加入当前生成树中。

2. 对于 u 的每个未被收入 S 的邻接点 v，执行

$$cur_mind[v] = min(cur_mind[v], cost(u, v))$$

即如果通过 u 到达 v 的代价比原来记录的更小，则更新 cur_mind[v]。

可以看到这个过程与迪杰斯特拉算法非常像，迪杰斯特拉算法是每次选择与源点 s 最近的顶点，而 Prim 算法是选择与已有生成树 S 最近的顶点。注意：Prim 算法中，cur_mind[v] 记录的并不是顶点 v 到源点 s 的最短距离，而是顶点 v 到集合 S 的最短距离，即 v 到 S 中的所有顶点的距离中最小的那个。

我们以图 5.17 a）所示的图，演示一遍 Prim 算法的过程：

1）任选一个点，假设选 A，收入最小生成树，更新 A 的邻接点 B 和 C 的 cur_mind[]，如 b）所示；此时集合 S 为 {A}。图中带下划线的数字表示还未收入生成树的各个顶点距集合 S 的最小代价（最短距离）；图中虚线表示该顶点最近是被哪条边所更新的，收入该顶点时沿着虚线就能找到边。

2）如 b），此时未收入的顶点中，到集合 S 代价最小（最近）的顶点是 B，于是收入 B，如 c）所示，收入 B 后，由于 $cur_mind[C] = 5 < cost(B, C) = 1$，也就是说对于新的集合 S={A, B}，C 到 B 的距离更小了，于是更新 C 到集合 S 的距离为 1。类似地，更新 cur_mind[E] 为 3、cur_mind[D] 为 10。

3）此时未收入的顶点中，到集合 S 代价最小（最近）的顶点是 C，于是收入 C，如 d）所示，并且对于 C 的邻接点 E，由于 $cur_mind[E] = 3 < cost(C, E) = 8$，因此无需更新。

4）如 d），此时 E 的距离最小，因此收入 E，由于 $cost(E, D) = 4 < cur_mind[D] = 10$，于是将 $cur_mind[D]$ 更新为 4，类似地，将 $cur_mind[F]$ 更新为 9。如 e）所示；

5）继续收入 D，然后更新 F，如 f）所示；

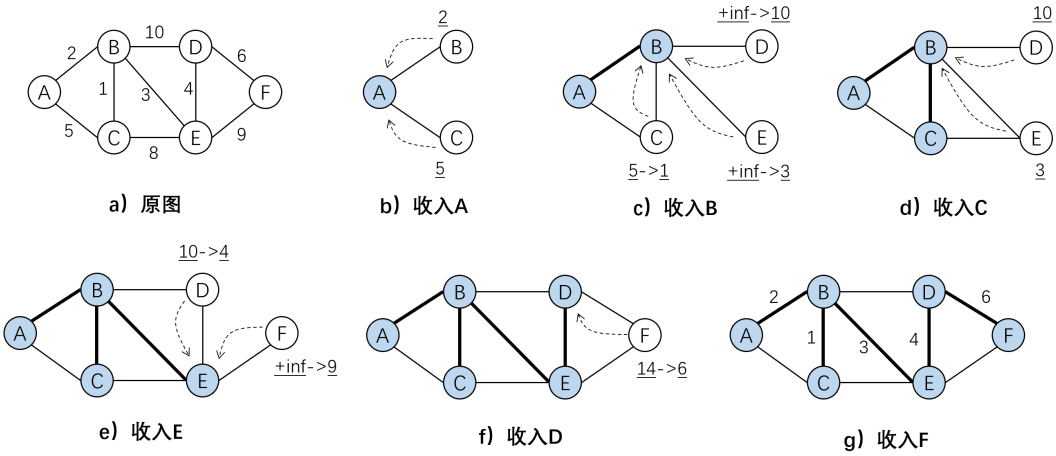



图 5.17 Prim 算法过程演示

6）继续收入 F，所有顶点均已被收入生成树，此时得到的即是最小生成树，各个边的权值之和即是最小代价：$2+1+3+4+6=16$。

可以看到对于连通图，Prim 算法每次一定能收入一个顶点到生成树中，因此共收入 $N$ 个顶点（包括源点 s）即可完成生成树的建立。

### 5.5.2　Kruskal（克鲁斯卡尔）算法

指挥官麦丽凯给出了一种完全不同的思路：她不喜欢一步一步细致、缜密地规划，而是将注意力放眼全局。麦丽凯指出：初始时，没有任何顶点和边被收入生成树，将图看成是 $N$ 个割裂的顶点构成了 $N$ 个连通块。首先挑选整个图中边权最小的边，将它以及它连通的两个顶点加入生成树，连通块的数量便还剩 $N-1$ 个。此后，每次选择剩余边中权值最小的，若它所连接的两个顶点处在不同连通块，则将他们连通并收入生成树，直至所有顶点均已连通。

该策略显然是正确的。因为如果两个连通块之间存在一条权值最小的边，那么最优解是一定要包含这条边的。这种放眼全局，通过不断地选择边权最小的边将不同连通块合并的策略，就是 **Kruskal（克鲁斯卡尔）算法**的核心思想。

具体来说，设无向连通图 $G=(V,E)$ 有 $N$ 个顶点，Kruskal 算法的过程如下：将图看成是 $N$ 个割裂的顶点构成了 $N$ 个连通块。然后反复按边权从小到大检查所有还未被收入生成树的边，若被检查的边所连接的两个顶点不在同一个连通块中，则将他们连通，并收入生成树，否则跳过，继续检查下一条边。如此重复，直至所有顶点均已连通；

我们以 5.18 a）所示的图演示一遍 Kruskal 算法的过程。

1）所有边中权值最小的是连接 BC 的边（为 1），将其收入，如 b）所示，此时 B、

C 已被连通；

2）此时剩余所有边中，边权最小的是连接 A、B 的边（为 2），由于 A、B 此时不在相同的连通块，因此将他们收入生成树，此时 A、B、C 已被连通，如 c）所示；

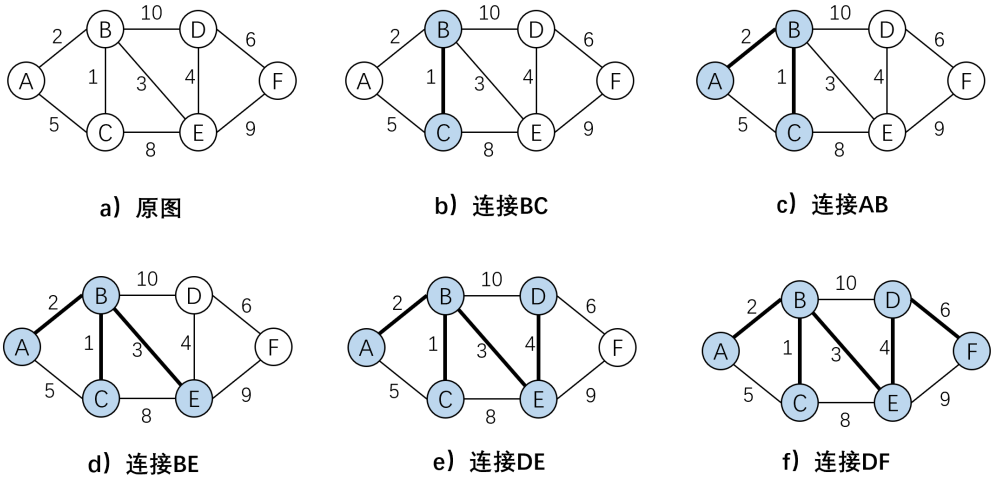



图 5.18 Kruskal 算法过程演示

3）此时剩余所有边中，边权最小的是连接 B、E 的边（为 3），由于 B、E 此时不在相同的连通块，因此将他们收入生成树，此时 A、B、C、E 已被连通，如 d）所示；

4）类似地，将 DE 收入生成树，此时 A、B、C、E、D 已被连通，如 e）所示；

5）此时剩余所有边中，边权最小的是连接 A、C 的边（为 5），但 A、C 已经处于同一个连通块，因此跳过，继续检查下一条边；

6）剩余所有边中，边权最小的是连接 D、F 的边（为 6），并且 D、F 处在不同的连通块，于是将它们收入生成树，如 f）所示，此时所有顶点均已被连通，算法结束。可以看到，在这个图的例子中，Kruskal 算法得到了与 Prim 算法相同的最小生成树。

在第四章中我们已经知道，具有 $N$ 个顶点的树，边的数量一定是 $N-1$ 条。因此 Kruskal 算法中，当被收入生成树的边的数量达到 $N-1$ 时，就达成了最小生成树。

### 5.5.3　算法实现

**Prim 算法的实现**

可以看出，对于 Prim 算法，我们需要动态维护当前生成树以及尚未收入的所有顶点到当前生成树的最小代价。我们通过一个 solved[] 数组标记每个顶点是否已经被收入生成树，cur_mind[] 数组记录尚未收入的所有顶点到当前生成树的最小代价。

Prim 算法的代码与 Dijkstra 算法非常相似。以邻接表为例，Prim 算法的实现如下所示（高亮部分是与 Dijkstra 的区别部分）：

```cpp
struct AdjEdge {
    int u;
    int weight;
};
vector<AdjEdge> AdjList[N];
const int inf = 0x3fffffff;
bool solved[N]; // 已初始化为 false
int cur_mind[N];    // 已初始化为 inf
int Pre[N];     // 已初始化为 -1

// 返回生成树的最小代价。
int Prim(int s) {
    cur_mind[s] = 0;
    // min_cost 用于累加生成树的最小代价，算法返回它
    int min_cost = 0;
    // 为了代码简洁，将收入 s 的操作放在循环里了，因此要做 N 次循环
    for(int i=0; i<N; i++) {
        int v = -1;
        int min_curmind = inf;
        for(int j=0; j<N; j++) {
            if(!solved[j] && cur_mind[j] < min_curmind) {
                min_curmind = cur_mind[j];
                v = j;
            }
        }

        // 图不连通
        if(v == -1) {
            return -1;
        }
        solved[v] = true;
        // 累加代价
        min_cost += cur_mind[v];

        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int weight = AdjList[v][j].weight;
            // 新加入集合的 v，能使的 u 到集合的距离更小
            if(weight < cur_mind[u]) {
                cur_mind[u] = weight;
                Pre[u] = v;
            }
        }
    }
    return min_cost;
}
```

算法的返回值即是最小生成树的代价（铺设线缆需要花费的总费用）。若需要知道

这棵最小生成树“长什么样”，则可以在每次选出顶点 v 时，打印出 Pre[v] 和 v，它们即构成了最小生成树中的一条边。同时可以看出，与迪杰斯特拉算法类似，“选点”的过程同样可以通过堆优化。

### 2.Kruskal 算法实现

Kruskal 算法需要每次选择边权最小的边，若每次遍历邻接表或邻接矩阵进行“选边”，代价显然无法接受。因此实现 Kruskal 算法通常是使用一个数组直接存储所有“边”，并且将所有边按边权排序，排好序后每次按顺序选择即可。这里引入 C++ 标准库中极为重要的一个函数：sort 函数。

```
// “边”的定义
struct Edge {
    int u,v;    // 边连接的两个顶点 u、v
    int weight; // 边的信息（边权）
    // 这是 C++ 语法：定义两个 Edge 结构体的“<”比较规则
    bool operator < (const Edge& other) {
        return this->weight < other.weight;
    }
};
Edge edges[M];  // 假设已经存储了图 G 中所有边的信息


int Kruskal() {
    // 使用 C++ 标准库的 sort() 函数，按照 Edge 的“<”比较规则，对 edges 数组进
    ↪  行排序。
    std::sort(edges, edges+M);
    // ...
}
```

通过上面的代码，就将 edges 这个数组按照其元素 weight 的大小关系，排成了一个从小到大的有序数组。关于结构体的比较规则定义、std::sort 的使用请参见第 **??** 页。

Kruskal 算法选边时，需要判断两个顶点是否在同一个连通块中，若不在，需要将两个连通块合并，这正是并查集提供的功能！我们这里直接使用 4.3 节中提供的并查集的 Find 与 Union 操作。若将所有边遍历一轮之后，收入生成树的边的数量不足 $N-1$，说明图不连通。可以写出 Kruskal 的代码如下：

```
int Kruskal() {
    // 按照 Edge 的“<”比较规则，对 edges 数组进行排序。
    std::sort(edges, edges+M);
    // 要返回的生成树最小代价
    int min_cost = 0;
```

```
    // 已收进来的边的数量
    int edgenum_in_tree = 0;

    // 并查集的 father 数组，最初所有顶点均不连通
    int father[N];
    for(int i=0; i<N; i++) {
        father[i] = i;
    }

    // 遍历所有边 (u, v)
    for(int i=0; i<M; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int u_id = Find(father, u); // u 所在的集合编号
        int v_id = Find(father, v); // v 所在的集合编号
        // 这条边连接的 u、v 不在同一个集合
        if(u_id != v_id) {
            Union(father, u, v);
            min_cost += edges[i].weight;
            // cout << “选择了一条边 u->v”
            edgenum_in_tree += 1;
            // 已收入了 N-1 条边，已经达成最小生成树
            if(edgenum_in_tree == N-1) {
                break;
            }
        }
    }
    /* 原图不连通，不存在包含所有顶点的生成树。*/
    if(edgenum_in_tree < N-1) {
        return -1;
    }
    return min_cost;
}
```

可以看出在原图不连通时，Kruskal 算法可以通过返回-1 给出“问题不可解”的结论，在可解时返回最小代价。

## 5.6　AOV 网与拓扑排序

某经营策略游戏有多种科技可以解锁，比如要解锁“机械制造”科技必须要先解锁“铁器制造”等。科技的解锁依赖关系被建模成了一棵科技树，如图 5.19 所示。

显然这是一个有向图，顶点表示解锁某种科技，而 A -> B 的有向边表示要解锁 B，就必须要先解锁 A。

这种用顶点表示活动、有向边表示活动之间先后依赖关系的有向图被称为 **AOV**




图 5.19 科技树

网（**Activity On Vertex Network，顶点表示活动的网**）。具体来说，

AOV 网是一个有向图，其中：
- 顶点表示活动或任务。
- 有向边 $u \to v$ 表示活动 $u$ 必须先于活动 $v$ 进行。

我们将某科技树简化，并将其各个顶点从 0 开始进行编号，得到一个如图 5.20 所示的有向图。

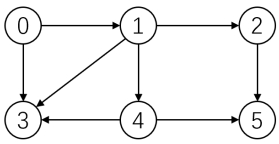



图 5.20 科技树模型

显然只要图中不存在环，则一定可以按照某种顺序解锁所有科技。比如对于图 5.20 所示的科技树，一种可行的科技解锁顺序是：014325，014325 这个序列就是一个**拓扑序列**。

**定义**

由有向无环图的所有顶点构成的序列，若满足如下条件，则称该序列是该图的一个**拓扑序列**：
对图中任意顶点 A 与 B，若存在 A 到 B 的路径，则序列中 A 一定在 B 的前面。

求解有向无环图的一个拓扑序列的过程，就叫做对这个有向无环图进行**拓扑排序**。

类似将所有顶点按大小排序，“拓扑排序”也是将所有顶点进行排序，得到一个序列，只不过不是依据“大小”，而是依据“拓扑（结构）”，所以叫“拓扑排序”。

**提示**

有些地方也会将“拓扑排序”作为名词使用。如：“我们对该图进行拓扑排序，得到的拓扑排序是 014325”。

对于 AOV 网进行拓扑排序的算法非常简单，也很好理解：

1. 从图中选择一个入度为 0 的顶点加入拓扑序列尾部；
2. 从图中删除该顶点以及它的所有出边；
3. 重复 1 和 2 的过程，直到图中不再存在入度为 0 的顶点。

经上述过程后，如果所有顶点都已被加入序列，拓扑排序成功。如果仍有顶点未被加入序列，说明图中存在有向环，拓扑排序失败。

拓扑排序的过程如图 5.21 所示：

1. 初始时仅顶点 0 入度为 0，将 0 加入拓扑序列尾部，并删除顶点 0 以及它的所有出边，如 b）所示；
2. 此时仅顶点 1 入度为 0，将 1 加入拓扑序列尾部，并删除顶点 1 以及它的所有出边，如 c）所示；
3. 此时顶点 2、4 的入度均为 0，任选一个顶点，假设选 4，将 4 加入拓扑序列尾部，并进行相应删除，如 d）所示；
4. 此时顶点 2、3 的入度均为 0，假设选 3，如 e）所示；
5. 此时仅剩顶点 2、5，其中 2 的入度为 0，5 的入度为 1，因此选择 2 加入序列尾部，如 f）所示；

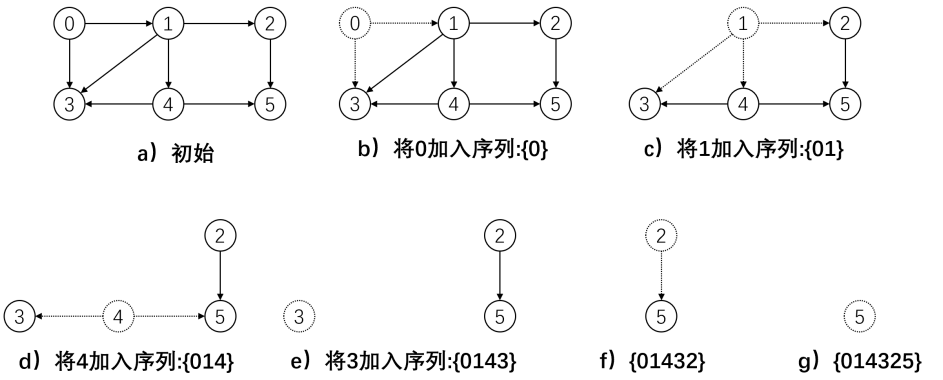



图 5.21 拓扑排序过程演示

6. 此时最后一个顶点 5 的入度为 0，将其加入序列尾部，如 g）所示，所有顶点均被加入序列，拓扑排序成功。

### 拓扑排序的实现

拓扑排序需要多次选取图中入度为 0 的顶点，若每次遍历邻接表或邻接矩阵，会产生较大的开销。因此通常使用一个数组来维护各顶点的入度值：

```
int in_deg[N];  // N 为顶点数量
```

最初时仅遍历一遍邻接表或邻接矩阵即可得到图最初的 in_deg[]，后面动态维护即可。还有一个问题，每次选择入度为 0 的顶点时，需要遍历一遍 in_deg[] 数组吗？事实上是不需要的。我们完全可以采用如下策略：

在最初的 in_deg[] 数组中选出所有入度为 0 的顶点，将它们加入一个队列（或栈）中，之后每次出队一个，加入拓扑序列，并且更新 in_deg[] 数组，若有新的顶点入度减少后变成 0，将它们加入队列即可。这样，重复操作直至队列为空，判断拓扑序列中元素个数是否等于 $N$ 即可。

假设 AOV 网有 $N$ 个顶点，编号为 0 到 $N-1$，以邻接表存储为例，拓扑排序的实现代码如下所示：

```
// AdjList 数组已存储某 AOV 网的邻接表，N 为图中顶点数量
bool TopologicalSort(vector<int> AdjList[], int N) {
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }

    vector<int> topo_seq;   // 用于存储拓扑序列
```

```
    // 初始化 in_deg[] 数组
    for(int i=0; i<N; i++) {
        for(int j=0; j<AdjList[i].size(); j++) {
            int u = AdjList[i][j];
            in_deg[u] += 1;
        }
    }
    // 将初始时所有入度为 0 的顶点入队
    queue<int> q;
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
        }
    }
    while(!q.empty()) {
        // u 是当前选出的，入度为 0 的顶点
        int u = q.front();
        q.pop();
        // 加入拓扑序列
        topo_seq.push_back(u);
        // 遍历所有 u 的邻接点 v
        for(int j=0; j<AdjList[u].size(); j++) {
            int v = AdjList[u][j];
            // 将边 u -> v 删除，v 的入度减一
            in_deg[v] -= 1;
            if(in_deg[v] == 0) {
                q.push(v);
            }
        }
    }

    // 拓扑序列中元素数量不足 N，说明原图存在环
    if(topo_seq.size() < N) {
        return false;
    }

    // cout << " 拓扑序列是：";
    // 打印 topo_seq 中的元素序列

    return true;
}
```

因为每个顶点入队、出队至多一次，每条边也只会被处理一次。因此算法的时间复杂度为：$O(N+M)$，空间方面，需要 $O(N)$ 的辅助空间，用于入度数组、队列和拓扑序列。

## 5.7　AOE 网与关键路径

王国罗德尔的指挥官们将“铺设通信线缆”这一工程使用了如图 5.22 所示的有向带权图来表示。这个有向带权图有唯一的起点 $v_0$ 表示工程开始以及唯一的终点 $v_6$ 表

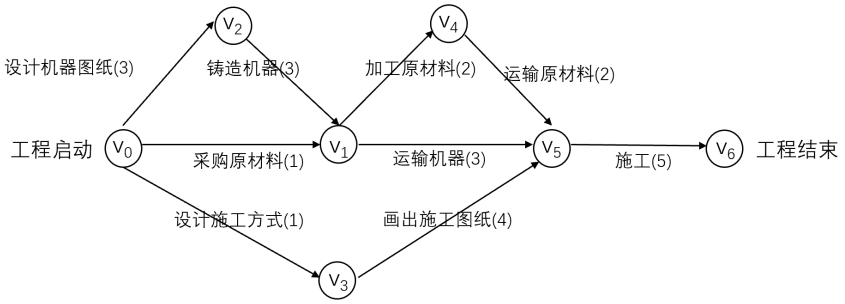



图 5.22 铺设线缆工程

示工程结束，每条边表示某个活动，边权为该活动需要的时间，并且边的方向表示活动之间的先后约束；每个顶点表示一个事件，如顶点 $v_2$ 表示“图纸已设计完成”，顶点 $v_5$ 表示“所有施工准备已完成”。

这种用顶点表示事件，有向边表示活动，边的权值表示完成该活动所需要的时间的有向带权图被称为 **AOE 网（Activity On Edge Network，边表示活动的网）**。AOE 网规定：

1） 只有当某顶点所代表的事件发生后，该顶点的各条出边所代表的活动才能开始；

2） 只有当某顶点的所有入边所代表的活动均完成后，该顶点所代表的事件才能发生。

在 AOE 网中，有且仅有一个入度为 0 的顶点，称为源点，也称开始顶点，也有且仅有一个出度为 0 的顶点，称为汇点，也称结束顶点。

显然，AOE 网中有些活动是可以并行进行的，虽然不同路径上的活动所需要的时间不同，但只有从源点到汇点所有路径上的活动都已完成，整个工程才算完成。因此，整个工程的完成时间取决于源点到汇点所有路径中长度最长（时间最长）的那条路径，这条路径便被称为**关键路径**，关键路径上的所有活动均被称为**关键活动**。若关键活动所需时间加长，则整个工程所需时间都会加长。

显然对于一个 AOE 网，关键路径可能有多条。

### 5.7.1　关键路径的求解

事实上，**对于顶点数量不多的 AOE 网，人工求解关键路径最简单的办法就是观察法**，直接看哪条路径上边的权值之和最大，谨慎些不要看错即可。

对于需要计算机解决的复杂图，为引入关键路径的求解方式，首先请大家思考如下几个问题：

**各事件最早发生时间的求解**

如果每项活动一旦具备条件就立刻开始，工程中的每个事件 $v_i$ 最早什么时候能发生？我们记这个时间为 $ve(i)$（e, **e**arliest），称为事件 $v_i$ 的**最早发生时间**。

显然对于任意事件 $i$，它必须等到它的所有前置活动中，最晚结束的那个结束后才能发生。记 $j_1$, ..., $j_n$ 为 $i$ 的 $n$ 条入边对应的顶点，cost 为相应边的权（活动花费的时间），则有：

$$ve(i) = \max\big(ve(j_1) + cost(j_1, i), ..., ve(j_n) + cost(j_n, i)\big)$$

那么，当 $i$ 的所有前置事件的最早发生时间都已求解之后，事件 $i$ 的最早发生时间也就求解好了。而开始顶点的 $ve(0)$ 显然为 0，于是，由源点开始，按照拓扑序列的顺序即可逐一求得所有顶点的最早发生时间 $ve(i)$。

**提示**

为什么是拓扑序列？因为在拓扑序列中，每个事件（顶点）都排在其所有前置事件之后，因此当我们处理 $v_i$ 时，它的所有前驱顶点一定都已经处理完毕，其最早发生时间也就都已求得，即等式右边的所有项均已求得：

$$ve(i) = \max\big(ve(j_1) + cost(j_1, i), ..., ve(j_n) + cost(j_n, i)\big)$$

如图 5.23 所示，该图的一个拓扑序列是 0231456，因此可以逐步求解出：

$ve(0) = 0$;
$ve(2) = max(ve(0) + cost(0, 2)) = max(0 + 3) = 3$;
$ve(3) = max(ve(0) + cost(0, 3)) = max(0 + 1) = 1$;
$ve(1) = max(ve(0) + cost(0, 1), ve(2) + cost(2, 1)) = max(0 + 1, 3 + 3) = 6$;
$ve(4) = max(ve(1) + cost(1, 4)) = 8$;
$ve(5) = max(ve(4) + cost(4, 5), ve(1) + cost(1, 5), ve(3) + cost(3, 5)) = 10$;
$ve(6) = max(ve(5) + cost(5, 6)) = 15$;

也就是说整个工程最早能在第 15 天结束。

**各事件最迟发生时间的求解**

现在换一个角度思考：既然工程最早能在第 15 天结束，那么每个事件 $v_i$ 最迟可以拖到什么时候开始，才不会导致工程延期？我们记这个时间为 $vl(i)$（l, **l**atest），称为事件 $v_i$ 的**最迟发生时间**。

显然对于任意事件 $i$，它必须要在所有后置事件中，所要求的最迟发生时间最早的（最小的）事件所允许的时间范围内开始。也就是说，记 $j_1$, ..., $j_n$ 为事件 $i$ 的所有出边所指向的顶点，$cost$ 为相应边的权（活动花费的时间），则一定有：

$$vl(i) = \min\big(vl(j_1) - cost(i, j_1), \ldots, vl(j_n) - cost(i, j_n)\big)$$

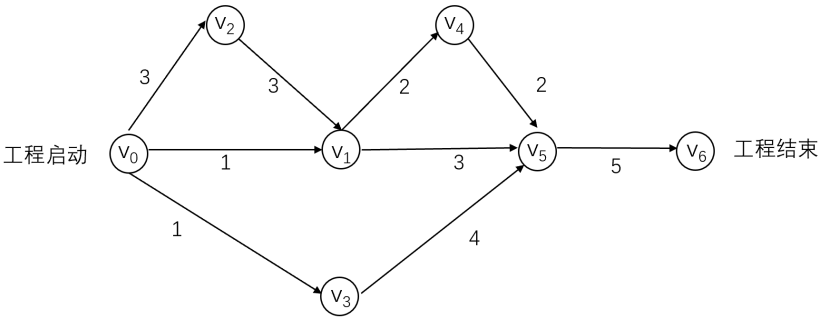



图 5.23

那么，当 $i$ 的所有后继事件的最迟发生时间都已求解之后（上式右边的所有项均已知），事件 $i$ 的最迟发生时间也就求解好了。而终点的最迟发生时间 $vl(N-1)$ 等于整个工程的最早完成时间，于是，由终点开始，按照**逆拓扑序列**的顺序即可逐一求得所有顶点的最迟发生时间 $vl(i)$。

> **逆拓扑序列**就是将拓扑序列倒过来。如拓扑序列是 0231456，逆拓扑序列就是 6541320。

对于图 5.23，我们已经根据该图的一个拓扑序列 0231456 求解出了 $ve$，同样地可以根据逆拓扑序列 6541320 求解出 $vl$：

$$
\begin{aligned}
&vl(6) = ve(6) = 15;\\
&vl(5) = \min(vl(6) - cost(5,6)) = 15 - 5 = 10;\\
&vl(4) = \min(vl(5) - cost(4,5)) = 10 - 2 = 8;\\
&vl(1) = \min(vl(4) - cost(1,4), vl(5) - cost(1,5)) = \min(8-2, 10-3) = 6;\\
&vl(3) = \min(vl(5) - cost(3,5)) = 10 - 4 = 6;\\
&vl(2) = \min(vl(1) - cost(2,1)) = 6 - 3 = 3;\\
&vl(0) = \min(vl(2) - cost(0,2), vl(3) - cost(0,3), vl(1) - cost(0,1)) = \min(3-3, 6-1, 6-1) = 0;
\end{aligned}
$$

### 求解关键路径

我们现在已经求解出了每个事件的最早发生时间以及最迟发生时间：

| | $v_0$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ |
|---|---|---|---|---|---|---|---|
| 最早发生时间 ve | 0 | 6 | 3 | 1 | 8 | 10 | 15 |
| 最迟发生时间 vl | 0 | 6 | 3 | 6 | 8 | 10 | 15 |

显然

最迟发生时间 - 最早发生时间

的差值就是不延期整个工程的前提下，活动可以推迟的时间数量。所以满足 $ve(i) = vl(i)$ 的所有事件才有可能是关键路径上的顶点。那么如何判断该顶点的哪条出边是关键活动，进而构造关键路径？

考虑到一条 $i$ 到 $j$ 的边表示的活动的时间余量可以表示为：

$$\begin{aligned} \text{余量}(i,j) &= l(i,j) - e(i,j) \\ &= vl(j) - cost(i,j) - ve(i) \end{aligned}$$

其中 $l(i,j)$ 为活动的最迟开始时间，$e(i,j)$ 为活动的最早开始时间。

若某活动为关键活动，其时间余量为 0：

$$vl(j) - cost(i,j) - ve(i) = 0$$

即

$$ve(i) = vl(j) - cost(i,j)$$

这就是判断关键活动的依据。

因此我们找出所有满足 $ve(i) = vl(i)$ 的关键事件，对于这些事件之间的有向边 $i \to j$，进一步判断是否满足：

$$ve(i) = vl(j) - cost(i,j)$$

若满足，说明这个活动是一个关键活动。这样，我们找到所有关键活动，这些关键活动便构成了关键路径。注意：AOE 网的关键路径可能不唯一，但通过这种方式将所有关键活动连接起来，可以准确求出所有关键路径。

根据以上讨论，对于有 $N$ 个顶点的 AOE 网，可以写出求解关键路径的代码如下（邻接表为例）。

**代码 5.1　拓扑排序求解关键路径**

```
struct AdjEdge {
    int to;         // 活动的终点
    int duration;   // 活动持续时间
};

// 当图中存在环时，返回 -1；否则返回关键路径的长度
int CriticalPath(int N, vector<AdjEdge> AdjList[]) {
    int in_deg[N] = {};
    // 计算各顶点的入度，为拓扑排序做准备
    for (int u = 0; u < N; u++) {
```

```
        for (int i = 0; i < AdjList[u].size(); i++) {
            int v = AdjList[u][i].to;
            in_deg[v]++;
        }
    }

    // 第一步：拓扑排序，计算事件最早发生时间 ve
    vector<int> ve(N, 0);   // 初始有 N 个元素，值均为 0 的 vector
    vector<int> topo_seq;
    queue<int> q;
    for (int i = 0; i < N; i++) {
        if (in_deg[i] == 0) {
            q.push(i);
        }
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        topo_seq.push_back(u);
        // 遍历 u 的所有邻接点，尝试更新它们的最早开始时间 ve
        for (int i=0; i<AdjList[u].size(); i++) {
            AdjEdge e = AdjList[u][i];
            ve[e.to] = max(ve[e.to], ve[u] + e.duration);
            if (--in_deg[e.to] == 0) {
                q.push(e.to);
            }
        }
    }

    // 图中存在环
    if (topo_seq.size() != N) {
        return -1;
    }

    // 假设不确定汇点的编号，只能遍历一遍所有顶点，ve 最大的那个就是汇点。
    int critical_path_length = 0;
    for (int i = 0; i < N; i++) {
        critical_path_length = max(critical_path_length, ve[i]);
    }

    // 第二步：用逆拓扑序列计算事件最迟发生时间 vl
    vector<int> vl(N, critical_path_length);    // 初始有 N 个元素，值均为
    ↪  critical_path_length 的 vector
    for (int i=N-1; i>=0; i--) {
        int u = topo_seq[i];
        for (int j=0; j<AdjList[u].size(); j++) {
```

```
57              AdjEdge e = AdjList[u][j];
58              vl[u] = min(vl[u], vl[e.to] - e.duration);
59          }
60      }
61
62      // 第三步：找出关键活动
63      for (int u=0; u<N; u++) {
64          // 若不满足 ve(u) == vl(u) 说明 u 不是关键事件，则 u 发出的边不可能是关
            ↪ 键活动
65          if(ve[u] != vl[u]) {
66              continue;
67          }
68          for (int i=0; i<AdjList[u].size(); i++) {
69              AdjEdge e = AdjList[u][i];
70              if (ve[u] == vl[e.to] - e.duration) {
71                  // 找到了一个关键活动：u -> e.to，记录下来
72              }
73          }
74      }
75
76      // 返回关键路径长度
77      return critical_path_length;
78  }
```

注意：假设不需要求出关键路径是什么（轨迹），而只是求出关键路径的长度即可，那么代码到第 49 行就已经达成任务了。另外，数组 ve[] 是各个顶点的最早发生时间，换个说法，对于有向无环图（DAG），以上代码求解出的 ve[] 数组也是源点到各个顶点的**最长路径长度**。

*** 思维拓展**

前 49 行求解最长路径长度的过程其实是一种**动态规划**的思想：将一个较大的问题拆分成若干个较小的子问题，通过求解子问题来推导出更大的子问题的解，最终推导出原问题的解。它的核心是：找到问题的子问题结构，以及子问题答案之间的依赖关系。如上述求最长路径长度的问题，假设汇点为 $N$，那么原问题的解 $ve[N]$ 可以由若干个子问题的解推导出来：

$$ve(N) = \max\big(ve(j_1) + cost(j_1, N), ..., ve(j_n) + cost(j_n, N)\big)$$

其中 $j_1, j_2, ..., j_n$ 是所有 $N$ 的入边对应的顶点。这些子问题的解又可以由它们的子问题的解推导出来。而 ve[0] 则是边界条件，是最小的不可再分的子问题。于是根据 ve[0] 一步一步向后推导，就能逐步得出更大的子问题的解，直至得出原问题的解。

在代码实现上，动态规划通常有两条路径：

- 自顶向下 + 记忆化搜索：从原问题出发，采用递归形式。当遇到一个子问题时，先去缓存（记忆）里查一下算过没有，算过了直接返回，没算过就递归计算并将结果存入缓存（记忆下来）。

- 自底向上 + 状态转移表：从最小的子问题开始算起，一步步迭代推导到原问题。这是最经

典、最常用的动态规划写法。代码 5.1 求解关键路径正是这种方式，拓扑排序解决的是“先计算谁”，动态规划解决的是“如何计算”。

对于上述代码，统计所有顶点入度的时间复杂度为 $O(N+M)$，计算 ve[]、vl[] 的过程为 $O(N+M)$，找出关键活动的过程复杂度为 $O(N+M)$，因此算法整体时间复杂度为 $O(N+M)$。空间方面，引入了各种长度为 $N$ 的数组以及一个队列，队列的空间最坏情况下为 $N$ 个元素，因此空间复杂度为 $O(N)$。

经过一场大战，王国成功击退了入侵者，虽然保住了所有据点，但王国的一些道路已经被摧毁，他们决定重新修建道路，将所有城镇连通。

# — BONFIRE ENCOUNTERED —

**习题 5** （参考答案见习题 5 参考答案，第 179 页）

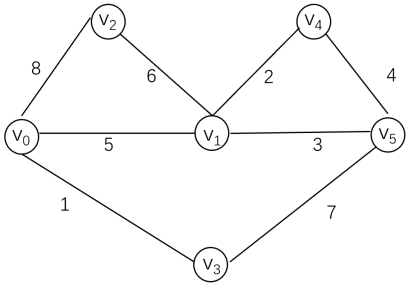
1. 对于一个有 $N$（$N > 0$）个顶点的无向连通图，下列说法错误的是_____

A. 其最小生成树一定唯一
B. 其最小生成树可能不唯一
C. 其最小生成树一定有 $N$ 个顶点
D. 其最小生成树一定有 $N - 1$ 条边

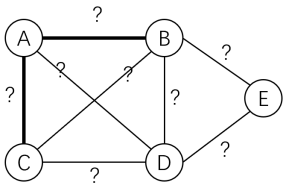
2. 下列说法正确的是_____

A. 对同一个图，Kruskal 算法和 Prim 算法得到的最小生成树一定相同
B. 只要无向连通图中没有权值相同的边，其最小生成树一定唯一
C. 只要无向连通图中有权值相同的边，其最小生成树一定不唯一
D. 地球的体积比太阳大

3. 使用 Kruskal 算法求解下图的最小生成树，依次选取的边的顺序是什么？




4. 下图是某个隐去了边权的无向连通图，现在用 Kruskal 算法求解其最小生成树，假设现在已经收入了 (A, B) 和 (A, C) 两条边，则下一个收入的边绝不可能是_____



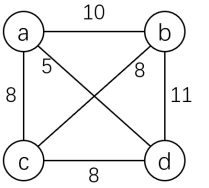

A. (B,C)　B. (B,D)　C. (C,D)　D. (D,E)

5. （某年 408 真题）下列关于最小生成树的说法中，正确的是_____

I. 最小生成树的代价唯一
II. 所有权值最小的边一定会出现在所有的最小生成树中
III. 使用 Prim 算法从不同顶点开始得到的最小生成树一定相同
IV. 使用 Prim 算法和 Kruskal 算法得到的最小生成树总不相同

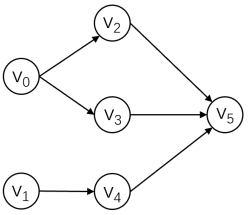
A. 仅 I　B. 仅 II　C. 仅 I、III　D. 仅 II、IV

6. （某年 408 真题）求下面带权图的最小生成树时，可能是 Kruskal 算法第 2 次选中但一定不是 Prim 算法（从 d 开始）第 2 次选中的边的是_____




A. (a, c)　B. (a, d)　C. (b, c)　D. (c, d)

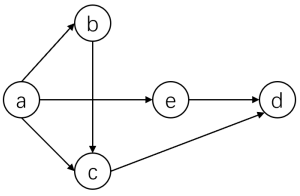
7. 对下图所示的有向无环图进行拓扑排序，不可能得到的拓扑序列是_____




A. $v_0, v_1, v_2, v_3, v_4, v_5$　B. $v_1, v_0, v_4, v_3, v_2, v_5$

C. $v_0, v_1, v_4, v_3, v_2, v_5$　D. $v_1, v_0, v_5, v_4, v_3, v_2$

8. （某年 408 真题）对下图所示的有向无环图进行拓扑排序，可能得到_____ 个不同的拓扑序列

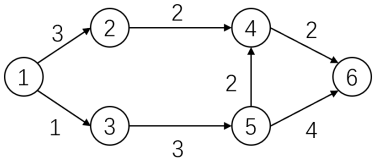



A. 1　B. 2　C. 3　D. 4

9. 对于有向图无环图 $G$ 的一个拓扑序列，若顶点 $v_1$ 在顶点 $v_3$ 之前，请判断下列命题的正确性。

A. 一定存在有向边：$(v_1, v_3)$
B. 可能存在有向边：$(v_3, v_1)$
C. 一定存在一条从 $v_1$ 到 $v_3$ 的路径
D. 可能存在一条从 $v_3$ 到 $v_1$ 的路径
E. 可能存在一条从 $v_1$ 到 $v_3$ 的路径

10. 对于下图所示的 AOE 网，其关键路径长度的是_____

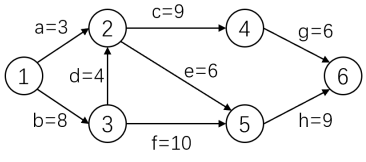



A. 6　B. 7　C. 8　D. 9

11. （某年 408 真题）若使用 AOE 网估算工程进度，下列叙述中正确的是_____

A. 关键路径是从源点到汇点边数最多的一条路径
B. 关键路径是从源点到汇点路径长度最长的一条路径
C. 增加任意一个关键活动的时间不会延长工程的工期
D. 缩短任意一个关键活动的时间将会缩短工程的工期

12. （某年 408 真题）下列 AOE 网表示一项包含 8 个活动的工程。则同时加快_____ 中的两个活动就可以缩短工程工期



A. c 和 e　B. d 和 c　C. f 和 d　D. f 和 h

13. 经历了一场战争后，王国的一些道路遭到了严重破坏。国王决定重建城市之间的道路，使得所有城市重新连通。王国共有 $N$ 座城市，编号为 0 到 $N-1$。工程师经过地质考察，给出了 $M$ 条可以修建的道路，每条道路连接两座不同的城市，并给出了修建这条道路所需的费用。此外，部分城市之间已经存在完好的旧道路。这些旧道路可以直接使用，不需要重新修建。

给出所有尚存的旧道路（保证不存在环）和可以修建的道路，请你帮助国王计算：在保证任意两座城市之间都能够互相到达的前提下，至少还需要花费多少费用？如果无论如何都无法使所有城市连通，则返回 -1。

```
struct Road {
    int u, v;   // 已有旧道路连通的两个城市编号
};
struct NewRoad {
    int u, v;   // 修建该道路后，将连通的两个城市的编号
    int weight; // 修建该道路需要的花费
};
int minimumCost(
    int N,
    vector<Road>& existing_roads,
    vector<NewRoad> new_roads
) {

}
```

14. 王国的重建工程被划分为 N 项任务，编号为 0 到 N-1。然而，部分任务之间存在先后依赖关系。例如，必须先完成地基施工，才能开始墙体建设；必须先完成墙体建设，才能安装城堡屋顶。工程师整理出了 M 条任务依赖关系。每条依赖关系由两个整数 u 和 v 表示，意味着任务 u 必须在任务 v 之前完成。请你帮助国王安排这 N 项任务的执行顺序，使得所有任务都能按照依赖关系完成，返回代表任务执行顺序的数组。如果存在任务依赖关系上的矛盾，导致所有任务无法安排出一个合法的执行顺序，则返回一个空数组。如果存在多种合法的执行顺序，返回其中任意一种即可。

```
// v1、v2 的长度均为 M，用于表示任务的依赖关系：对于 0 <= i < M，任务 v1[i] 必须
↪  在 v2[i] 之前完成。
vector<int> TopologicalSort(int N, int M, vector<int>& v1, vector<int>& v2) {

}
```

*15. 王国的重建工程有 N 项任务，编号为 $1, 2, \ldots, N$。为了尽快完成不同任务，各方工作人员可以同时进行多项互不冲突的任务。对于第 $i$ 项任务：
- 完成它需要花费 $T_i$ 个时间单位。
- 在开始第 $i$ 项任务之前，必须先完成若干项指定的任务。
- 如果某项任务没有前置依赖，那么它可以立即开始。

给出 $T[]$ 以及每项任务的直接前置依赖 Pre[]，请你计算：完成全部 $N$ 项任务，最少需要多少时间（保证依赖关系不存在矛盾）？

```
// T[] 与 Pre[] 长度均为 N，T[i] 表示任务 i 的花费时间；Pre[i] 表示任务 i 的所有直接前置依赖
int MinTime(int N, int T[], vector<int> Pre[]) {

}
```

# 第六章　排序

排序就是按照关键字的大小顺序，将一组元素排成有序序列的过程。例如要对一组玩家排序，将分数作为关键字，就得到了玩家的分数排行榜，将等级作为关键字，就得到了等级排行榜，等。大家在此前一定也写过一些排序算法的代码。对于待排序列中任意两个关键字相等的元素 $E_1$、$E_2$，若排序前 $E_1$ 在 $E_2$ 的前面，经某排序算法排序后仍然如此，则称该排序算法是**稳定的**；若排序后有可能使得 $E_1$ 在 $E_2$ 的后面，则称该排序算法是**不稳定的**。

排序当然可以从小到大或从大到小排，**按照惯例，本章中若无特别说明，均默认从小到大排序**，即排序后的序列 $(E_1, E_2, ..., E_n)$，其关键字满足：

$$k_1 <= k_2 <= ... <= k_n$$

提到排序，相信大家很自然地就能想到以下非常直观的几个算法：

1） 每次在序列中选择一个最小的放在最前面，也就是第 $i$ 次选择序列中第 $i$ 小的元素放在第 $i$ 个位置，选择 $n$ 次就排好序了。这种算法就是**选择排序**算法。

2） 从序列的开头到结尾，不断地比较 a[i] 与 a[i+1]，若 a[i] > a[i+1] 就交换二者，这样做，一趟下来就能将整个序列中最大的元素“浮”到末尾。重复 $n$ 次这个操作，排序就排好了。这种算法就是**冒泡排序**算法。因每次将最大的元素“浮”到末尾的过程就像一个气泡向上冒一样而得此名。

3） 将待排序列看成前 $k$（初始认为 $k = 0$ 或 $k = 1$）个元素已经排好序的序列。每次取第 $k + 1$ 个元素，插入到前面有序序列的合适位置，使得新的前 $k + 1$ 个元素有序。重复 $n$ 次这个操作排序就排好了。这就是**插入排序**。

除了这几种算法之外，先贤们还发明出了很多更加高效、有趣的排序算法，我们将在本章介绍各种排序算法。

我们曾在 5.5.3 节引入的，C++ 标准库中为我们提供的 std::sort() 函数使用的是一种混合了快速排序、堆排序和插入排序的组合算法，它的基本思路是：

1） 正常情况下使用快速排序。快速排序平均效率很高，平均时间复杂度为 $O(N \log N)$。算法同时监控快速排序的递归深度。

2） 如果快速排序递归过深，说明可能出现了不理想的划分，便切换为堆排序，使得最坏情况下仍然保证 $O(N \log N)$ 的时间复杂度。

3） 处理较小的子区间时使用插入排序。因为对于规模较小的数据，插入排序通常具有较低的常数开销。

而 std::stable_sort() 为了保证排序的稳定性，通常采用归并排序实现，最差时间复杂度同样为 $O(nlogn)$。如果你感兴趣，可以在本章内容结束后回看这部分内容。

不开源了哦

# 习题参考答案

## 习题 7 参考答案

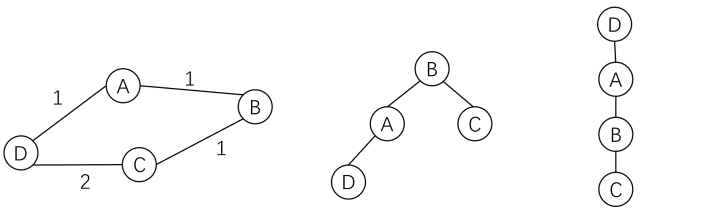
1. 选 A。极限思维，若所有边权值都相等，任选 $N-1$ 条无回路的边，构成的一定是最小生成树。C、D 显然正确。

2. 选 B。D 显然错误；考虑 B，无论 Prim 算法还是 Kruskal 算法，每次选边时，选择的都是距离或权值最小的一条边，因此若图中不存在权值相同的边，每次选择的边一定是唯一的，最终得到的最小生成树就是唯一的，因此 B 正确；对于 A 和 C 则很容易举出反例。

3. 依次选取权值为：1,2,3,5,6 的边。选了 1、2、3 后不能选 4，会有环。

4. 选 A 选项。顶点 B 和 C 此时已经处在同一个连通块了，根据 Kruakal 算法，不能选。

5. 选 A 选项。对于 II，考虑所有边权值相等的情况，若图中存在环则一定有些边不会出现在最小生成树中，错误；对于 III，考虑下图：

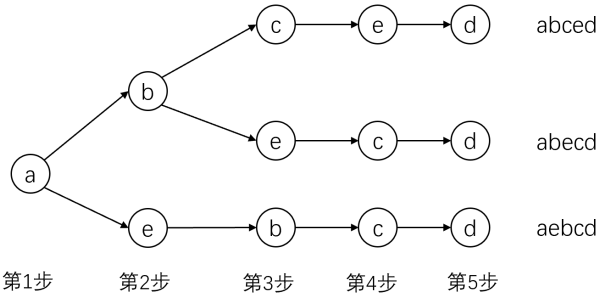



III 错误；对于 IV，考虑仅有两个顶点、一条边的图，其最小生成树只可能是它本身，两种算法得到的一定相同。

6. 选 C。容易得知 Kruskal 算法第二次选择的边一定是三条权值为 8 的边之一：(b,c)、(a,c)、(d,c)。而 Prim 算法从 d 开始，第一次选择的一定是 (d, a)，第二次可能选择 (a, c) 或 (d, c)，因此 (b,c)、(a,c)、(d,c) 中，Prim 算法一定不会选择的就是 (b,c)。

7. 选 D。$v_2$、$v_3$、$v_4$ 均存在一条到 $v_5$ 的路径，因此拓扑序列中不能排在 $v_5$ 的后面。

8. 选 C。由每一步可选的顶点构成的选择树如下所示：

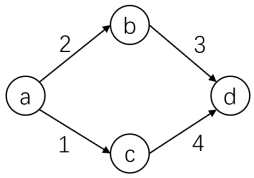



9. A、B、C、D 均错误，仅 E 正确。

对于 A、C，考虑本习题第 7 题的图的 A 选项的拓扑序列，$v_1$ 排在 $v_3$ 前面，但是不存在 $v_1$ 到 $v_3$ 的路径，因此 A、C 错误；对于 B、D，拓扑排序要求若 $v$ 到 $u$ 有一条路径，则 $u$ 一定排在 $v$ 的后面，因此 B、D 错误。E 显然正确。

10. 选 C。观察法很容易看出关键路径有 2 条，长度是 8。

11. 选 B。A 显然错误；AOE 网中关键路径可能存在多条，但增加关键活动的时间一定会增加工程的工期，C 选项说法有误。而减小关键活动的时间则不一定会减少工程的工期，如下图：




上下两条路都是关键路径，边 $(a,c)$ 是关键活动，但减少 $(a,c)$ 的时间不会减少工程的工期。因此 D 错误。

12. 选 C。通过观察法可以看出图中存在 3 条关键路径：bdcg、bdeh 和 bfh。仅 f 和 d 组合到一起能够构成三条路径的必经之路，因此加快 f 和 d 能使得三条路径同时变短。其余三种组合都至少有一条它们不能使之变短的关键路径。

13. 这显然是求最小生成树的问题，只不过已经选好了一些边，这些边不计入花费。我们根据 Kruskal 算法从剩下的边中进行选择即可。本题意在帮助大家复习并查集实现以及 Kruskal 算法。代码如下：

```
1 struct Road {
2     int u, v;
```

```
};
struct NewRoad {
    int u, v;   // 修建该道路后，将连通的两个城市的编号
    int weight; // 修建该道路需要的花费
};

// 提供给 std::sort() 的比较器
bool CmpNewRoad(NewRoad& r1, NewRoad& r2) {
    return r1.weight < r2.weight;
}

int Find(int father[], int x) {
    int ret = x;
    while(ret != father[ret]) {
        ret = father[ret];
    }
    int to_adjust = x;
    while(to_adjust != ret) {
        int fa = father[to_adjust];
        father[to_adjust] = ret;
        to_adjust = fa;
    }
    return ret;
}

int Union(int father[], int x, int y) {
    int x_id = Find(father, x);
    int y_id = Find(father, y);
    if(x_id != y_id) {
        father[x_id] = y_id;
        return 1;
    }
    return 0;
}

int minimumCost(
    int N,
    vector<Road>& existing_roads,
    vector<NewRoad> new_roads
) {
    int ret = 0;
    int father[N];
    for(int i=0; i<N; i++) {
        father[i] = i;
    }
    int edge_num = 0;
    // 根据尚存道路计算已有的边数，并合并这些道路连通的顶点
```

```
    for(int i=0; i<existing_roads.size(); i++) {
        Union(father, existing_roads[i].u, existing_roads[i].v);
        edge_num += 1;
    }
    // 按 CmpNewRoad 的比较规则对 new_roads 进行排序
    sort(new_roads.begin(), new_roads.end(), CmpNewRoad);

    // 按从小到大遍历边
    for(int i=0; i<new_roads.size(); i++) {
        int fau = Find(father, new_roads[i].u);
        int fav = Find(father, new_roads[i].v);
        if(fau != fav) {
            Union(father, new_roads[i].u, new_roads[i].v);
            ret += new_roads[i].weight;
            edge_num += 1;
        }
    }
    if(edge_num < N-1) {
        return -1;
    }
    return ret;

}
```

在尚存的旧道路保证不存在环的前提下，通过边数是否等于 $N-1$ 即可判断连通性。若尚存的旧道路有可能存在环，则可以通过并查集判断所有城市是否处于同一个连通块：

```
// 所有顶点处于同一个连通分量 <=> 所有顶点与 0 号顶点的连通分量编号相同
int root = Find(father, 0);
for(int i = 1; i < N; i++) {
    if(Find(father, i) != root) {
        return -1;
    }
}
```

14. 本题意在帮助大家练习拓扑排序的代码实现。没有什么难度，灵活处理输入数据即可。代码如下。

```
// v1、v2 的长度均为 M，用于表示任务的依赖关系：对于 0 <= i <= M，任务 v1[i] 必须在 v2[i] 之前完成。
vector<int> TopologicalSort(int N, int M, vector<int>& v1, vector<int>& v2) {
    vector<int> ret;
```

```
    vector<int> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    queue<int> q;
    // 根据任务的依赖关系建图
    for(int i=0; i<M; i++) {
        int u = v1[i];
        int v = v2[i];
        AdjList[u].push_back(v);
        in_deg[v] += 1;
    }
    // 入度为 0 的顶点入队
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
        }
    }
    while(!q.empty()) {
        int vertex = q.front();
        q.pop();
        ret.push_back(vertex);
        for(int i=0; i<AdjList[vertex].size(); i++) {
            int u = AdjList[vertex][i];
            in_deg[u] -= 1;
            if(in_deg[u] == 0) {
                q.push(u);
            }
        }
    }
    if(ret.size() < N) {
        return vector<int>();
    }
    return ret;
}
```

15. 需要注意的是，与求解关键路径长度不同，本题未必只有一个源点，但动态规划的思想仍然是成立的：我们依然可以根据拓扑序列的顺序，递推地从所有无前置任务的顶点推出各个任务的完成时间。具体来说，我们设置数组 max_len[]，max_len[i] 表示完成 i 的所有前置任务所需的时间。那么任务 i 的完成时间就是 max_len[i] + T[i]。

```cpp
// T[] 与 Pre[] 长度均为 N，T[i] 表示任务 i 的花费时间；Pre[i] 表示任务 i 的所有直接前置依赖
int MinTime(int N, int T[], vector<int> Pre[]) {
    struct AdjEdge {
        int u;
        int weight;
        // 如果你一路走到了这里，相信自己学会构造函数对你来说应该很简单了。
        AdjEdge(int u, int weight) : u(u), weight(weight) {}
    };
    vector<AdjEdge> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    // 遍历所有有向边，建立邻接表
    for(int i=0; i<N; i++) {
        vector<int> pres = Pre[i];
        for(int j=0; j<pres.size(); j++) {
            int u = pres[j];
            // 一条 u -> i 的有向边，其权值为 T[u]
            AdjList[u].push_back(AdjEdge(i, T[u]));
            in_deg[i] += 1;
        }
    }
    // 求解各顶点的最长路径长度（从任意源点到该顶点的路径长度中最长的那个）
    int max_len[N];
    // 数组初始化为 0
    memset(max_len, 0, sizeof(max_len));
    queue<int> q;
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
            max_len[i] = 0;
        }
    }

    while(!q.empty()) {
        int u = q.front();
        q.pop();
        // 遍历 u 的所有邻接点 v
        for(int i=0; i<AdjList[u].size(); i++) {
            int v = AdjList[u][i].u;
            int weight = AdjList[u][i].weight;
            // 尝试更新 v 的 max_len[]
            max_len[v] = max(max_len[v], max_len[u] + weight);
            in_deg[v] -= 1;
            if(in_deg[v] == 0) {
```

```
                q.push(v);
            }
        }
    }
    /* 注意：顶点的最长路径长度表示的是，完成该顶点（任务）的所有前置任务
    所需的最长时间，因此最长路径长度再加上该任务所需时间才是该任务的最早
    完成时间 */
    int ret = 0;
    for(int i=0; i<N; i++) {
        ret = max(ret, max_len[i]+T[i]);
    }
    return ret;

}
```

事实上以上代码的图模型更接近 AOV 网，用顶点表示活动，而非 AOE 网。u 的邻接表中的权值可以理解为 u 的点权（活动需要的时间）。

动态规划的过程也可以直接设置数组 done[]，直接表示各个任务完成的最早时间，将数组初始化为各个任务所需的时间 T[i]，之后进行递推，这样是更简洁的：

```
for (int i=0; i<AdjList[u].size(); i++) {
    int v = AdjList[u][i];
    done[v] = max(done[v], done[u] + T[v]);
    in_deg[v] -= 1;
    if(in_deg[v] == 0) {
        q.push(v);
    }
}
```

## 习题 6 参考答案

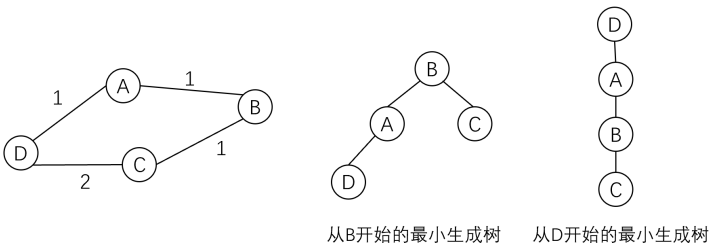
1. 选 A。极限思维，若所有边权值都相等，任选 $N-1$ 条无回路的边，构成的一定是最小生成树。C、D 显然正确。

2. 选 B。D 显然错误；考虑 B，无论 Prim 算法还是 Kruskal 算法，每次选边时，选择的都是距离或权值最小的一条边，因此若图中不存在权值相同的边，每次选择的边一定是唯一的，最终得到的最小生成树就是唯一的，因此 B 正确；对于 A 和 C 则很容易举出反例。

3. 依次选取权值为：1,2,3,5,6 的边。选了 1、2、3 后不能选 4，会有环。

4. 选 A 选项。顶点 B 和 C 此时已经处在同一个连通块了，根据 Kruakal 算法，不能选。

5. 选 A 选项。对于 II，考虑所有边权值相等的情况，若图中存在环则一定有些边不会出现在最小生成树中，错误；对于 III，考虑下图：

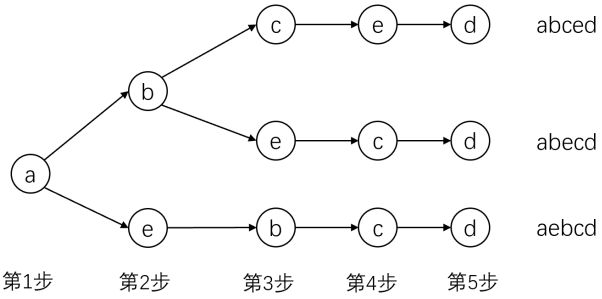



III 错误；对于 IV，考虑仅有两个顶点、一条边的图，其最小生成树只可能是它本身，两种算法得到的一定相同。

6. 选 C。容易得知 Kruskal 算法第二次选择的边一定是三条权值为 8 的边之一：(b,c)、(a,c)、(d,c)。而 Prim 算法从 d 开始，第一次选择的一定是 (d, a)，第二次可能选择 (a, c) 或 (d, c)，因此 (b,c)、(a,c)、(d,c) 中，Prim 算法一定不会选择的就是 (b,c)。

7. 选 D。$v_2$、$v_3$、$v_4$ 均存在一条到 $v_5$ 的路径，因此拓扑序列中不能排在 $v_5$ 的后面。

8. 选 C。由每一步可选的顶点构成的选择树如下所示：

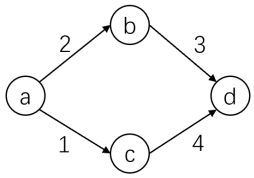



9. A、B、C、D 均错误，仅 E 正确。

对于 A、C，考虑本习题第 7 题的图的 A 选项的拓扑序列，$v_1$ 排在 $v_3$ 前面，但是不存在 $v_1$ 到 $v_3$ 的路径，因此 A、C 错误；对于 B、D，拓扑排序要求若 $v$ 到 $u$ 有一条路径，则 $u$ 一定排在 $v$ 的后面，因此 B、D 错误。E 显然正确。

10. 选 C。观察法很容易看出关键路径有 2 条，长度是 8。

11. 选 B。A 显然错误；AOE 网中关键路径可能存在多条，但增加关键活动的时间一定会增加工程的工期，C 选项说法有误。而减小关键活动的时间则不一定会减少工程的工期，如下图：




上下两条路都是关键路径，边 $(a, c)$ 是关键活动，但减少 $(a, c)$ 的时间不会减少工程的工期。因此 D 错误。

12. 选 C。通过观察法可以看出图中存在 3 条关键路径：bdcg、bdeh 和 bfh。仅 f 和 d 组合到一起能够构成三条路径的必经之路，因此加快 f 和 d 能使得三条路径同时变短。其余三种组合都至少有一条它们不能使之变短的关键路径。

13. 这显然是求最小生成树的问题，只不过已经选好了一些边，这些边不计入花费。我们根据 Kruskal 算法从剩下的边中进行选择即可。本题意在帮助大家复习并查集实现以及 Kruskal 算法。代码如下：

```
1 struct Road {
2     int u, v;
```

```
};
struct NewRoad {
    int u, v;   // 修建该道路后，将连通的两个城市的编号
    int weight; // 修建该道路需要的花费
};

// 提供给 std::sort() 的比较器
bool CmpNewRoad(NewRoad& r1, NewRoad& r2) {
    return r1.weight < r2.weight;
}

int Find(int father[], int x) {
    int ret = x;
    while(ret != father[ret]) {
        ret = father[ret];
    }
    int to_adjust = x;
    while(to_adjust != ret) {
        int fa = father[to_adjust];
        father[to_adjust] = ret;
        to_adjust = fa;
    }
    return ret;
}

int Union(int father[], int x, int y) {
    int x_id = Find(father, x);
    int y_id = Find(father, y);
    if(x_id != y_id) {
        father[x_id] = y_id;
        return 1;
    }
    return 0;
}

int minimumCost(
    int N,
    vector<Road>& existing_roads,
    vector<NewRoad> new_roads
) {
    int ret = 0;
    int father[N];
    for(int i=0; i<N; i++) {
        father[i] = i;
    }
    int edge_num = 0;
    // 根据尚存道路计算已有的边数，并合并这些道路连通的顶点
```

```
    for(int i=0; i<existing_roads.size(); i++) {
        Union(father, existing_roads[i].u, existing_roads[i].v);
        edge_num += 1;
    }
    // 按 CmpNewRoad 的比较规则对 new_roads 进行排序
    sort(new_roads.begin(), new_roads.end(), CmpNewRoad);

    // 按从小到大遍历边
    for(int i=0; i<new_roads.size(); i++) {
        int fau = Find(father, new_roads[i].u);
        int fav = Find(father, new_roads[i].v);
        if(fau != fav) {
            Union(father, new_roads[i].u, new_roads[i].v);
            ret += new_roads[i].weight;
            edge_num += 1;
        }
    }
    if(edge_num < N-1) {
        return -1;
    }
    return ret;

}
```

在尚存的旧道路保证不存在环的前提下，通过边数是否等于 $N-1$ 即可判断连通性。若尚存的旧道路有可能存在环，则可以通过并查集判断所有城市是否处于同一个连通块：

```
// 所有顶点处于同一个连通分量 <=> 所有顶点与 0 号顶点的连通分量编号相同
int root = Find(father, 0);
for(int i = 1; i < N; i++) {
    if(Find(father, i) != root) {
        return -1;
    }
}
```

14. 本题意在帮助大家练习拓扑排序的代码实现。没有什么难度，灵活处理输入数据即可。代码如下。

```
// v1、v2 的长度均为 M，用于表示任务的依赖关系：对于 0 <= i <= M，任务 v1[i] 必须在 v2[i] 之前完成。
vector<int> TopologicalSort(int N, int M, vector<int>& v1, vector<int>& v2) {
    vector<int> ret;
```

```
    vector<int> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    queue<int> q;
    // 根据任务的依赖关系建图
    for(int i=0; i<M; i++) {
        int u = v1[i];
        int v = v2[i];
        AdjList[u].push_back(v);
        in_deg[v] += 1;
    }
    // 入度为 0 的顶点入队
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
        }
    }
    while(!q.empty()) {
        int vertex = q.front();
        q.pop();
        ret.push_back(vertex);
        for(int i=0; i<AdjList[vertex].size(); i++) {
            int u = AdjList[vertex][i];
            in_deg[u] -= 1;
            if(in_deg[u] == 0) {
                q.push(u);
            }
        }
    }
    if(ret.size() < N) {
        return vector<int>();
    }
    return ret;
}
```

15. 需要注意的是，与求解关键路径长度不同，本题未必只有一个源点，但动态规划的思想仍然是成立的：我们依然可以根据拓扑序列的顺序，递推地从所有无前置任务的顶点推出各个任务的完成时间。具体来说，我们设置数组 max_len[]，max_len[i] 表示完成 i 的所有前置任务所需的时间。那么任务 i 的完成时间就是 max_len[i] + T[i]。

```cpp
// T[] 与 Pre[] 长度均为 N，T[i] 表示任务 i 的花费时间；Pre[i] 表示任务 i 的所有直接前置依赖
int MinTime(int N, int T[], vector<int> Pre[]) {
    struct AdjEdge {
        int u;
        int weight;
        // 如果你一路走到了这里，相信自己学会构造函数对你来说应该很简单了。
        AdjEdge(int u, int weight) : u(u), weight(weight) {}
    };
    vector<AdjEdge> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    // 遍历所有有向边，建立邻接表
    for(int i=0; i<N; i++) {
        vector<int> pres = Pre[i];
        for(int j=0; j<pres.size(); j++) {
            int u = pres[j];
            // 一条 u -> i 的有向边，其权值为 T[u]
            AdjList[u].push_back(AdjEdge(i, T[u]));
            in_deg[i] += 1;
        }
    }
    // 求解各顶点的最长路径长度（从任意源点到该顶点的路径长度中最长的那个）
    int max_len[N];
    // 数组初始化为 0
    memset(max_len, 0, sizeof(max_len));
    queue<int> q;
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
            max_len[i] = 0;
        }
    }

    while(!q.empty()) {
        int u = q.front();
        q.pop();
        // 遍历 u 的所有邻接点 v
        for(int i=0; i<AdjList[u].size(); i++) {
            int v = AdjList[u][i].u;
            int weight = AdjList[u][i].weight;
            // 尝试更新 v 的 max_len[]
            max_len[v] = max(max_len[v], max_len[u] + weight);
            in_deg[v] -= 1;
            if(in_deg[v] == 0) {
```

```
                q.push(v);
            }
        }
    }
    /* 注意：顶点的最长路径长度表示的是，完成该顶点（任务）的所有前置任务
    所需的最长时间，因此最长路径长度再加上该任务所需时间才是该任务的最早
    完成时间 */
    int ret = 0;
    for(int i=0; i<N; i++) {
        ret = max(ret, max_len[i]+T[i]);
    }
    return ret;

}
```

事实上以上代码的图模型更接近 AOV 网，用顶点表示活动，而非 AOE 网。u 的邻接表中的权值可以理解为 u 的点权（活动需要的时间）。

动态规划的过程也可以直接设置数组 done[]，直接表示各个任务完成的最早时间，将数组初始化为各个任务所需的时间 T[i]，之后进行递推，这样是更简洁的：

```
for (int i=0; i<AdjList[u].size(); i++) {
    int v = AdjList[u][i];
    done[v] = max(done[v], done[u] + T[v]);
    in_deg[v] -= 1;
    if(in_deg[v] == 0) {
        q.push(v);
    }
}
```

## 习题 1 参考答案

1. 选 ABD。

2. 选 C。栈的插入删除只能在栈顶进行，C 错误。

3. 选 C。队列的入队只能在队尾，出队只能在队首，不支持任意位序元素出队。

4. 选 D。2134 可由入入出出入出入出达成；2341 可由入入出入出入出出达成；3241 可由入入入出出入出出达成；栈满足后进先出，如论如何 4 一定比 3 先出，2 一定比 1 先出，D 无法达成。

5. 选 B。对于 A，顺序栈的 Push 和 Pop 操作均在顺序表的表尾实现，时间均可达到 O(1)；对于 C，仅有 head 指针的单链表实现“头删”，即出队，可达到 O(1)，但要实现“尾删”，即出队，则需要先找到最后一个节点，时间为 O(n)。单链表中额外记录一个尾指针 rear，则入队和出队均可达成 O(1) 时间。对于 D，循环队列的入队和出队无需移动元素。

6. 选 D。通过本题给了我们一个提示，对于循环队列来说，初始 front 和 rear 不一定非要指向 0，因为从环的几何结构来讲，“队首元素”保存在 0 位置和保存在 i 位置是没有区别的。仅有初始状态（队空）会满足 front == rear。

7. 选 A、C、D、G。对于 B，无论栈中有多少个元素，一次出栈操作只需要访问当前栈顶元素，然后修改栈顶指针即可，时间复杂度为 O(1)。
对于 C，可由两栈的栈顶下标直接分别计算得到两栈的元素数量，时间均为 O(1)，再相加即可得到总数量，总体时间为 O(1)，C 正确；
对于 D，要注意：当栈 S2 中没有元素，而栈 S1 将空间全部占满时，同样可以用 S1.top+1==S2.top 判断空间满，当 S2 将空间占满时同样如此，D 正确；
E、F 显然错误；
对于 G，当栈满时，S1.top 指向 S1 栈顶元素的下一个位置，即 S2 的栈顶元素位置；对应地，S2.top 指向 S1 的栈顶元素位置，S1.top>S2.top 成立，G 正确。

8. 选 C。队列的特性决定了第 i 个入队的元素一定是第 i 个出队的，而栈则不同，栈的出栈顺序是会受到操作过程影响的，参考本习题第 4 题。

9. 选 AC。A、B、C 项均是 5 排在第一个，那么一定是先将 1、2、3、4 进入其中一个队列，5 进入另一个队列，5 出队后，接下来只能 1、2、3、4 依次出队。对于 D 项，1 先进入一个队列，然后 2、3、4、5 使用另一个队列先进先出达成序列 2，3，4，

5，之后 1 再出队即可。

10. 选 BCD。数组实现的循环队列入队和出队时间复杂度可以达到 $O(1)$，链式队列并不会更低。事实上，若链式队列只记录头指针，并且无法快速地找到尾指针，那么入队操作要先从头遍历找到尾指针，时间复杂度更高。

11. 选 A。对于循环双链表，可以在 O(1) 时间立即拿到尾节点指针，因此在头、尾操作均为 O(1)。

12. 选 C。A 项可以通过①①②②①①③③得到；B 项可通过②①①①①①③得到；D 项输入与输出刚好相反，显然通过栈就能逆序：②②②②②②③③③③③③。对于 C，3 在 1、2 前输出，因此 1、2 必按顺序先入栈，所以 1 必晚于 2 输出，C 错误。

13. 选 C。这是一个非常有意思的题。不妨假设仅有左边能出队。无论如何 a 一定先入队，摆在最中央的位置。接下来，每一个元素入队，你都可以选择将其“摆在队中已有元素的左边或右边”。所以你需要以 a 为中心努力地去构造四个选项所给出的序列。如 A 选项要求的序列是 b,a,c,d,e，于是你将 b 摆在 a 左边，将 c 摆在 ba 的右边，将 d 摆在 bac 的右边，再将 e 摆在 bacd 的右边，便得到了 A 项所要求的序列，从左边出队，便得到了 b,a,c,d,e。B、D 项也都可以通过同样的方法得到。

14. 选 D。参见上一题解法。

15. 选 B。详细过程已经在 3.3.1 节中介绍过，不再赘述。

16. 选 B。本题比较简单，注意出栈顺序别搞错，同时注意二元运算符 op 的操作数顺序是 b op a 而不是 a op b，不要搞错就行。

17. 本题意在帮助大家学会 C++ 标准库中提供的栈—stack 的基本使用方法。注意 top() 仅返回栈顶元素而不将其弹出即可。

1） s.push(3);

2） s.size(); // 1；此时栈中元素：3

3） s.push(5);

4） s.top(); // 5；栈中元素：3，5

5） s.top(); // 5；栈中：3，5

6） s.size(); // 2；栈中：3，5

7） s.push(6);// 栈中：3，5，6

8） s.pop(); // 栈中：3，5

9） s.pop(); // 栈中：3

10） s.size(); // 1；栈中：3

11） s.top(); // 3；栈中：3

为了帮助大家学习到更实用的可编译 C++ 代码，在不影响理解核心算法思想的前提下，本书后面的算法实现中需要使用到栈和队列的地方将采用本题及下题引入的 stack<> 和 queue<> 完成。

18. 本题意在帮助大家练习 C++ 标准库中提供的栈—stack 以及队列—queue 的基本使用方法。使用它们需要 include 相应头文件。iostream 是 C++ 的标准输入输出流，可以简单理解为 stdio.h。cout 是 iostream 中提供的向标准输出打印的函数，可以简单理解为 printf。

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;

int arr[5] = {1,2,3,4,5};

int main() {
    stack<int> s;
    queue<int> q;
    for(int i=0; i<5; i++) {
        s.push(arr[i]);
    }
    while(!s.empty()) {
        cout << s.top();    // 打印栈顶元素
        q.push(s.top());    // 栈顶元素入队
        s.pop();    // 弹出栈顶元素
    }
    // 打印一个换行符，相当于'\n'
    cout << endl;
    while(!q.empty()) {
        cout << q.front();  // 打印队首元素
        q.pop();    // 队首元素出队
    }

```

```
26      return 0;
27  }
```

关于 iostream, using namespace std; 这些，大概是入门 C++ 的“Hello, World”内容。请大家参考 **??**(第 **??** 页) 中的相关内容。

19. 这个问题如果你在思考的过程中自然地想到了使用某种“一端进，两端出”的数据结构，那么你可以开一瓶香槟庆祝一下。如果你只想到了时间复杂度为 $O(n \times k)$ 的算法，也可以奖励自己一杯奶茶。

本题最容易想到的方法是：从前向后遍历一遍 delay[] 数组，检查每一个延迟数值 delay[i] 时，从 i 向前找，直到遇见下标 0 或 i-k+1 的元素，在这个过程中记录下一个最大值，作为 i 时刻的延迟最大值，即 ans[i]。该方法的时间复杂度为 $O(n \times k)$。当 k 较大时，复杂度可能接近 $O(n^2)$。

但是，以上方法与“天气预报问题”（见—waypoint-3—（第 38 页）处）的朴素解法有着类似的问题：当我们检查后面的元素时，前面的元素其实已经被检查过若干遍了。我们能不能在每个元素第一次被检查的时候，就想办法将它保存到某个结构 some_structure 中，直至该元素“不可能成为后面任何一个 i 的解”，再将它丢弃呢？

我们从前向后遍历，当检查下标为 i 的元素时：
1）显然，下标 <=i-k 的元素已经不再可能成为后面任何一个元素（包括 i）的解。所以需要将所有下标 <=i-k 的元素移出 some_structure。这正是“先进先出”。
2）当我们检查 i 时，由于 i 可以作为自己的解，所以 some_structure 中所有值小于 i 的元素均已不再可能成为后面任何一个元素的解，移出它们。经过分析不难发现，该特性决定了 some_structure 中元素单调不增。所以移出时应该将 delay[i] 与“后进来的”元素先比较。

所以这个结构应该是一个“尾端能进能出、首端能出”的一个结构。**并且当检查元素 i 时，移出满足上述 1）、2）条件的元素后，队首元素就是该元素 i 的解。**

如正文中介绍过，通常称这种结构为**双端队列**。此处我们是在解决问题（而不是封装某种数据结构给别人或自己复用），因此我们不兴师动众地封装一个“双端队列”的数据结构，并为他写出“首端出队”、“尾端入队、尾端出队”等操作，而是直接将操作展开，作为逻辑的一部分实现。完整代码如下所示。

```
1  void MaxDelay(int delay[], int n, int k, int ans[]) {
2      // deque[] 保存候选元素的下标。
3      // deque[front] 是队首，deque[rear - 1] 是队尾。
```

```
    int deque[n];
    int front = 0;
    int rear = 0;

    for (int i = 0; i < n; i++) {
        // 从队首移出队列中所有下标小于等于 i-k 的元素【上面的 1)】
        while (front < rear && deque[front] <= i - k) {
            front++;     // 注意入队元素数量一定不会超过 n，因此不用考虑循环
        }

        // 从队尾移除所有不可能再成为最大值的元素。【上面的 2)】
        while (front < rear &&
            delay[deque[rear - 1]] <= delay[i]) {
            rear--;
        }

        // 当前元素加入队尾。
        deque[rear] = i;
        rear++;

        // 队首就是当前窗口中的最大值。
        ans[i] = delay[deque[front]];
    }
}
```

### 习题 2 参考答案

6. 显然，本题我们只要关注堆顶元素（即最小的那一个）是什么即可：

1） min_heap.push(5); // 堆中元素：5；堆顶为 5

2） min_heap.top(); // 获取到的为 5

3） min_heap.push(3); // 堆中元素：5、3；堆顶为 3

4） min_heap.push(4); // 堆中元素：5、3、4，堆顶为 3

5） min_heap.top(); // 获取到的为 3

6） min_heap.pop(); // 删除堆顶元素 3，4 成为新的堆顶

7） min_heap.top(); // 获取到的为 4

7. 每次在堆中选出两个最小的来合并，计算相应体力消耗，并将合并后的新柴薪加入堆中。做 n-1 次即可。

```
int MinCost(int w[], int n) {
    priority_queue<int, vector<int>, greater<int>> min_heap;
    // 在下方完成你的代码
    int total_cost = 0;

    for(int i=0; i<n; i++) {
        min_heap.push(w[i]);
    }
    // 做 n-1 次
    for(int i=1; i<=n-1; i++) {
        // 取出两个重量最小的：w1 和 w2
        int w1 = min_heap.top();
        min_heap.pop();
        int w2 = min_heap.top();
        min_heap.pop();

        // 体力消耗为 w1+w2
        int union_cost = w1 + w2;
        total_cost += union_cost;

        // 合成一堆后放回堆中
        min_heap.push(union_cost);
    }
    return total_cost;
}
```

13. 本题递归访问二叉树中的每一个节点即可。我们记录一个引用变量（或指针）max_appeal，当访问到叶节点时，若该节点的“诱人值”大于已知的最大诱人值 max_appeal，则更新。遍历一遍下来 max_appeal 即是最大诱人值。计算诱人值，需要在递归过程中记录递归深度。代码如下：

```
// 访问节点 root，其深度为 depth
void visit(node* root, int depth, double& max_appeal) {
    if(root == NULL) return ;

    // 叶子节点，有宝藏的房间
    if(root->lchild == NULL && root->rchild == NULL) {
        double appeal = (1.0/dpeth) * root->weight;
        // 本房间的“诱人值”比之前遇到的所有房间都大，更新 max_appeal
        if(appeal > max_appeal) {
            max_appeal = appeal;
        }
    }
    if(root->lchild != NULL) {
        visit(root->lchild, depth+1, max_appeal);
    }
    if(root->rchild != NULL) {
        visit(root->rchild, depth+1, max_appeal);
    }

}

double CalcAppeal(node* T) {
    double ans = 0;
    // your codes...
    visit(T, 1, ans);
    return ans;
}
```

14. 本题的核心思想在于，对于以节点 u 为起点的逃生路线有：

$$MinCost(u) = weight(u) + \min(MinCost(left), MinCost(right))$$

left、right 分别表示 u 的左右子树。想通了这一点，题目就变得非常简单。请参见代码与注释。

```
int MinEscapeCost(node* root) {
    // 叶子节点，出口找到了
    if(root->lchild == NULL && root->rchild == NULL) {
        return root->weight;
```

```
    }
    // 左边是死路，只能从右边走
    if(root->lchild == NULL) {
        return root->weight + MinEscapeCost(root->rchild);
    }
    // 右边是死路
    if(root->rchild == NULL) {
        return root->weight + MinEscapeCost(root->lchild);
    }

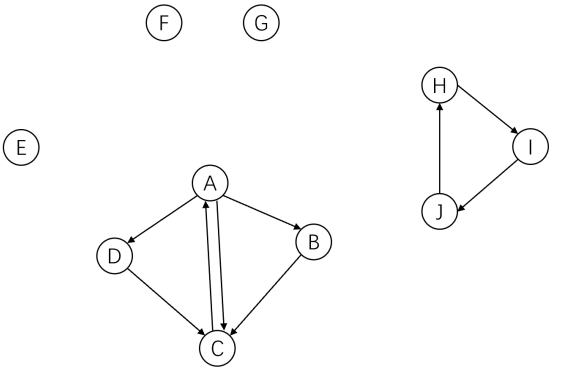
    // 两边都能走，递归计算两边代价后，选代价小的那一条路
    int ltree_cost = MinEscapeCost(root->lchild);
    int rtree_cost = MinEscapeCost(root->rchild);

    return root->weight + min(ltree_cost, rtree_cost);
}
```

## 习题 3 参考答案

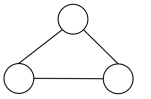
1. 选 B。顶点 A 与顶点 E 存在相互连通的路径: A -> D -> E 以及 E -> D -> A，选项 B 错误。

2. 选 D。A 项，存在两顶点 A、E，A 到 E 无路径，所以 $G_1$ 不是强连通的；对于 $G_2$，F 到 G 无路径，因此 $G_2$ 不是强连通的；$G_3$ 任意两顶点之间都有路径，是强连通的。B 项，图 $G$ 存在以下强连通分量：




显然数量大于 3。C 项，顶点 A 出度为 3，入度为 1，C 正确。D 项，顶点 F 出度为 0，入度为 1，D 错误。

3. 选 A。下图所示的图是一个无向连通图，但不具有 II 与 III 的特性。由于无向图中一条边必然带来 2 个度，因此所有顶点度数之和一定是 2 的倍数，I 正确。



4. 要使得边数达到最大，一定是任意两个顶点之间都有一条边。因此最大边数是从 $N$ 个顶点选出 2 个的组合数：

$$C_N^2 = \frac{N(N-1)}{2}$$

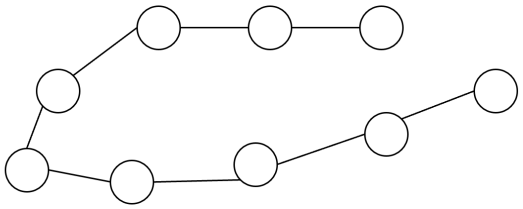
另一种理解方式：第 1 个顶点可以与另外 $N-1$ 个顶点各形成一条边，共 $N-1$ 条；第 2 个顶点可以和剩下的 $N-2$ 个顶点各形成一条边，共 $N-2$ 条..... 第 $N-1$ 个顶点可以和剩下的 1 个顶点组成 1 条边，此时，任意两个顶点都有边存在了，达到最大。因此最大边数为

$$(N-1)+(N-2)+...+2+1 = \frac{(N-1)(1+N-1)}{2} = \frac{N(N-1)}{2}$$

事实上，对于不存在重复边、不存在自环（即指向自身的边）的图，通常称其为**简单图**。若无特殊说明，本书中提到的所有图均为简单图。

对于有 $N$ 个顶点的无向图，其边数最大为 $\frac{N(N-1)}{2}$，有 $\frac{N(N-1)}{2}$ 条边的无向图就被称为**完全图**，也被称为**简单完全图**，完全图中的任意两个顶点之间均存在边。而对于有向图，显然边数最大为 $2 \cdot \frac{N(N-1)}{2} = N(N-1)$，因此，将有 $N(N-1)$ 条边的有向图称为**有向完全图**，有向完全图中的任意两个顶点之间都存在方向相反的两条边。

5. 要保证连通性，边数最少的情况就是“用一根线将所有顶点串起来”，如图所示：



在此情况下，再少一条边，则图将不连通。显然此情况边数等于顶点数减一。即：**有 $N$ 个顶点的无向图，要保证它是连通图，至少要有 $N-1$ 条边。换句话说，有 $N$ 个顶点的无向图，若边数小于 $N-1$，则它一定是不连通的。**

也可以这样理解：对于 $N$ 个顶点、无边的无向图，显然有 $N$ 个连通块。之后每添加 1 条边，一定能让两个本不连通的块连通起来。因此，添加 $N-1$ 条边后，就只剩一个连通块了。

6. 选 C：16。要使得图依然不连通，能添加的最大边数就是将另外 6 个顶点添加到任意两个顶点之间都有边，即另外 6 个顶点构成的完全图，也就是添加 $\frac{6\cdot(6-1)}{2} = 15$ 条边。因此再添加一条边，共 16 条边，就一定会使得第 7 个顶点与这 6 个顶点连通。

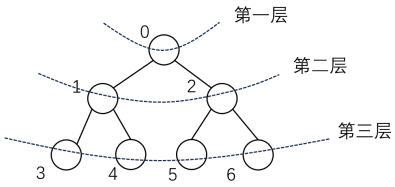
7. 选 A。图的邻接矩阵 AdjM[i][j] 存储的是编号为 $i$ 的顶点到编号为 $j$ 的顶点的边的信息。0 表示无边，那么编号为 $i$ 的顶点的度就等于邻接矩阵第 $i$ 行（或第 $i$ 列）中非零元素的数量。

8. 选 B。如果是有向图，那么 AdjM[i][*]，即第 $i$ 行的非 0 元素数量，是顶点 $i$ 的出度，而 AdjM[*][i]，即第 $i$ 列的非 0 元素数量，是顶点 $i$ 的入度，二者加起来才是顶点 $i$ 的度。

9. 选 C。邻接表存储方式，为每个顶点维护一个线性表，存放其所有**出边**的信息，顶点 A 有 3 条出边，顶点 C 有 1 条出边，选 C。

10. 选 D。D 项序列 {E,C,D,A,B}，探索完前两个顶点 E、C，与 C 邻接的顶点尚有 A 未被访问，仍可以向更深处探索，而序列中下一个探索了 D，说明不是深度优先的策略。

11. 选 C。本题意在更直观地揭示树的层序遍历与图的广度优先遍历之间的关联。它们都是像涟漪一样从中心向外“扩散”：




12. 选 A。广度优先是越早看见的顶点越先被访问，是“先进先出”的，因此通常用队列实现，而不是栈；深度优先是“后进先出”的，通常用栈实现，A 错误。

13. 1）邻接矩阵版：

```
bool AdjM[N][N];    // 假设该数组已经正确存储了 G 的邻接矩阵
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
    int ret = 0;
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++) {
            if(AdjM[i][j] == 1) {
                ret += 1;
            }
        }
    }
    return ret;
}
```

2）邻接表版：

```
vector<int> AdjList[N]; // 该数组已经正确存储了 G 的邻接表
// 返回所有顶点的度数之和
int DegreeSum(int N) {
    // 在此处完成你的代码
    int ret = 0;
    for(int i=0; i<N; i++) {
        ret += AdjList[i].size();
    }
    return ret;
}
```

事实上，对于无向图，一条边必定连接两个顶点，为这两个顶点各贡献一个度。因此，无向图中所有顶点的度数之和一定是图的边数的 2 倍。这个事实被称为**“牵手定理”**。

类似地，对于有向图，一条有向边始于一个顶点（贡献 1 个出度）、终于一个顶点（贡献 1 个入度），因此有向图中所有顶点的出度之和（或入度之和）等于图的边数 $M$，而有向图的总度数之和（入度 + 出度）同样是边数的 2 倍（$2M$）。

14. 显然，本题从起点出发，DFS 遍历一遍，每访问到一个新的顶点，就将计数器加一即可。“访问”一个节点的过程就是将全局计数器加一的过程，而不需要对顶点自身做任何其他操作。代码如下：

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

void DFS(int v, int& count, vector<int> AdjList[N]) {
    count += 1;
    visited[v] = true;
    for(int i=0; i<AdjList[v].size(); i++) {
        int adj_vertex = AdjList[v][i];
        if(!visited[adj_vertex]) {
            DFS(adj_vertex, count, AdjList);
        }
    }
}

int CountAccessibleTrees(vector<int> AdjList[N]) {
    int count = 0;
    // 从起点 0 出发，全局计数器 count 以引用传递，每访问一个顶点时计数。
    DFS(0, count, AdjList);
    return count;
}
```

15. 这是非常经典的求图的连通块的问题。我们检查一遍图的所有顶点（代码中的循环①），若其没被访问过，说明这是一个新的需要计数的连通块，于是计数加一，并以该顶点为起点执行 BFS 或 DFS，将该连通块的所有顶点标记为已访问。反之若顶点已被访问，说明它属于之前遍历过的某个连通块，已经被计数过了，跳过即可。

```
const int N/* = 100*/;
bool visited[N];    // 已初始化为 false

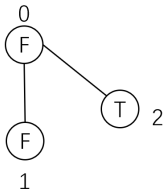
// 以 v 为起点，遍历 v 所在的连通块
void BFS(int v, vector<int> AdjList[N]) {
    queue<int> q;
    q.push(v);
    visited[v] = true;
```

```
    while(!q.empty()) {
        int cur_vertex = q.front();
        q.pop();
        for(int i=0; i<AdjList[v].size(); i++) {
            int adj_vertex = AdjList[cur_vertex][i];
            if(!visited[adj_vertex]) {
                q.push(adj_vertex);
                visited[adj_vertex] = true;
            }
        }
    }
}

int CountConnectedComponents(vector<int> AdjList[N]) {
    int count = 0;
    for(int i=0; i<N; i++) {    // 循环①
        if(!visited[i]) {   // 该顶点所在的连通块没被计数过
            count += 1;
            BFS(i, AdjList);
        }
    }
    return count;
}
```

*16. 本题思路比较简单，通过 BFS 或 DFS 从起点开始搜索，当搜索到'T' 时即找到了一条路径。问题在于搜索过程中如何将涉足过的轨迹记录下来？

①对于递归的 DFS 代码，算法一定是一步一个脚印地，沿着合法路径在前进或回溯。当走进一条错误的路径之后，如何正确地进行**回溯**？如图所示，




若从 0 出发进入了 1，此时记录的路径是“01”，回到 0 后，要将路径回溯到“0”。显然，由于回退时总是将“后路过的顶点先从路径中拿掉”，因此可以使用一个栈保存所经过的路径。回溯时，将顶点从栈顶弹出即可。

②而对于非递归的 BFS 或 DFS 代码，不同的是，非递归算法的下一个“出发点”是由队列（或栈）中元素决定的，顶点的访问顺序未必是一条合法路径。所以对于非递归代码，要记录路径，可以设一个 Pre[] 数组，如 Pre[3] 值为 0 就表示顶点 3 的上一步是顶点 0。

我们给出非递归的 BFS 与递归的 DFS 代码如下：

**代码 6.1　非递归的 BFS 解法**

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
int Pre[N]; // 记录路径
bool visited[N];

// 以 v 为起点，BFS 搜索一条找到'T'的路径。若找到了，返回'T'所在顶点的编号。
↪ 若'T'不可达，返回-1。
int BFS(int v) {
    queue<int> q;
    int ret = -1;
    if(trees[v] == 'T') {
        return v;
    }
    q.push(v);
    visited[v] = true;
    while(!q.empty()) {
        int step = q.front();
        q.pop();
        for(int i=0; i<N; i++) {
            if(Matrix[step][i] != 0 && !visited[i]) {
                // 记录轨迹的上一个顶点编号
                Pre[i] = step;
                if(trees[i] == 'T') {
                    ret = i;
                    return ret;
                }
                q.push(i);
                visited[i] = true;
            }
        }
    }
    return ret;
}

void FindPath(int u) {
    for(int i=0; i<N; i++) {
        Pre[i] = -1;
```

```
        visited[i] = false;
    }
    int flower_idx = BFS(u);
    // 没有路径
    if(flower_idx == -1) {
        cout << "......" << endl;
        return;
    }
    /* 从 flower_idx 开始，通过 Pre[] 数组向前找到的路径顶点顺序是反的，用一个栈
    ↪ 正向打印 */
    stack<int> to_print;
    for(int i=flower_idx; i!=-1; i=Pre[i]) {
        to_print.push(i);
    }
    // 打印出路径
    while(!to_print.empty()) {
        cout << to_print.top();
        to_print.pop();
    }
    cout << endl;
}
```

以上 BFS 算法要访问约 $N$ 个顶点，访问每个顶点时要遍历该顶点的邻接矩阵以找出邻接点，时间为 $O(N)$，因此总的时间复杂度为 $O(N^2)$。

递归的 DFS 代码：

**代码 6.2　递归的 DFS 解法**

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵
    { 0, 1, 1, 1, 0, 0, 0 },
    { 1, 0, 0, 1, 1, 0, 0 },
    { 1, 0, 0, 1, 0, 0, 0 },
    { 1, 1, 1, 0, 0, 1, 0 },
    { 0, 1, 0, 0, 0, 1, 0 },
    { 0, 0, 0, 1, 1, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
// 存在解返回 true，否则返回 false
bool DFS(int v, stack<int>& track, bool visited[]) {
    visited[v] = true;
    track.push(v);  /* 先假设经由该顶点 v 能到达樱树，在轨迹中记录此顶点，若后面
    ↪ 发现无法到达，回溯。 */
```

```
    bool solved = true;    // 若后面发现无法到达，回溯时别忘了将 solved 置为 false
    if(trees[v] == 'T') {
        return solved;
    }

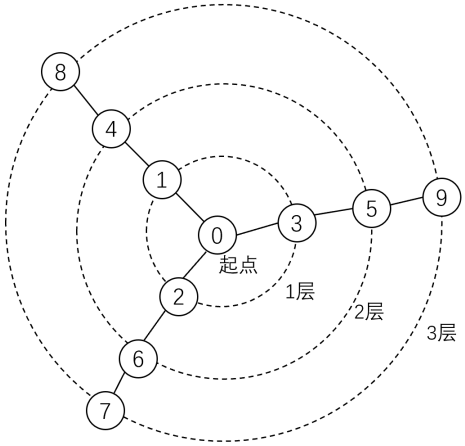
    // 从此顶点 v 开始，向邻接点递归地探索
    for(int i=0; i<N; i++) {
        if(!visited[i] && Matrix[v][i] == 1) {
            // 以此顶点 v 为（半路）起点，可以找到樱树，返回 true
            if(DFS(i, track, visited) == true) {
                solved = true;
                return solved;
            }
        }
    }
    /* 若代码执行至此，说明事实上由此顶点 v 作为（半路）起点的所有路径均无法找到樱树，于是从轨迹中弹出此顶点，回溯到上一个顶点，继续探索其他路径。 */
    track.pop();
    solved = false;
    return solved;
}

void FindPath(int u) {
    // 动态记录轨迹
    stack<int> track;
    bool visited[N];
    for(int i=0; i<N; i++) {
        visited[i] = false;
    }
    // 存在解
    if(DFS(u, track, visited) == true) {
        // 栈中保存的轨迹是相反的，保存到一个 vector 中之后从后向前打印
        vector<int> to_print;
        while(!track.empty()) {
            to_print.push(track.top());
            track.pop();
        }
        // 打印出路径
        for(int i=to_print.size()-1; i>=0; i--) {
            cout << to_print[i];
        }
        cout << endl;
    } else {    // 不存在解
        cout << "......" << endl;
    }
}
```

该算法的性能瓶颈在于 DFS，最坏情况下，假设每个顶点都有接近 N 个邻接点，则 DFS 被调用接近 N 次，并且每次调用执行接近 N 次循环，因此算法最坏时间复杂度为 $O(N^2)$。

*17. 事实上，上题的非递归 BFS 解法，求出来的那条路径一定是最短路径。为什么？

如图所示，由于 BFS 总是先搜索完当前层的所有顶点，才会进入下一层，因此它一定会按照从近到远的顺序发现顶点。所以当 BFS 第一次发现某个顶点时，就已经找到了从起点到达该顶点、经过边数最少的路径。




**当图中所有边的权重都相同时，路径的总权重只取决于它经过了多少条边。而 BFS 恰好能够保证：第一次到达某个顶点时，经过的边数最少。因此对于无权图或等权图，BFS 可以用于求解最短路径问题。**

而对于带权图，经过的边数最少的路径不一定是总长度最短的路径。因此 BFS 算法无法求解带权图的最短路径。而 DFS 可以将所有的路径都探索一遍，通过穷举路径的方式计算出路径最短的是哪一条。因此通过 DFS 方式穷举路径的方法，是一种求带权图最短路径的算法，我们将在下一题中体会。

*18. 如上题所述，对于带权图无法使用 BFS 求解最短路径，但是可以 DFS 递归地将所有路径穷举一遍。在搜索到'T'后回溯，继续寻找其他路径。一种简单的想法是，在 DFS 过程中，将当前已经找到的每一条路径轨迹及其长度都记录下来，最后再选择一条长度最短的输出其轨迹。我们做如下定义：

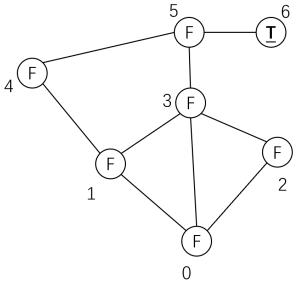
```
// 定义一条路径
struct track {
    stack<int> nodes;   // 轨迹序列，如 0356
    int length;         // 该路径的长度
```

```
};
track t;    // 当前走过的路径
vector<track> tracks;   // 用于保存所有路径
```

于是我们可以在 DFS 递归过程中动态维护当前走过的路径 t（包括轨迹顶点集 nodes 以及轨迹的长度 length），当找到'T'时，就将当前记录的路径 t 放入 tracks 尾部：

```
tracks.push_back(track);
```

需要注意的是，在寻找路径的过程中，每个顶点有可能被多次访问，因此回溯时不仅要维护 t.nodes 与 t.length，还要将当前节点重新标记为未访问。比如在下图中，假设已经找到了一条路径 01356，回溯时若不把这些顶点的 visited[] 标记为 false，会导致后面重新从 0 出发时，无法达到 1 和 3，从而错过最短路径：“0356”。




完整的代码如下所示。

```
const int N/* = 7 */;
char trees[N] /*= {'F', 'F', 'F', 'F', 'F', 'F', 'T'}*/;    // 保存顶点数据
int Matrix[N][N] /*= {    // 邻接矩阵，0 表示无路，非 0 表示通行时间
    { 0, 3, 2, 2, 0, 0, 0 },
    { 3, 0, 0, 2, 5, 0, 0 },
    { 2, 0, 0, 1, 0, 0, 0 },
    { 2, 2, 5, 0, 0, 6, 0 },
    { 0, 1, 0, 0, 0, 2, 0 },
    { 0, 0, 0, 6, 2, 0, 1 },
    { 0, 0, 0, 0, 0, 1, 0 },
}*/;
struct track {
    stack<int> nodes;
    int length;
};

// pre_v 用于访问 Matrix[pre_v][v] 以累加路径长度
```

```
// t 是当前走过的路径
// tracks 是当前已知的所有能够到达'T'的路径
void DFS(int v, int pre_v, track& t, bool visited[], vector<track>& tracks) {
    // 记录当前路径
    visited[v] = true;
    t.nodes.push(v);
    // 这个 if 只是为了处理初始传入时，我们传的是 pre_v 与 v 相等。
    if(pre_v != v) {
        t.length += Matrix[pre_v][v];
    }


    if(trees[v] == 'T') {
        // 找到终点了，说明当前路径是一条起 -> 终的完整路径，记录到 tracks 中
        tracks.push_back(t);

        // 回溯，继续寻找其他路径
        visited[v] = false;
        t.nodes.pop();
        t.length -= Matrix[pre_v][v];
        return ;
    }

    // DFS 探索路径
    for(int i=0; i<N; i++) {
        if(Matrix[v][i] != 0 && !visited[i]) {
            DFS(i, v, t, visited, tracks);
        }
    }

    // 以当前顶点出发的所有路径已探索完，回溯，继续探索其他路径
    visited[v] = false;
    t.nodes.pop();
    t.length -= Matrix[pre_v][v];
}

void FindPath(int u) {
    vector<track> tracks;
    track t; t.length = 0;
    bool visited[N];
    for(int i=0; i<N; i++) {
        visited[i] = false;
    }

    // 传入 u 时，u 无前驱，认为 u 的前驱是 u 即可
    DFS(u, u, t, visited, tracks);
    // tracks 为空说明从未找到过任何一条路径，不可达。
```

```
65      if(tracks.empty()) {
66          cout << "......" << endl;
67      } else {
68          // 判断一下哪条路径长度最短
69          int min_length = 0x3fffffff;
70          int ans_idx = -1;   // 最短路径的下标
71          vector<int> min_path;
72          for(int i=0; i<tracks.size(); i++) {
73              if(tracks[i].length < min_length) {
74                  min_length = tracks[i].length;
75                  ans_idx = i;
76              }
77          }
78          vector<int> to_print;
79          // 将最短的那条路径倒过来，放到 to_print 中
80          while(!tracks[ans_idx].nodes.empty()) {
81              to_print.push_back(tracks[ans_idx].nodes.top());
82              tracks[ans_idx].pop();
83          }
84          for(int i=to_print.size()-1; i>=0; i--) {
85              cout << to_print[i] ;
86          }
87          cout << endl;
88      }
89  }
```

最坏情况下，假设 N 个顶点任意两顶点之间都存在边，边的数量达到了 $\frac{N(N-1)}{2}$（见本习题第 4 题），从起点出发，下一步有 $(N-1)$ 条路，这 $(N-1)$ 个新的起点，每个起点又有 $(N-2)$ 条路可以选择，……，于是需要辗转的顶点数量为

$$(N-1)\cdot(N-2)\ldots\cdot 2\cdot 1$$

时间复杂度为 $O(N!)$ 量级。对于稍微复杂一点的图，这是根本不可能接受的！

事实上，在 DFS 过程中，可以再维护一个变量，记录当前已经找到的所有可行路径的最短长度，在递归到某顶点时，若此时当前路径长度已经大于最短长度，则无需继续递归下去，而是直接回溯即可。这种优化策略通常被称为**剪枝**，意为在递归树中剪掉一些无意义的路线。

此外，完全可以在 DFS 搜索过程中只维护和更新“当前已找到的所有路径中最短的那一条路径”，而不必将所有路径都存储起来，这对于存储空间是一个极大的优化。感兴趣的同学可以自己尝试写出代码。

但是，以上两种优化策略也仅仅是杯水车薪，带来的优化比较有限。在 5.4 节中，我们将介绍一种极具创造性的算法—迪杰斯特拉算法，来解决**单源最短路径问题**。

19. 计算所有顶点的度即可。代码如下：

```
int IsExistEL(MGraph G) {
    int odd_deg_num = 0;    // 计算度为奇数的顶点个数
    for(int i=0; i<G.numVertices; i++) {
        int i_degree = 0;   // 计算顶点 i 的度
        for(int j=0; j<G.numVertices; j++) {
            if(G.Matrix[i][j] == 1) {
                i_degree += 1;
            }
        }
        if(i_degree%2 == 1) {   // 顶点 i 是度为奇数的顶点
            odd_deg_num += 1;
        }
    }
    return (odd_deg_num == 0) || (odd_deg_num == 2);
}
```

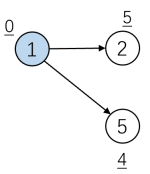
显然时间复杂度为 $O(n^2)$，空间复杂度为 $O(1)$。

20. 遍历一遍邻接矩阵，计算各顶点的入度和出度即可。

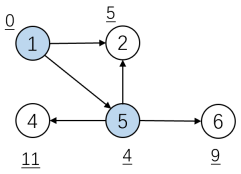
```
int PrintKVertices(MGraph G) {
    int ret = 0;    // 计算 K 顶点的个数
    for(int i=0; i<G.numVertices; i++) {
        int out_deg = 0;    // 顶点 i 的出度
        int in_deg = 0;     // 入度
        for(int j=0; j<G.numVertices; j++) {
            // i 到 j 有路径，i 的出度 +1
            if(G.Matrix[i][j] != 0) out_deg += 1;
            // j 到 i 有路径，i 的入度 +1
            if(G.Matrix[j][i] != 0) in_deg += 1;
        }
        // 出度大于入度，是一个 K 顶点，输出
        if(out_deg > in_deg) {
            cout << G.VerticesList[i];
            ret += 1;
        }
    }
    return ret;
}
```

## 习题 4 参考答案

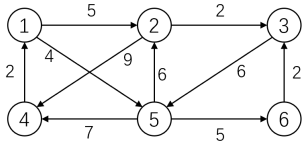
1. 选 B。每一步所选的点，及选完该点后所有点的当前已知最短路径 cur_mind[] 如下图所示（带下划线的数字表示该顶点的 cur_mind[]）。



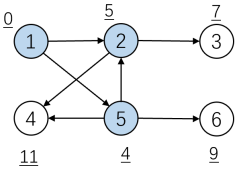

第一步，选1



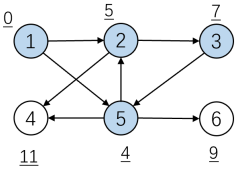

第二步，选5



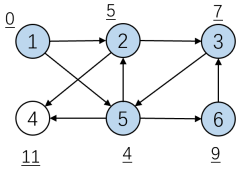

原图



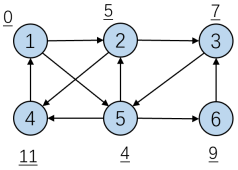

第三步，选2




第四步，选3




第五步，选6

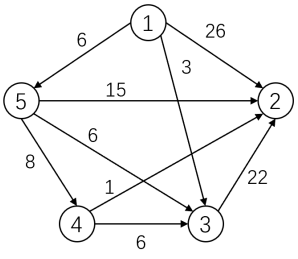


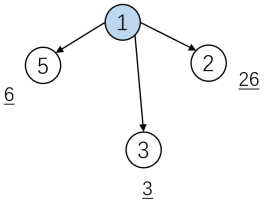

第六步，选4

于是选点顺序是 1,5,2,3,6,4，选 B。

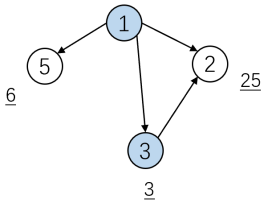
2. 选 C。过程与上题类似。需要注意 c 到 f 是有一条边的，别看漏了就行。

3. 选 C。求解前两个顶点（不包括源点）的最短路径过程如下所示。

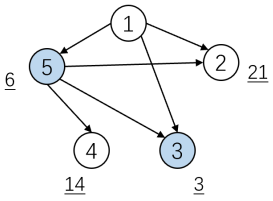



第一步




第二步




第三步

4. a）若存在从源点可达的负环，则源点 s 与任何一个顶点 v 的最短路径，都可以通过“先走到负环，然后一直绕圈，再走到 v”的方式，将最短路径“刷到负无穷”，因此若图中存在源点可达的负环，通常认为最短路径问题不可解，或者为负无穷；而

Bellman-Ford 算法可以在图存在负环时给出“不可解”的结论，Dijkstra 算法则难以做到这一点。

b）若存在源点可达的负权边，那么迪杰斯特拉算法将失效，求解出可能错误的最短路径；而 Bellman-Ford 及 SPFA 算法可以应对存在负权边的图，给出正确的最短路径。

三种算法总结如下：

| 图的特点 | Dijkstra | Bellman-Ford | SPFA |
|---|---|---|---|
| 所有边权非负 | 可以 | 可以 | 可以 |
| 存在负权边，无可达负环 | 不保证正确 | 可以 | 可以 |
| 存在可达负权环 | 不保证正确，无法可靠检测 | 可以检测 | 可以检测 |

5. 本题意在给大家练习求解最短路径的迪杰斯特拉算法、SPFA 算法的代码。显然，我们使用算法求出各顶点的最短路径，返回维火女所在顶点的最短路径即可。使用迪杰斯特拉算法求解的代码如下：

```
// N 已定义
struct AdjEdge {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<AdjEdge> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {
    const int inf = 0x3fffffff;
    int cur_mind[N];
    bool solved[N];
    // 记录维火女所在的牢房编号 firekeeper_idx，函数最后返回
    ↪  cur_mind[firekeeper_idx]。
    int firekeeper_idx = -1;

    // 初始化的过程顺便找一下维火女在哪
    for(int i=0; i<N; i++) {
        cur_mind[i] = inf;
        solved[i] = false;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }
    cur_mind[s] = 0;

```

```
    for(int i=0; i<N-1; i++) {
        // 从尚未求解的顶点中选出一个 cur_mind[] 最小的顶点 v
        int min_curmind = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(cur_mind[j] < min_curmind && !solved[j]) {
                v = j;
                min_curmind = cur_mind[j];
            }
        }

        if(v == -1) {
            break;
        }
        solved[v] = true;
        // 已经计算出维火女了，就别再计算其他的了，赶紧去救她吧！
        if(v == firekeeper_idx) {
            return cur_mind[v];
        }

        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int time_cost = AdjList[v][j].time_cost;
            if(cur_mind[v] + time_cost < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + time_cost;
            }
        }

    }
    return cur_mind[firekeeper_idx] == inf? -1 : cur_mind[firekeeper_idx];
}
```

使用 SPFA 算法求解的代码如下：

```
// N 已定义
struct AdjEdge {
    int u;          // 顶点编号
    int time_cost;  // 花费时间（边权）
};
vector<AdjEdge> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间需要花费的最短时间
int MinTime(int s) {
    const int inf = 0x3fffffff;
    int cur_mind[N];
    bool in_queue[N];
```

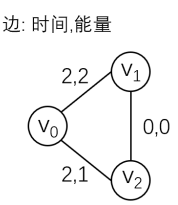
```
    /* 该场景明确图中无负边，因此无需进行负环
    检测，也就不需要 inq_cnt[] 数组，但这里还是
    以注释给出，帮助大家再次复习若需要检测负环
    时，代码该怎么写。 */
    // int inq_cnt[N];
    int firekeeper_idx = -1;
    for(int i=0; i<N; i++) {
        cur_mind[i] = inf;
        in_queue[i] = false;
        // inq_cnt[i] = 0;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }
    cur_mind[s] = 0;
    queue<int> q;
    q.push(s);
    in_queue[s] = true;
    while(!q.empty()) {
        int v = q.front();
        q.pop();
        in_queue[v] = false;
        for(int i=0; i<AdjList[v].size(); i++) {
            int u = AdjList[v][i].u;
            int time_cost = AdjList[v][i].time_cost;
            if(cur_mind[v] + time_cost < cur_mind[u]) {
                cur_mind[u] = cur_mind[v] + time_cost;
                if(!in_queue[u]) {
                    in_queue[u] = true;
                    q.push(u);
                    // inq_cnt[u] += 1;
                }

                // 若可能存在负环，在此处进行负环判断

            }
        }
    }
    return cur_mind[firekeeper_idx] == inf ? -1 : cur_mind[firekeeper_idx];
}
```

6.

若使用迪杰斯特拉算法，就要同时记录每个点的当前已知最短时间及对应最短时间路径的最小能量消耗：cur_mint[i] 表示顶点 i 到 s 的最短时间，cur_mine[i] 表示顶点 i 到 s 的最短时间路



径下的最小消耗。

这样，每次“选点”时，就不能只选择时间花费最小的，而是要选择时间花费最小的条件下，能量消耗最小的那个点。如右图，以 $v_0$ 为起点，若直接选择了 $v_1$，则会得到错误答案。在更新时类似，当时间更短，或时间相等但能量消耗更少时更新即可。代码如下：

```
// N 已定义
struct AdjEdge {
    int u;
    int time_cost;      // 花费时间
    int energy_cost;    // 和敌人战斗需要消耗的能量值
};
vector<AdjEdge> AdjList[N];    // 已存储邻接表信息
char ver_info[N];   // 已存储顶点信息
// 返回以 s 为起点到达维火女的房间花费时间最短的路径中，消耗能量最少的那条路径所消耗的能量值
int MinEnergyCost(int s) {
    int cur_mint[N];    // 当前已知的各顶点最短时间
    int cur_mine[N];    // 对应最短时间路径中的最小能量消耗
    bool solved[N];
    const int inf = 0x3fffffff;
    int firekeeper_idx = -1;
    for(int i=0; i<N; i++) {
        cur_mint[i] = inf;
        cur_mine[i] = inf;
        solved[i] = false;
        if(ver_info[i] == 'T') {
            firekeeper_idx = i;
        }
    }

    cur_mint[s] = 0;
    cur_mine[s] = 0;

    for(int i=0; i<N-1; i++) {
        int min_timecost = inf;
        int min_energycost = inf;
        int v = -1;
        for(int j=0; j<N; j++) {
            if(!solved[j] && (cur_mint[j] < min_timecost ||
                cur_mint[j] == min_timecost && cur_mine[j] < min_energycost)
            ) {
                min_timecost = cur_mint[j];
                min_energycost = cur_mine[j];
                v = j;
            }
```

```
        }

        if(v == -1) {
            break;
        }
        solved[v] = true;
        if(v == firekeeper_idx) {
            break;
        }
        for(int j=0; j<AdjList[v].size(); j++) {
            int u = AdjList[v][j].u;
            int time_cost = AdjList[v][j].time_cost;
            int energy_cost = AdjList[v][j].energy_cost;
            if(cur_mint[v] + time_cost < cur_mint[u]) {
                cur_mint[u] = cur_mint[v] + time_cost;
                cur_mine[u] = cur_mine[v] + energy_cost;
            }
            else if(cur_mint[v] + time_cost == cur_mint[u] && cur_mine[v] +
            ↪  energy_cost < cur_mine[u]) {
                cur_mine[u] = cur_mine[v] + energy_cost;
            }
        }
    }
    return cur_mine[firekeeper_idx] == inf ? -1 : cur_mine[firekeeper_idx];
}
```

事实上本题可以从另一个角度去理解：有些时候我们可能在求时间最短的最短路径，有些时候求路程最短，有些时候求花费最少……，时间、路程、花费的大小比较规则就是比数值。

而本题“时间”和“能量”共同组成了某个东西，拟叫它“代价”好了，那么本题其实就是在求“代价”最短的最短路径。只不过“代价”有着不太一样的大小比较规则：对于两个表示“代价”的变量 a 和 b，

1）a“小于”b 定义为：a 的时间比 b 小，或 a 与 b 时间相等并且 a 的能量比 b 小；

2）a“等于”b 定义为：a 与 b 的时间、能量完全相等；

3）a“大于”b 定义为：a 的时间比 b 大，或 a 与 b 时间相等并且 a 的能量比 b 大；

因此，我们在每次“选点”时，就像要以时间、路程、花费等为标尺选出最小的一样，本题我们就是在选出“代价”最小的；在更新时，是在比较“代价”的大小，若以 v 为中介，能够使得 u 的“代价”更小，则更新 u。这也从另一个角度理解了以上代码实现。

同时，通过 C++ 的运算符重载功能，我们可以对这种结构体的大小比较进行定

义：

```
struct cost {
    int time ;
    int energy;
    // 定义 cost 类型对象的“<”比较规则
    bool operator < (const cost& other) const {
        return this->time < other.time ||
            (this->time == other.time && this->energy < other.energy);
    }

    // 定义两个代价相加规则
    cost operator + (const cost& other) const {
        cost ret;
        ret.time = this->time + other.time;
        ret.energy = this->energy + other.energy;
        return ret;
    }
};
cost cur_minc[N];   // 这一个数组就包含了 cur_mint 和 cur_mine 的功能
```

然后每次“选点”时：

```
cost min_cost;
min_cost.time = inf;
min_cost.energy = inf;
int v = -1;
for(int j=0; j<N; j++) {
    if(!solved[j] && cur_minc[j] < min_cost ) {
        min_cost = cur_minc[j];
        v = j;
    }
}
...
```

更新时：

```
for(int j=0; j<AdjList[v].size(); j++) {
    int u = AdjList[v][j].u;
    cost c;
    c.time = AdjList[v][j].time_cost;
    c.energy = AdjList[v][j].energy_cost;
    if(cur_minc[v] + c < cur_minc[u]) {
        cur_minc[u] = cur_minc[v] + c;
    }
```

```
9 }
```

这种运算符重载规则，可以在我们使用 C++ 标准库提供的排序函数 std::sort() 时来自定义一些排序规则，我们将在 5.5.3 节（第 115 页）中看到。

## 习题 5 参考答案

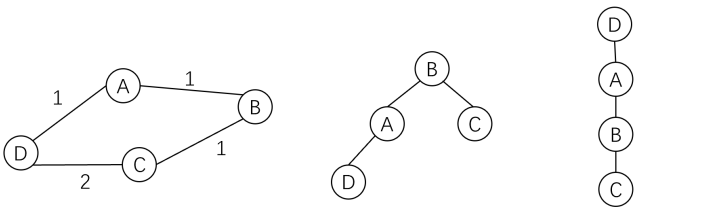
1. 选 A。极限思维，若所有边权值都相等，任选 $N-1$ 条无回路的边，构成的一定是最小生成树。C、D 显然正确。

2. 选 B。D 显然错误；考虑 B，无论 Prim 算法还是 Kruskal 算法，每次选边时，选择的都是距离或权值最小的一条边，因此若图中不存在权值相同的边，每次选择的边一定是唯一的，最终得到的最小生成树就是唯一的，因此 B 正确；对于 A 和 C 则很容易举出反例。

3. 依次选取权值为：1,2,3,5,6 的边。选了 1、2、3 后不能选 4，会有环。

4. 选 A 选项。顶点 B 和 C 此时已经处在同一个连通块了，根据 Kruakal 算法，不能选。

5. 选 A 选项。对于 II，考虑所有边权值相等的情况，若图中存在环则一定有些边不会出现在最小生成树中，错误；对于 III，考虑下图：



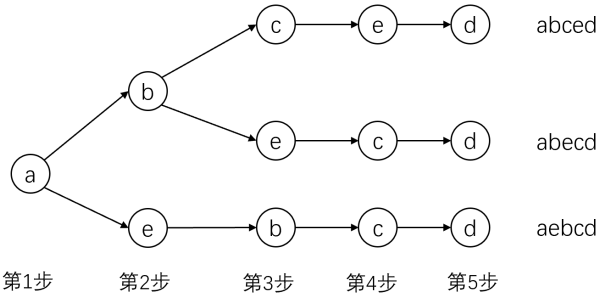

从B开始的最小生成树　从D开始的最小生成树

III 错误；对于 IV，考虑仅有两个顶点、一条边的图，其最小生成树只可能是它本身，两种算法得到的一定相同。

6. 选 C。容易得知 Kruskal 算法第二次选择的边一定是三条权值为 8 的边之一：(b,c)、(a,c)、(d,c)。而 Prim 算法从 d 开始，第一次选择的一定是 (d, a)，第二次可能选择 (a, c) 或 (d, c)，因此 (b,c)、(a,c)、(d,c) 中，Prim 算法一定不会选择的就是 (b,c)。

7. 选 D。$v_2$、$v_3$、$v_4$ 均存在一条到 $v_5$ 的路径，因此拓扑序列中不能排在 $v_5$ 的后面。

8. 选 C。由每一步可选的顶点构成的选择树如下所示：

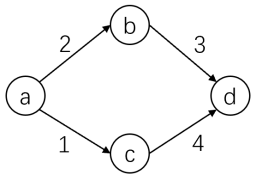



9. A、B、C、D 均错误，仅 E 正确。

对于 A、C，考虑本习题第 7 题的图的 A 选项的拓扑序列，$v_1$ 排在 $v_3$ 前面，但是不存在 $v_1$ 到 $v_3$ 的路径，因此 A、C 错误；对于 B、D，拓扑排序要求若 $v$ 到 $u$ 有一条路径，则 $u$ 一定排在 $v$ 的后面，因此 B、D 错误。E 显然正确。

10. 选 C。观察法很容易看出关键路径有 2 条，长度是 8。

11. 选 B。A 显然错误；AOE 网中关键路径可能存在多条，但增加关键活动的时间一定会增加工程的工期，C 选项说法有误。而减小关键活动的时间则不一定会减少工程的工期，如下图：




上下两条路都是关键路径，边 $(a,c)$ 是关键活动，但减少 $(a,c)$ 的时间不会减少工程的工期。因此 D 错误。

12. 选 C。通过观察法可以看出图中存在 3 条关键路径：bdcg、bdeh 和 bfh。仅 f 和 d 组合到一起能够构成三条路径的必经之路，因此加快 f 和 d 能使得三条路径同时变短。其余三种组合都至少有一条它们不能使之变短的关键路径。

13. 这显然是求最小生成树的问题，只不过已经选好了一些边，这些边不计入花费。我们根据 Kruskal 算法从剩下的边中进行选择即可。本题意在帮助大家复习并查集实现以及 Kruskal 算法。代码如下：

```
1 struct Road {
2     int u, v;
```

```
};
struct NewRoad {
    int u, v;   // 修建该道路后，将连通的两个城市的编号
    int weight; // 修建该道路需要的花费
};

// 提供给 std::sort() 的比较器
bool CmpNewRoad(NewRoad& r1, NewRoad& r2) {
    return r1.weight < r2.weight;
}

int Find(int father[], int x) {
    int ret = x;
    while(ret != father[ret]) {
        ret = father[ret];
    }
    int to_adjust = x;
    while(to_adjust != ret) {
        int fa = father[to_adjust];
        father[to_adjust] = ret;
        to_adjust = fa;
    }
    return ret;
}

int Union(int father[], int x, int y) {
    int x_id = Find(father, x);
    int y_id = Find(father, y);
    if(x_id != y_id) {
        father[x_id] = y_id;
        return 1;
    }
    return 0;
}

int minimumCost(
    int N,
    vector<Road>& existing_roads,
    vector<NewRoad> new_roads
) {
    int ret = 0;
    int father[N];
    for(int i=0; i<N; i++) {
        father[i] = i;
    }
    int edge_num = 0;
    // 根据尚存道路计算已有的边数，并合并这些道路连通的顶点
```

```
    for(int i=0; i<existing_roads.size(); i++) {
        Union(father, existing_roads[i].u, existing_roads[i].v);
        edge_num += 1;
    }
    // 按 CmpNewRoad 的比较规则对 new_roads 进行排序
    sort(new_roads.begin(), new_roads.end(), CmpNewRoad);

    // 按从小到大遍历边
    for(int i=0; i<new_roads.size(); i++) {
        int fau = Find(father, new_roads[i].u);
        int fav = Find(father, new_roads[i].v);
        if(fau != fav) {
            Union(father, new_roads[i].u, new_roads[i].v);
            ret += new_roads[i].weight;
            edge_num += 1;
        }
    }
    if(edge_num < N-1) {
        return -1;
    }
    return ret;

}
```

在尚存的旧道路保证不存在环的前提下，通过边数是否等于 $N-1$ 即可判断连通性。若尚存的旧道路有可能存在环，则可以通过并查集判断所有城市是否处于同一个连通块：

```
// 所有顶点处于同一个连通分量 <=> 所有顶点与 0 号顶点的连通分量编号相同
int root = Find(father, 0);
for(int i = 1; i < N; i++) {
    if(Find(father, i) != root) {
        return -1;
    }
}
```

14. 本题意在帮助大家练习拓扑排序的代码实现。没有什么难度，灵活处理输入数据即可。代码如下。

```
// v1、v2 的长度均为 M，用于表示任务的依赖关系：对于 0 <= i <= M，任务 v1[i] 必须在 v2[i] 之前完成。
vector<int> TopologicalSort(int N, int M, vector<int>& v1, vector<int>& v2) {
    vector<int> ret;
```

```
    vector<int> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    queue<int> q;
    // 根据任务的依赖关系建图
    for(int i=0; i<M; i++) {
        int u = v1[i];
        int v = v2[i];
        AdjList[u].push_back(v);
        in_deg[v] += 1;
    }
    // 入度为 0 的顶点入队
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
        }
    }
    while(!q.empty()) {
        int vertex = q.front();
        q.pop();
        ret.push_back(vertex);
        for(int i=0; i<AdjList[vertex].size(); i++) {
            int u = AdjList[vertex][i];
            in_deg[u] -= 1;
            if(in_deg[u] == 0) {
                q.push(u);
            }
        }
    }
    if(ret.size() < N) {
        return vector<int>();
    }
    return ret;
}
```

15. 需要注意的是，与求解关键路径长度不同，本题未必只有一个源点，但动态规划的思想仍然是成立的：我们依然可以根据拓扑序列的顺序，递推地从所有无前置任务的顶点推出各个任务的完成时间。具体来说，我们设置数组 max_len[]，max_len[i] 表示完成 i 的所有前置任务所需的时间。那么任务 i 的完成时间就是 max_len[i] + T[i]。

```cpp
// T[] 与 Pre[] 长度均为 N，T[i] 表示任务 i 的花费时间；Pre[i] 表示任务 i 的所有直接前置依赖
int MinTime(int N, int T[], vector<int> Pre[]) {
    struct AdjEdge {
        int u;
        int weight;
        // 如果你一路走到了这里，相信自己学会构造函数对你来说应该很简单了。
        AdjEdge(int u, int weight) : u(u), weight(weight) {}
    };
    vector<AdjEdge> AdjList[N];
    int in_deg[N];
    for(int i=0; i<N; i++) {
        in_deg[i] = 0;
    }
    // 遍历所有有向边，建立邻接表
    for(int i=0; i<N; i++) {
        vector<int> pres = Pre[i];
        for(int j=0; j<pres.size(); j++) {
            int u = pres[j];
            // 一条 u -> i 的有向边，其权值为 T[u]
            AdjList[u].push_back(AdjEdge(i, T[u]));
            in_deg[i] += 1;
        }
    }
    // 求解各顶点的最长路径长度（从任意源点到该顶点的路径长度中最长的那个）
    int max_len[N];
    // 数组初始化为 0
    memset(max_len, 0, sizeof(max_len));
    queue<int> q;
    for(int i=0; i<N; i++) {
        if(in_deg[i] == 0) {
            q.push(i);
            max_len[i] = 0;
        }
    }

    while(!q.empty()) {
        int u = q.front();
        q.pop();
        // 遍历 u 的所有邻接点 v
        for(int i=0; i<AdjList[u].size(); i++) {
            int v = AdjList[u][i].u;
            int weight = AdjList[u][i].weight;
            // 尝试更新 v 的 max_len[]
            max_len[v] = max(max_len[v], max_len[u] + weight);
            in_deg[v] -= 1;
            if(in_deg[v] == 0) {
```

```
                q.push(v);
            }
        }
    }
    /* 注意：顶点的最长路径长度表示的是，完成该顶点（任务）的所有前置任务
    所需的最长时间，因此最长路径长度再加上该任务所需时间才是该任务的最早
    完成时间 */
    int ret = 0;
    for(int i=0; i<N; i++) {
        ret = max(ret, max_len[i]+T[i]);
    }
    return ret;

}
```

事实上以上代码的图模型更接近 AOV 网，用顶点表示活动，而非 AOE 网。u 的邻接表中的权值可以理解为 u 的点权（活动需要的时间）。

动态规划的过程也可以直接设置数组 done[]，直接表示各个任务完成的最早时间，将数组初始化为各个任务所需的时间 T[i]，之后进行递推，这样是更简洁的：

```
for (int i=0; i<AdjList[u].size(); i++) {
    int v = AdjList[u][i];
    done[v] = max(done[v], done[u] + T[v]);
    in_deg[v] -= 1;
    if(in_deg[v] == 0) {
        q.push(v);
    }
}
```